

ADVANTEST[®]

ADVANTEST CORPORATION

ASX/U-51
Hybrid Controller
User's Manual

MANUAL NUMBER 8409598-08

Applicable OS
ASX/U-51

Trademarks

The following are trademarks and registered trademarks used in this manual.

- Advantest, ADVANsite, ATL, and ATLworks are trademarks of Advantest Corporation.
- Sun, Sun Microsystems, the Sun logo, Ultra, Solaris, and NFS are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.
- All SPARC trademarks are used under license of SPARC International, Inc. and are also trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries. Products bearing SPARC trademarks are based on an architecture developed by Sun Microsystems, Inc.
- IBM, PowerPC, and the PowerPC architecture are trademarks or registered trademarks of International Business Machines Corp.
- UNIX is a registered trademark in the United States and other countries, licensed exclusively through The Open Group.
- VxWorks, Wind River Systems, the Wind River Systems logo, wind, CrossWind, Tornado, and WindSh are trademarks or registered trademarks of Wind River Systems, Inc.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Revision History

Rev.	Date	Notes
01	May 31/01	
02	Oct 1/01	
03	Dec 25/01	
04	Jul 15/02	
05	Nov 1/02	
06	Nov 12/02	
07	May 15/03	
08	Jul 25/03	

Table of Contents

Preface	Preface-1
Requested Knowledge and Target Readers	Preface-1
Contents of This Manual	Preface-1
Tornado Manuals	Preface-2
Related Manuals	Preface-3
Typographic Convention	Convention-1
Typeface Conventions	Convention-1
1. Overview	1-1
1. 1 Introduction	1-1
1. 2 Operating Environment	1-3
1. 2. 1 Required Hardware	1-3
1. 2. 2 Required Software	1-3
1. 3 Before Using	1-5
1. 3. 1 Setup Host CPU Environment	1-5
1. 3. 1. 1 NFS Setup	1-5
1. 3. 1. 2 Tornado Environment Setup	1-6
2. Introduction to the Hybrid Controller	2-1
2. 1 System Outline	2-1
2. 2 Program Development: From Programming to Execution	2-4
2. 2. 1 Creating Simple Programs	2-4
2. 2. 2 Outline of the ATL Interface	2-6
2. 3 Library Functions	2-10
3. System Basis	3-1
3. 1 Controller Operating Modes	3-1
3. 2 Loading and Unloading C-program	3-2

3. 3	Program Execution and Control	3-3
3. 3. 1	Program Start-up	3-3
3. 3. 2	Program Termination	3-4
3. 3. 3	Termination Status	3-5
3. 3. 4	Releasing Resources	3-5
3. 4	ATL Interface	3-7
3. 4. 1	Program Configuration Method	3-7
3. 4. 2	ATL-separated Method	3-7
3. 4. 2. 1	Preparing for Device Measurement: Loading an ATL Test Plan Program	3-8
3. 4. 2. 2	Executing an ATL Program	3-8
3. 4. 3	ATL-mixed Method	3-11
3. 4. 3. 1	Overview	3-12
3. 4. 3. 2	Program Syntax	3-14
3. 4. 3. 3	Preparing Device Measurement	3-16
3. 4. 3. 4	Executing Embedded ATL Subroutines	3-16
3. 4. 3. 5	Executing an ATL External Subroutine Program	3-19
3. 4. 3. 6	The Scope of Variables Declared in an Embedded ATL Subroutine	3-19
3. 4. 3. 7	ATL Program Variable Area	3-19
3. 4. 4	Expressing Fixed-Bit Width Integers and Floating-Point Numbers with Unit	3-20
3. 4. 5	Passing Arguments to an ATL Subroutine	3-21
3. 4. 6	Data Exchange by System Variables	3-25
3. 5	Event Processing	3-28
3. 5. 1	Event Class	3-28
3. 5. 2	Event Handler	3-29
3. 5. 3	Synchronous Event Class	3-30
3. 5. 3. 1	Synchronous Event	3-30
3. 5. 3. 2	Synchronous Event Handler	3-30
3. 5. 4	Asynchronous Event Class	3-32
3. 5. 4. 1	Asynchronous Event	3-32
3. 5. 4. 2	Asynchronous Event Handler	3-33
3. 6	Implementation of Error Number errno	3-35
3. 7	Hardware Exception	3-36
3. 7. 1	Exception and Signal	3-36
3. 7. 2	Exception Processing	3-36
3. 7. 3	Exception Error Messages	3-37
3. 7. 4	Return from Exception	3-39
3. 8	File I/O	3-40
3. 8. 1	File	3-40
3. 8. 2	Standard I/O and VKT	3-40

3. 9	Added or Modified ATL Statements	3-41
3. 9. 1	Added ATL Statements	3-41
3. 9. 1. 1	RETURNC Statement	3-41
3. 9. 2	Modified ATL Statements	3-41
3. 9. 2. 1	GOTO PRO Statement	3-41
3. 9. 2. 2	GOTO CONTINUE Statement	3-41
3. 9. 2. 3	ONRTN Statement	3-42
3.10	Control Box and O Command	3-43
3.10. 1	Program Name and Test Number	3-43
3.10. 2	Test Mode	3-43
4.	/HYBRID - Setting the Controller Operating Mode	4-1
4. 1	Overview	4-1
4. 2	Syntax	4-2
4. 3	Function	4-4
4. 4	Interactive Setting	4-5
4. 5	Error Message	4-8
4. 5. 1	/HYBRID Error Messages and Causes	4-8
5.	/test_main - Program Execution	5-1
5. 1	Syntax	5-1
5. 2	Function	5-2
5. 3	Error Message	5-3
6.	Programming	6-1
6. 1	Header File	6-1
6. 2	Entry Point	6-2
6. 3	Data Initialization	6-3
6. 4	Recommending the Use of Function Prototypes	6-4
6. 5	Library Functions and Error Processing	6-6
6. 6	Problems Particular to VxWorks	6-7
6. 6. 1	Releasing Resources	6-7
6. 6. 2	The Scope of Symbols with External Linkage	6-7
6. 6. 3	Handling Uninitialized External Variables	6-8

6. 6. 4	Recommended Standard and Compatibility	6-8
6. 7	Program Example	6-9

7. C Program Development Environment 7-1

7. 1	C (C++) Compiler	7-1
7. 1. 1	Environment Variables Required for Execution	7-2
7. 1. 2	Compiling C (C++) Source Program	7-2
7. 1. 3	Atcc-specific Pragma	7-5
7. 1. 3. 1	#pragma Load - Dynamic Link for Library Module	7-5
7. 2	Linker	7-7
7. 3	Developing C++ Programs	7-9
7. 4	Automatic Compilation - make -	7-10
7. 4. 1	Outline of the make Command	7-10
7. 4. 2	Makefile Sample	7-11
7. 5	Tornado Development Tool	7-15
7. 5. 1	Starting Tornado	7-15
7. 5. 1. 1	Preparation	7-15
7. 5. 1. 2	Starting Tornado Registry	7-16
7. 5. 2	Launcher	7-17
7. 5. 2. 1	Starting and Terminating the Launcher	7-17
7. 5. 2. 2	Window Configuration	7-17
7. 5. 2. 3	Starting a Target Server	7-19
7. 5. 3	Tornado Online Manual	7-21
7. 5. 4	Debugging Using CrossWind	7-21
7. 5. 4. 1	Debugging Procedure	7-21
7. 5. 4. 2	Starting and Terminating CrossWind	7-23
7. 5. 4. 3	Outline of CrossWind Functions	7-23
7. 5. 4. 4	CrossWind Operating Procedure	7-29
7. 5. 5	Browser	7-29
7. 5. 6	WindSh	7-31
7. 6	Other Development Tools	7-32
7. 6. 1	Symbol Checker - atsymchk -	7-32
7. 6. 2	Tester Processor Reset - tpreset -	7-33

8. System Configuration 8-1

8. 1	Overview	8-1
8. 2	Tester Processor Configuration	8-2

8. 3	ADVANsite Network	8-6
8. 4	Implementation of VxWorks	8-9
8. 4. 1	VxWorks Configuration	8-9
8. 4. 2	Multitasking	8-10
8. 4. 3	System Tasks	8-11
8. 4. 4	Limitations: Restricted VxWorks Functions	8-12
8. 5	I/O System	8-15
8. 5. 1	Files and Devices	8-15
8. 5. 2	ASX File Device	8-15
8. 5. 3	NFS Devices	8-16
8. 5. 4	Non-NFS Network Devices	8-17
8. 5. 5	Controller Terminal (VKT) I/O	8-18
8. 5. 6	Default Path	8-18
8. 6	File System and Security	8-19
 Appendix 1.Using the Serial Interface		A1-1
A1. 1	Connecting External Devices	A1-1
A1. 2	Driver Specifications	A1-2
A1. 3	Programming	A1-2
 List of Figures		F-1
 List of Tables		T-1

Preface

Requested Knowledge and Target Readers

This manual is intended for users who are familiar with the C language. To get the most out of this manual, the users should at least be familiar with the following terminology:

Compiler, preprocessor, macro, function, function prototype, operator, storage class, scope, cast, pointer, array, structure, union, and floating-point constant.

Also, some contents of this manual require knowledge of UNIX, VxWorks, and Tornado. We recommend you to read the Tornado manuals to get better understanding.

➔ For more information on the Tornado manuals, refer to ["Tornado Manuals" on page Preface-2](#).

Contents of This Manual

This manual consists of the following chapters and appendixes.

1. ["Overview" on page 1-1](#)

This chapter provides information required to understand this manual and to use the hybrid controller correctly.

2. ["Introduction to the Hybrid Controller" on page 2-1](#)

This chapter provides a brief introduction to the hybrid controller for new users. After reading this chapter, the user will have an overview of the hybrid controller and will be able to create a basic program.

3. ["System Basis" on page 3-1](#)

This chapter explains the basics of the system, including theory of operation, basic system interface functions, hardware exceptions, file I/O, controller commands, and test modes.

4. ["/HYBRID - Setting the Controller Operating Mode" on page 4-1](#)

This chapter explains the /HYBRID utility program used to set the operating mode of the controller.

5. ["/test_main - Program Execution" on page 5-1](#)

This chapter explains the /test_main utility program used to execute a C program.

6. ["Programming" on page 6-1](#)

This chapter explains required information for programming, programming techniques, VxWorks-specific issues, and program compatibility.

[7. "C Program Development Environment" on page 7-1](#)

This chapter explains how to use the C compiler, linker, Tornado CrossWind debugger as the C program development environment, and also explains other commands for supporting this development environment.

[8. "System Configuration" on page 8-1](#)

This chapter explains the tester processor and environment processor from both hardware and software standpoints. The main topics are the hardware configuration of the tester processor and the implementation of its operating system, VxWorks. This chapter is intended for more advanced users.

[Appendix 1. "Using the Serial Interface" on page A1-1](#)

This chapter explains the specifications and use of the serial interfaces mounted on the tester processor.

➔ For more information on the hybrid controller library functions, refer to [ASX/U-51 Hybrid Controller Reference Manual\(No. : 8409600\)](#).

Tornado Manuals

Tornado manuals are provided in bound form or in hypertext HTML format.

The bound manuals are sold as "Tornado Manuals."

The hypertext manuals are contained in Tornado, and by installing Tornado, they can be opened as online manuals with a Web browser. The online manuals include all Tornado and VxWorks manuals.

The Tornado manuals are listed below. Manuals marked with an asterisk (*) are only provided in hypertext form.

"Tornado Release Notes"

"Tornado Installation Guide"

"Tornado User's Guide"

"Tornado API Programmer's Guide"

"Tornado API Reference*"

"VxWorks Programmer's Guide"

"VxWorks Network Programmer's Guide"

"VxWorks Reference Manual"

"GNU ToolKit User's Guide"

"GNU Make User's Guide"

"GDB User's Guide"

➔ For more information on these manuals, refer to "Tornado User's Guide".

The manual names listed above are used in this manual. Note that none of the Tornado manuals are listed in ["Related Manuals" on page Preface-3](#) in this manual.

Related Manuals

ASX/U-51 Hybrid Controller Reference Manual(No. : 8409600)

ADVANsite Test System Software Installation/System Start User's Manual VOL1 Basic OS Installation(No. : 8227728)

ADVANsite Test System Software Installation/System Start Users Manual VOL2 Tester Environment Software Installation(No. : 8227729)

ATL-51 Test Plan Program Reference Manual(No. : 8431498)

XATL/U-51 Cross Compiler User's Manual(No. : 8256237)

T5500 Series Tester Utility Program Reference Manual(No. : 8256295)

T5500 Series Controller Commands Reference Manual(No. : 8256297)

T5500 Series Memory Test System Control Box/Trigger Panel User's Manual(No. : 8256298)

Error Code Table Reference Manual(No. : 8141091)

ATLworks Tool User's Manual(No. : 8311174)

Typographic Convention

Typeface Conventions

The typefaces and symbols listed in [Table C-1](#) have special meanings in this manual, as explained below.

Table C-1 Notation

Item	Example
Keyboard input	<code>/HYBRID/1/S sample.o</code>
Simultaneous key input	<code>[CTRL]-[C], [CTRL]-[X]</code>
Name of reference manual	"Tool Reference Manual"
Name of chapter or section to be referenced	1. "Overview"
Variable (replaced with a specific name or value)	<i>variable</i>
File or path name	<code>filename</code>
Graphic object of button, or menu name	[File] menu

If code or keyboard input text exceeds the line length for the page (or surrounding frame), a backslash '\' is used to indicate that the text is continued.

(Example)

```
% ccppc -ansi -O2 -Wall -fno-stdinc -fno-common -mlongcall \
  -DCPU=PPC604 -I$(WIND_BASE)/target/h -I$(AT_ASYSYSTEM) -c sample.c
```

The actual text does not reflect the backslash '\' and line feed in this example.

[Table C-2](#) lists the terms used to give specific meanings or to indicate specific purposes in this manual.

Table C-2 Terms with Specific Meanings or Purposes

Term	Meaning or Purpose
Hybrid controller	This means the tester controller having the functions explained in this manual. This term is used to distinguish the new tester controller from the older tester controller used for ASX/U-51 REV 6.00 or earlier.
Controller	Tester controller software
Tester processor	A tester processor mounted on a test system. This manual refers to TP5 or TP6.
Host system Host	A workstation connected to an ADVANsite test system. The host means the workstation that provides various services for the tester processor.

Term	Meaning or Purpose
Target system Target	The target system means the tester processor mounted on a test system. This term is used to distinguish a tester processor from a host system.
C language	C language which complies with ANSI standards (ANSI/ISO/IEC 9899)
C++ language	C++ language which complies with ANSI standards (ISO/IEC 14882)
C program C++ program ATL program	A program written in the C, C++, or ATL language. These terms are used when it is not necessary to distinguish between source and object programs or when the program type can be determined from the context. Unless stated otherwise, "C program" can also refer to a C++ program.
POSIX	Portable Operating System Interface. IEEE is the main force behind the standardization of the system interface. (ISO/IEC 9945: ANSI/IEEE Std 1003.1)

1. Overview

This manual explains functions of the ASX/U-51 hybrid controller.

The hybrid controller is a tester controller that allow the user to create a device test program for a memory test system that uses both the C and C++ languages in addition to the ATL language.

1. 1 Introduction

Advantest memory test systems have traditionally provided the unique ATL language as system interfaces because control instructions can easily be programmed for the test systems. To date, users have built up enormous program and technical assets. Advantest believes that protecting these assets are the system supplier's responsibility.

At the same time, the number of test engineers who have learned to use the C and C++ languages in schools have been increasing by the wide spread use of computers. It is true that the ATL language is being an obstacle for such engineers to develop test programs.

In view of these circumstances, a hybrid controller has been developed to satisfy new requirements without altering the environment of the ASX/U-51 system software. Users familiar with C or C++ can develop device test programs by learning the tester control instructions and some a few syntax of the ATL language.

The hybrid controller has the following features:

- ASX/U-51 system software

The hybrid controller incorporates the entire environment of the ASX/U-51 system software without any adverse effects. Programs created using the existing ATL language can be executed without making any changes.

- C-Language programming

The hybrid controller can execute test programs created with either C or C++. Since C programs are compiled into instructions which are directly executable on the tester processor, such instructions which do not contain I/Os can be executed at high speed comparing to ATL programs which are executed using interpreter system.

- Describing the ATL statement in a C program (ASX/U-51 REV 6.02 and later)

The ATL statement can be described in a C program by using the pragma function (#pragma) that is a function of C and C++ languages. This integrates the C and C++ languages and the ATL language. The device test program can consist of only the source program in the C or C++ language.

- ATL interface

By using the ATL interface, processes can be executed in the ATL language from the C program.

- Sharing System Variables

Controller system variables are shared by the ATL and C programs. Those values can be referenced and changed from the C program. The hybrid controller also provides new system variables to exchange data with ATL programs.

- Tester access library

Using the library functions of the hybrid controller allows C programs to acquire and process device test results. In addition, some test conditions can be set from C programs. This allows the user to incorporate device test algorithms into a C program.

- Event processing

By setting a callback function for an event that occurs during device testing or ATL program execution, a C program can process the event. When MRA4 is also used, MRA4 events can be processed in a C program asynchronously.

1. 2 Operating Environment

This section explains the hardware, software configurations, and host environments required to use the hybrid controller.

1. 2. 1 Required Hardware

Table 1-1 shows the required hardware configuration.

Table 1-1 Required Hardware Configuration

Test System	Required Hardware Option
T5593	None
T5592	128 MB of additional memory for tester processor
T5586 and T5585	128 MB of additional memory for tester processor
T5377, T5376 and T5375	None
T5371	128 MB of additional memory for tester processor

- Test system

ADVANSite test systems which are capable of using the hybrid controller.

The test system tester processor should be TP5 or TP6. The table above lists test systems that are equipped with these tester processors.

- Hardware option

Refers to optional hardware that must be added to the test system to enable the use of the hybrid controller.

For more information on the optional hardware installation, contact the nearest Advantest sales office.

1. 2. 2 Required Software

This section lists the required software.

➔ For information on how to install the software, refer to the installation guide provided with the software.

- ASX/U-51 REV 6.00 or later

System software for T5500 series memory test systems

 Tip

The license for the hybrid controller should be installed before you use the hybrid controller.

➔ For more information, refer to [ADVANSite Test System Software Installation/System Start Users Manual VOL2 Tester Environment Software Installation\(No. : 8227729\)](#).

- Tornado 2

Software from Wind River Systems, Inc. that provides a cross-development environment for the C and C++ languages. This software is bundled to ASX/U-51 system software of REV 6.00 and later and when the SYSTEST51SU system software is installed, this software is also automatically installed.

- XATL/U-51 cross compiler

Cross compiler for the ATL language. This software is optional and must be purchased separately from the system software. Required compiler versions, which depend on the program configuration method, are as follows.

Table 1-2 Required XATL/U-51 Cross Compiler

Program configuration method	XATL/U-51 cross compiler
ATL-separated method	XATL/U-51(Product: PXTL51-00S***) REV 6.00 and later XATL/U-51(Product: PXTL51-36S***) REV 3.00 and later
ATL-mixed method	XATL/U-51(Product: PXTL51-00S***) REV 6.02 and later XATL/U-51(Product: PXTL51-36S***) REV 3.01 and later

➔ For more information on the program configuration method, refer to [3. 4. 1 "Program Configuration Method" on page 3-7](#).

1. 3 Before Using

1. 3. 1 Setup Host CPU Environment

1. 3. 1. 1 NFS Setup

The hybrid controller usually uses NFS to load programs in the tester processor. Therefore, file systems containing the programs should be exported to the tester processor. The following section describes how to export file systems.

1) Login to the system with the root account.

```
% su
password: *****
#
```

2) Open the `/etc/dfs/dfstab` file, then edit the file to add the following statement:

```
share -F nfs -o rw=asxvxtarget pathname
```

Specify the path name of the file system being exported for *pathname* in the above statement.

Example: The following example is used to export `/export/home`.

```
share -F nfs -o rw=asxvxtarget /export/home
```

3) If the NFS server has already been activated, execute the following command to export the file systems:

```
# shareall
```

To check whether the NFS server is active or not, execute the following command:

```
# ps -e | grep nfsd
```

If the following message is returned, the NFS server is currently active:

```
296 ?          0:00 nfsd
```

- 4) If the NFS server is not active, execute the following command to start the NFS server then export the file systems.

```
# /etc/init.d/nfs.server start
```

The NFS setup procedure is now completed.

Once this procedure is executed, the directories specified in the `/etc/dfs/dfstab` file are automatically exported to the tester processor when the host system is restarted. Therefore, it is not necessary to make this setting every time you restart the host system.

➔ For more information, refer to `share(1M)` and `share_nfs(1M)` in the "Solaris Online Manual."

1. 3. 1. 2 Tornado Environment Setup

Before you use Tornado software, you need to set environment variables described in this section. If these settings are specified in the startup script of the login shell, it is not necessary to set them whenever you use Tornado software. The setup method described in this section is for the C shell. If you use another shell, set the following environment variables using the appropriate method for the shell:

- **WIND_BASE**

Specify the directory in which Tornado is installed. Usually, the directory to be installed is `/export/home/asxvx/tornado`.

```
% setenv WIND_BASE /export/home/asxvx/tornado
```

- **WIND_HOST_TYPE**

Specify the host environment of Tornado. This should always be set to `sun4-solaris2`.

```
% setenv WIND_HOST_TYPE sun4-solaris2
```

- **WIND_REGISTRY**

Specify the ADVANsite system host name as the host used to execute the Tornado registry.

```
% setenv WIND_REGISTRY `/usr/bin/uname -n`
```

- **PATH**

Add the following directory to the command search path of the shell:

`$WIND_BASE /host/$WIND_HOST_TYPE/bin`

```
% setenv PATH ${WIND_BASE}/host/${WIND_HOST_TYPE}/bin:${PATH}
```

- LD_LIBRARY_PATH

Add the following directories to the dynamic link library search path.

1. \$WIND_BASE /host/\$WIND_HOST_TYPE/lib
2. /usr/dt/lib
3. /usr/openwin/lib

```
% setenv LD_LIBRARY_PATH ${WIND_BASE}/host/${WIND_HOST_TYPE}/lib:  
/usr/dt/lib:/usr/openwin/lib:${LD_LIBRARY_PATH}
```

2. Introduction to the Hybrid Controller

This chapter explains the functions of the hybrid controller for a new user.

The general procedure for program development and an outline of the system operations are presented first. In later sections, the practical development procedures are shown using program examples to understand the principle of the program operation intuitively.

2. 1 System Outline

Test programs written in the C or C++ language as well as the ATL language can be executed using the hybrid controller. The controller has two operating modes to specify either ATL language or the C or C++ language as the main programming language. It operates according to the selected mode.

- ATL mode

This operating mode uses the ATL test plan program as the main program.

➔ For more information, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

- Hybrid mode

This operating mode uses the C program as the main program.

The hybrid mode executes programs whose device test algorithms were written in the C or C++ language. A device test is performed while the C program calls ATL processes in which tester control instructions such as pin conditions and clock settings are described. The test results can be acquired and processed by the C program.

Accordingly, ATL processes can simply consist of tester control instructions only.

ATL processes can be described in either of the following ways:

- ATL-separated method

In this method, ATL processes are described in the ATL test plan program. The C program and the ATL program are compiled separately and separate object files are created.

- ATL-mixed method (ASX/U-51 REV 6.02 and later)

In this method, ATL processes are described in the C program. When the C program is compiled, ATL object codes are output to the C object file. Therefore, only one object file which configures the program is created.

As an introduction, program development which uses the ATL-separated method is described below.

➔ For more information on the ATL-mixed method, refer to [3. 4. 3 "ATL-mixed Method" on page 3-11](#).

[Table 2-1](#) shows the procedure which is used to develop a program that runs in the hybrid controller.

Table 2-1 Program Development Procedure with the ATL-Separated Method

No.	Item	Tools
(1)	Creating C source programs	Any text editor
(2)	Creating ATL source programs	Any text editor
(3)	Compiling and linking C source programs	C compiler and linker
(4)	Compiling ATL source programs	XATL/U-51 Cross Compiler
(5)	Starting the controller	Execute the test command on the host system
(6)	Setting the hybrid mode	/HYBRID utility program
(7)	Executing the program	Space key, C-BOX start switch, start instruction from the prober or handler /test_main utility program
(8)	Debugging	ATLworks, Tornado CrossWind debugger, ATL debugger, or other evaluation tools

Figure 2-1 shows the flow from the creation of a program to its execution.

Operational flows used to create and execute C and ATL programs are shown separately for each type of program:

- C program creation and execution

C or C++ source program is compiled into an object program that can be executed directly on the tester processor, using the C compiler for the hybrid controller. The object program is transferred from the host system disk to the tester processor and executed.

The entry point of the C program is the test_main() function. Program processing starts from this function. When this function ends, the program execution completes.

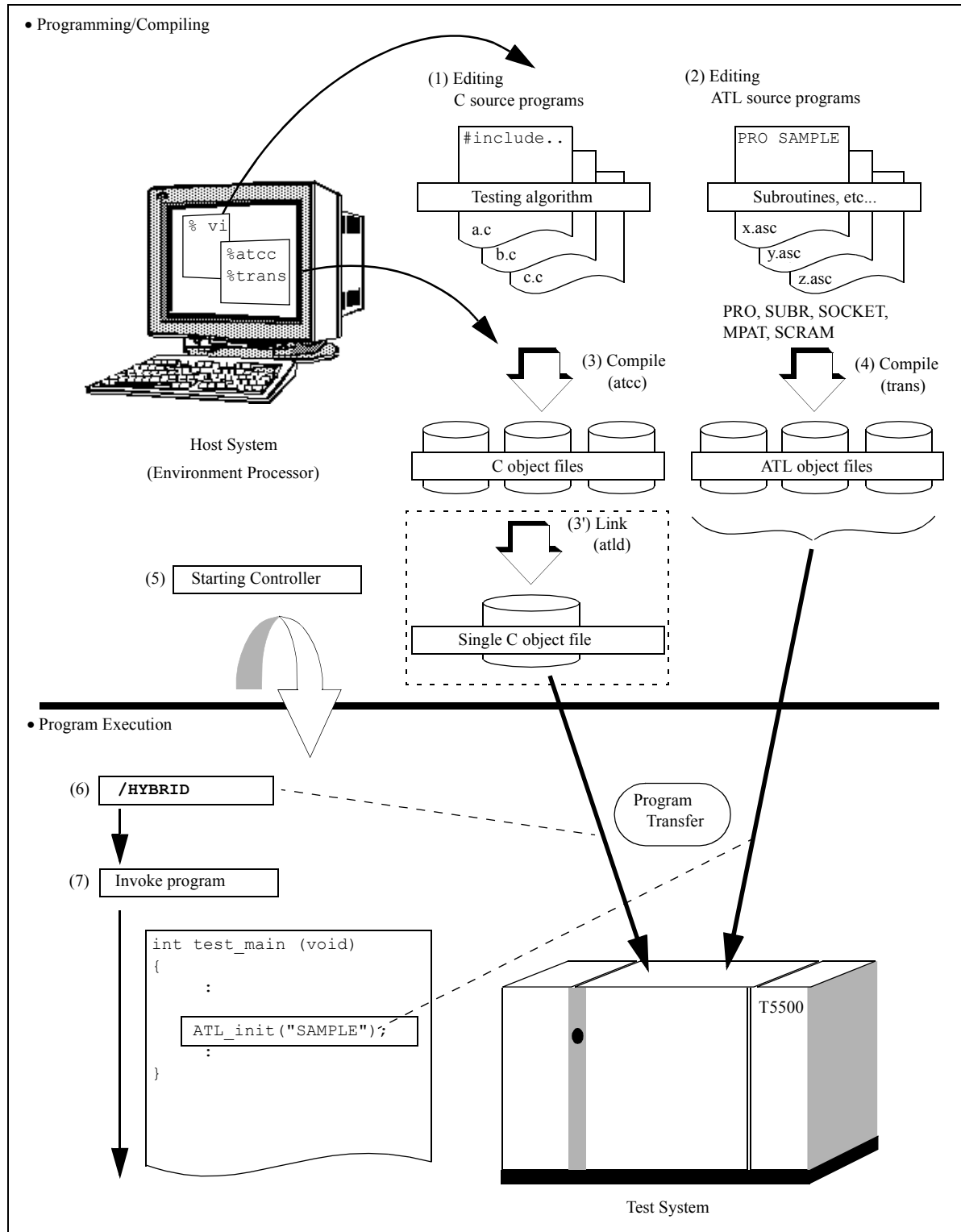
- ATL program creation and execution

The XATL/U-51 Cross Compiler is used to compile a ATL language source program into an ATL object program. The ATL object program is transferred to the tester processor when the library function ATL_init() is called from the C program. The transferred ATL program is executed when it is called from the C program with the library function.

As explained above, the general program flow is controlled by the C program.

➔ For more information on items about ATL, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#), [XATL/U-51 Cross Compiler User's Manual\(No. : 8256237\)](#), [T5500 Series Tester Utility Program Reference Manual\(No. : 8256295\)](#), and [T5500 Series Memory Test System Control Box/Trigger Panel User's Manual\(No. : 8256298\)](#).

Figure 2-1 ATL-Separated Method Operation Conceptual Diagram



2. 2 Program Development: From Programming to Execution

This subsection explains the program development procedure shown in the preceding subsection using specific examples. It also explains the Hybrid Controller functions using an actual program as an example.

The program example described here is simple one and have not been checked for errors. If you intend to use a function shown in the following examples in an actual program, make sure to check the program for errors.

2. 2. 1 Creating Simple Programs

The first step is to create and execute a program that displays "Hello world." to the controller terminal (VKT).

The following example shows the source code for the program `hello.c`:

◆ Program Development Procedure

```
1: #include "AT/hybrid.h"
2:
3: int test_main(void)
4: {
5:     AT_printf("Hello world.\n");
6:     return(0);
7: }
```

This example contains several important elements.

The header file "AT/hybrid.h" in line 1 must be included in all programs that are used with the hybrid controller. The header file provided by the hybrid controller is located in the system directory pointed to by the `AT_ASYSTEM` environment variable.

The `test_main()` function in line 3 is the program entry point. Processing always starts with this function. This is true for all hybrid controller programs.

Line 5 calls the library function `AT_printf()`, which displays the text on the VKT. This function receives arguments in the same format as `printf()` of the C standard library and produces output on the VKT. Although `printf()` is a print function for standard output, note that the standard output is not assigned to the VKT in the hybrid controller. This is also true for standard input and standard error output.

➔ For more information, refer to [3. "System Basis" on page 3-1](#).

Once the source program `hello.c` has been completed, compile the source program into code that can be executed on the tester processor. Execute the following command on the host system to compile the source program.

```
% atcc -c hello.c
```

This section makes no mention of the compiler options.

➔ For more information, refer to [7. "C Program Development Environment" on page 7-1](#).

Compiling creates an object file named `hello.o`.

Tip

To use the C compiler, you must make sure that the environment variables to be used by the compiler for referencing are set correctly.

➔ For more information, refer to [7. "C Program Development Environment" on page 7-1](#).

This completes the work which is done on the host system. Next, start the controller with the test command and execute the program. The program can be executed by entering commands from the VKT using the following example.

```
% test

CONTROLLER READY

/HYBRID/1/E hello.o

H>(Press the space bar.)
Hello world.

H>(Press the ESC key.)
%
```

The `/HYBRID` utility program is used to set the operation mode of the controller and transfers the specified object file to the tester processor. The '1/E' switch sets Station 1 to the hybrid mode. In the hybrid mode, the controller displays the **[H>]** prompt on the VKT. When the '1/D' switch is specified, Station 1 returns to the ATL mode. The `/HYBRID` utility program also provides a user interface to set conditions interactively.

➔ For more information, refer to [4. "/HYBRID - Setting the Controller Operating Mode" on page 4-1](#).

There are several ways to execute a program, in the example above the program is executed by using the VKT space bar. When the program execution starts, the expected output will be displayed on the VKT. When the program ends, the controller displays the prompt again and waits for command input.

When the ESC key is pressed, the controller terminates.

➔ For more information on how to operate the controller, refer to [T5500 Series Controller Commands Reference Manual\(No. : 8256297\)](#).

2. 2. 2 Outline of the ATL Interface

The ATL interface is used to link an ATL program to the C program to execute the ATL program while executing the C program. This function allows you to process the ATL program as follows:

- Executing an internal subroutine in the ATL test plan program
If a label name is specified, an internal subroutine in the ATL test plan program can be executed. Arguments can then be passed to the internal subroutine that was called.
- Executing a TEST statement in the ATL test plan program
A TEST statement in the ATL test plan program can be executed by specifying the test number.
- Calling an ATL external subroutine program
An ATL external subroutine program can be called from the C program. Arguments can then be passed to the ATL external subroutine program.
- Exchanging data between a C program and an ATL program
Data can be exchanged between a C program and an ATL program through the controller system variables.
- Processing events during ATL program execution
If a runtime error or failure occurs during execution of an ATL program, a user-defined function in a C program can take over the processing. In other words, a callback function can be set for execution if an event occurs.

The internal subroutine is executed as follows:

To execute the ATL test plan program, it is necessary to load the program into the tester processor first. It is done using an ATL_init() function. This function also fetches socket program definitions to prepare the test system for device measurements. When the ATL test plan program is loaded, its internal subroutine is called with the ATL_exec() function and the subroutine can be executed. In general, there are almost no restrictions for executing called subroutine except some items. The ATL_exec() function can also execute a TEST statement and call an external subroutine program.

"[Example of Using the ATL Interface](#)" on page 2-7 shows a program example that outputs "Hello world." from C program hello2.c and "HELLO WORLD." from ATL program hello2.asc.

◆ Example of Using the ATL Interface

C program: hello2.c

```
1: #include "AT/hybrid.h"
2:
3: int test_main(void)
4: {
5:     AT_printf("Hello world.\n");
6:
7:     ATL_init("HELLO2");
8:     ATL_exec(ATL_GOSUB, "DOIT", 0);
9:
10:    return(0);
11: }
```

ATL program: hello2.asc

```
1: PRO HELLO2
2:
3: DOIT:
4:     WRITE "HELLO WORLD.", /
5:     GOTO CONTINUE
6:
7: END
```

The first five lines in the C program is the same as the program shown in ["Program Development Procedure" on page 2-4](#). On line 7, PRO HELLO2 is loaded into the tester processor using ATL_init(). Specify the program name given after the title PRO in the ATL test plan program as the argument for this function.

On line 8, internal subroutine DOIT of the ATL program is called with the ATL_exec() function. The first argument ATL_GOSUB of the function is the command for execution of the internal subroutine. The second argument is the label name of the internal subroutine. This function returns to the caller by the GOTO CONTINUE statement in the ATL program.

Compile the programs created. The C program is compiled in the same way as the program shown in ["Program Development Procedure" on page 2-4](#). The object file named hello2.o is created as a result of compilation.

```
% atcc -c hello2.c
```

The XATL/U-51 Cross Compiler is used to compile an ATL program.

```
% trans hello2.asc
```

 Tip

The ATL program, which is used in the ATL-separated method, must be compiled by using one of the XATL/U-51 cross compilers shown below:

*XATL/U-51 (Product: PXTL51-00S***) REV 6.00 and later*

*XATL/U-51 (Product: PXTL51-36S***) REV 3.00 and later*

Start the controller and enter the following commands to execute the program so that you can get the expected output.

```
% test

CONTROLLER READY

/HYBRID/1/E hello2.o

H>(Press the space key.)
Hello world.
HELLO WORLD.

H>(Press the ESC key.)
%
```

Up to now, the main program development processes for the hybrid controller have been explained. However, program debugging, which is one of the most important processes in program development, is not explained yet. However, a practical device test program in general is much larger and more complicated than the example shown here, so that debug process using a debugger is needed. The hybrid controller uses ATLworks or the Tornado CrossWind debugger to debug C programs. These programs are GUI-based tools. C programs that are executed on the tester processor can be debugged on the host system using one of those tools. To debug ATL programs, use ATLworks or the ATL debugger.

ATLworks is capable of debugging programs efficiently because it can automatically identify whether the target program is a C program or an ATL program and can debug the program in the same window regardless of the program type. If you do not use ATLworks, the hybrid controller requires using different debuggers for C and ATL programs.

Table 2-2 Debugging Environment of the Hybrid Controller

Program Type	Debugger and Tool
C program	ATLworks, Tornado CrossWind debugger
ATL program	ATLworks, ATL debugger

2. 2 Program Development: From Programming to Execution

These debugging tools are not explained in this section.

- ➔ For more information on ATLworks, refer to [ATLworks Tool User's Manual\(No. : 8311174\)](#).
- ➔ For more information of debuggers that use Tornado CrossWind, refer to [7. "C Program Development Environment" on page 7-1](#).
- ➔ For more information on the ATL debugger, refer to [T5500 Series Tester Utility Program Reference Manual\(No. : 8256295\)](#).

2. 3 Library Functions

The hybrid controller provides programming interfaces to each opened functions as the library functions.

The library functions of the hybrid controller consist of not only the ATL interface and event processing functions as the basic system interface but also various other functions which are used to access the tester. In addition, the C programming environment uses the ANSI C standard library and POSIX-compliant interfaces to enable highly portable programming.

When these library functions are combined, a user C program can be used to develop a device test program that does not contain any control structures in the ATL program side. You can use a C program to handle data processing using test results (i.e. categorizing failed devices, sort them to Category N, and remove them from the target devices to be re-tested). As a result, an ATL program can be created mainly from statements for setting the tester hardware (power supply, pin conditions, clock, ALPG, and fail memory). The ATL language is best suited for this purpose and keeps programming simple.

Because a C or C++ language programmer needs to master only a minimum number of the ATL formats and statements, the programmer is freed from the burden of mastering the entire ATL language. Since the ATL program side handles the hardware-dependent portions of the tester, the C program has high compatibility at the source level.

This manual does not explain the library functions in detail.

➔ For more information on the library functions, refer to [ASX/U-51 Hybrid Controller Reference Manual\(No. : 8409600\)](#).

3. System Basis

This chapter explains the theory of operation, program execution control, and basic system interface functions of the hybrid controller in relation to VxWorks.

VxWorks is a real-time operating system with a multi-tasking function, which is used to control the tester processor. To understand some of the contents of this chapter, a basic understanding of VxWorks OS is required. For further understanding of this chapter, refer to ["VxWorks Programmer's Guide"](#).

3. 1 Controller Operating Modes

The hybrid controller can execute either an ATL test plan program or a C program as a main program. The operating mode of the controller specifies the method adapted by the controller to control the program execution based on the main program type. The controller supports the following two modes:

- ATL mode
When the ATL mode is selected, the ATL test plan program is used as the main program. In this mode, the hybrid controller function is unavailable and the operating environment earlier than ASX/U-51 REV 6.00 is set.
- Hybrid mode
Aimed at the C program as the main program. The hybrid controller functions can be used in this mode.

Either operating mode can be set for each test station. A program is executed according to the operating mode of the corresponding test station. The operating mode is set using the /HYBRID utility program.

➔ For more information, refer to [4. "/HYBRID - Setting the Controller Operating Mode" on page 4-1](#).

When the current station^{*1} is in hybrid mode, the 'H>' prompt is displayed on the VKT.

^{*1} The current station refers to the station used by the most recent program. If no programs have been executed, station 1 is treated as the current station.

Tip

You cannot execute an ATL test plan program using the /PRO utility program at a station in the hybrid mode.

3. 2 Loading and Unloading C-program

To execute a C program, it is necessary transfer the target C program from the host system disk to the tester processor memory. This process is called program loading. A C program is an object file generated by compiling a C or C++ source program or by linking object files generated in this way.

➔ For more information on how to compile the source program, refer to [7. "C Program Development Environment" on page 7-1](#).

Tip

The hybrid controller does not handle an executable object file that has been completely link-edited (with no unresolved symbols), such as an UNIX program. Instead, it handles a relocatable object file that is partially link-edited. Unresolved symbols with external linkages are resolved when the file is loaded.

When the hybrid mode is selected using the /HYBRID utility program, the C program name (object file name) to be loaded can be specified. The C program specified in the /HYBRID utility program is unconditionally loaded by the utility program. When program execution starts, the controller exclusively loads C program according to the setting of the station that issued the program execution start instruction. In other words, if the station that issued the instruction is in hybrid mode and the C program is not loaded yet for the station, the hybrid controller unloads the currently loaded program and loads a new C program for the station.

Tip

The license for the hybrid controller should be installed before you load C programs.

➔ For more information, refer to [ADVAnsite Test System Software Installation/System Start Users Manual VOL2 Tester Environment Software Installation\(No. : 8227729\)](#).

To update the C program loaded on the tester processor when the C program is recompiled and updated on the host system, the loaded program must be unloaded first. The C program can be unloaded in either of the following ways:

- /HYBRID utility program
If there is no error, the /HYBRID utility program unconditionally unloads the loaded C program.
- U command
The loaded C program is unloaded by selecting 1 (CLEAR SAME AS (2,4)) or 2 (CLEAR VARIABLE & PROGRAM AREA) with the U command (controller command).

3. 3 Program Execution and Control

This section explains start-up and termination of a program, which are the most basic functions in the execution of a C program.

3. 3. 1 Program Start-up

In the hybrid mode, the hybrid controller executes a program when a program execution start instruction is issued to the test system by one of the following means:

- Pressing the space bar on the VKT
- Sending a start request (from a handler or prober) to the GPIB interface of the test system
- Pressing the START button on the control box

A program can also be executed by using the /test_main utility.

➔ For more information on this utility, refer to [5. "/test_main - Program Execution" on page 5-1](#).

On receiving a program execution start instruction, the controller loads the program (if it has not been loaded yet for the station from which the instruction is issued) to the tester processor and starts executing the program. Program processing starts from the beginning of the test_main() function. This function must be defined in a user program in the following format.

```
int test_main(void);
```

No arguments are passed to the test_main() function. The function must return int. When program execution starts, a runtime error occurs if one of the following conditions is true:

- Required memory has not been added.
- The Ethernet connection between the tester processor and host system is faulty.
- The test_main() function has not been defined

If a hardware configuration error occurs, check the test system configuration. For more information, refer to [1. 2. 1 "Required Hardware" on page 1-3](#).

From the standpoint of VxWorks, a user program is executed by the tUsrMain task whose entry point is the test_main() function. All user programs start from this task. This task is called the main task of the user because all tasks terminate when this task ends during multitask processing. When a program execution start instruction is received, the controller activates the user's main task and passes the tester processor control to the task. [Table 3-1](#) shows the configuration of the task tUsrMain.

Table 3-1 Configuration of the TUsrMain Task

Item	Set Value
Task name	tUsrMain()
Entry point	test_main()
Task option	VX_FP_TASK
Task priority	120
Stack size	32 MB

3. 3. 2 Program Termination

A user program terminates in any of the following cases:

- The test_main() function returns.
- exit() or abort() is called.
- An END or STOP statement is executed in an ATL program.

The return value of the test_main() function or the value passed to exit() means the program termination status. When abort() is used, the termination status is set to a value other than 0. When the ATL program is terminated with an END or STOP statement, the termination status is set to 0. The value of this termination status affects the next program execution. The meaning of the termination status is explained in [3. 3. 3 "Termination Status" on page 3-5](#). If one of the above methods is used to terminate the program without setting a termination status, the post-processing is not performed according to the termination status described later.

A user program is forcibly terminated in the following cases. The termination status is not defined when a program is forcibly terminated.

- [CTRL]-[C] input from VKT.
- The [RESET] switch in the control box (C-BOX) is pressed.
- The tUsrMain task is deleted using a system function of VxWorks.
- The \$R command is executed on the ATL debugger.

If the tUsrMain task is deleted using the VxWorks system function, an error occurs, and the message below is displayed on VKT.

```
Error 7:
Illegal operation: task tUsrMain was deleted.
```

When a user program terminates, the controller performs the following processing:

- 1) All user-class tasks (priority of 110 or higher) are terminated.

➔ For more information on the task classes, refer to [8. 4 "Implementation of VxWorks" on page 8-9](#).

- 2) A user-defined function registered by AT_atexit() is called.
→ For more information, refer to [3. 3. 4 "Releasing Resources" on page 3-5](#).
- 3) Frees all memory spaces allocated by the hybrid controller library functions.
- 4) Closes files opened for an ASX file device.
- 5) If the termination status is defined, the processing described in the next section is executed.

3. 3. 3 Termination Status

The controller processes according to the termination status explained in [Table 3-2](#).

Table 3-2 Meanings of Termination Statuses

Termination Status	Explanation
0	Normal termination If the conditions below are all satisfied, the controller updates each counter, controls the handler or prober, and executes test mode processing^{*1}. If the conditions are not satisfied, the controller performs none of the above. 1) No runtime errors occurred during the execution of ATL program. 2) The runtime stack for the C program did not overrun.
Other than 0	Abnormal termination The controller does not update counters, control the handler or prober, nor execute test mode processing.

**1 If the program is terminated by an END or STOP statement written in the ATL external subroutine, none of the above processes are executed.*

If the runtime stack overruns, the following message is displayed on the VKT and an error occurs.

```
Error 6
stack frame was corrupted or overrun.
```

If the termination status is not 0, the following message is displayed on the VKT and an error occurs.
XXXX is a decimal value representing the termination status.

```
Error: 8
Program terminated with status: XXXX
```

3. 3. 4 Releasing Resources

As explained in [3. 3. 1 "Program Start-up" on page 3-3](#), the user program terminates when the main task ends. When the user program terminates, the controller automatically releases system resources (such as memory and files) allocated using the hybrid controller library functions. However, the system resources allocated using library functions provided by VxWorks are not automati-

cally released. If the system resources are only reserved but not released, not only the user program but the system as well will be disabled, since the resources are limited. As a result, user programs are responsible for releasing resources reserved using the library functions of VxWorks.

➔ For more information on the system resources, refer to [6. 6. 1 "Releasing Resources" on page 6-7](#).

For this reason, the controller provides a mechanism to call a user-defined function when the program terminates. The `AT_atexit()` function is the interface for this function. By using `AT_atexit()`, the user can register a single, arbitrary function to be called at program termination. For registration using `AT_atexit()`, the function should be in the following format.

```
void user_defined_cleanup_function(void);
```

The example shown below is programmed to call the `my_cleanup_procedure()` function when the program is terminated.

◆ Registration of Termination Function

```
void my_cleanup_procedure(void)
{
    :
    /* user defined cleanup procedures */
    :
}
int test_main(void)
{
    :
    AT_atexit(my_cleanup_procedure);
    :
}
```

! Important

Since a user-defined function registered with `AT_atexit()` is executed in the context of the `tUser-Proc` system task, you cannot perform any process other than releasing the user-reserved system resources. You especially cannot use the `exit()` and `abort()` functions, or functions that change the behavior of the task. If these functions are used, the system may stop responding to requests.

3. 4 ATL Interface

This section describes the interface between the C and C++ languages, and the ATL language. Because the ATL language is used to execute tester control instructions such as setting the pin conditions and clock, this section describes the basic functions necessary for testing devices.

3. 4. 1 Program Configuration Method

The hybrid controller supports two types of methods, which has different program configurations, to describe ATL processes. One method describes the ATL processes in the ATL test plan program and is called the ATL-separated method. A program created by using the ATL-separated method is called the ATL-separated program. The other method describes the ATL processes directly in the C program and is called the ATL-mixed method. A program created by using the ATL-mixed method is called the ATL-mixed program. This method can be used with ASX/U-51 REV 6.02 and later. The interface to execute ATL processes differs with the program configuration method.

[Table 3-3](#) lists the features of each method.

Table 3-3 Comparing the ATL-separated Method and the ATL-mixed Method

Item	ATL-separated method	ATL-mixed method (ASX/U-51 REV 6.02 and later)
ATL description	Describes in the ATL test plan program	Describes in the C program
Compilation method	Split compilation (The C compiler and the XATL/U-51 cross compiler compile the C program and the ATL program respectively)	Batch compilation (The C compiler compiles ATL processes by using the XATL/U-51 cross compiler)
Object program configuration	C program and ATL program	C program only (ATL program codes are embedded in a C object file)
Main ATL interface functions	ATL_init(), ATL_exec()	ATL_init2(), ATL_proc(), ATL_exec()

3. 4. 2 ATL-separated Method

This section describes the ATL interface and system operation when the program is configured by using the ATL-separated method.

3. 4. 2. 1 Preparing for Device Measurement: Loading an ATL Test Plan Program

When the ATL-separated method is used, a program which consists of the C program and the ATL test plan program is used. To link the C program and ATL test plan program, the ATL test plan program should be loaded into the test processor.

The `ATL_init()` function is used to load an ATL test plan program.

```
ATL_init("SAMPLE"); /* load 'PRO SAMPLE' */
```

The `ATL_init()` function fetches parallel test definitions and various test conditions, and sets up the test system for a device test.

This function also creates an ATL linkage table for subsequent calls to `ATL_exec()` function with the `ATL_GOSUB` or `ATL_TEST` command to resolve the branch addresses for labels and TEST statements in the ATL test plan program. When the `ATL_init()` function is executed, the control box (C-BOX) displays the name of the loaded ATL test plan program.

The `ATL_init()` function can be executed repeatedly in a program.

Every time the `ATL_init()` function is called, the ATL test plan program is updated without changing the previous test conditions and measurement results.

Tip

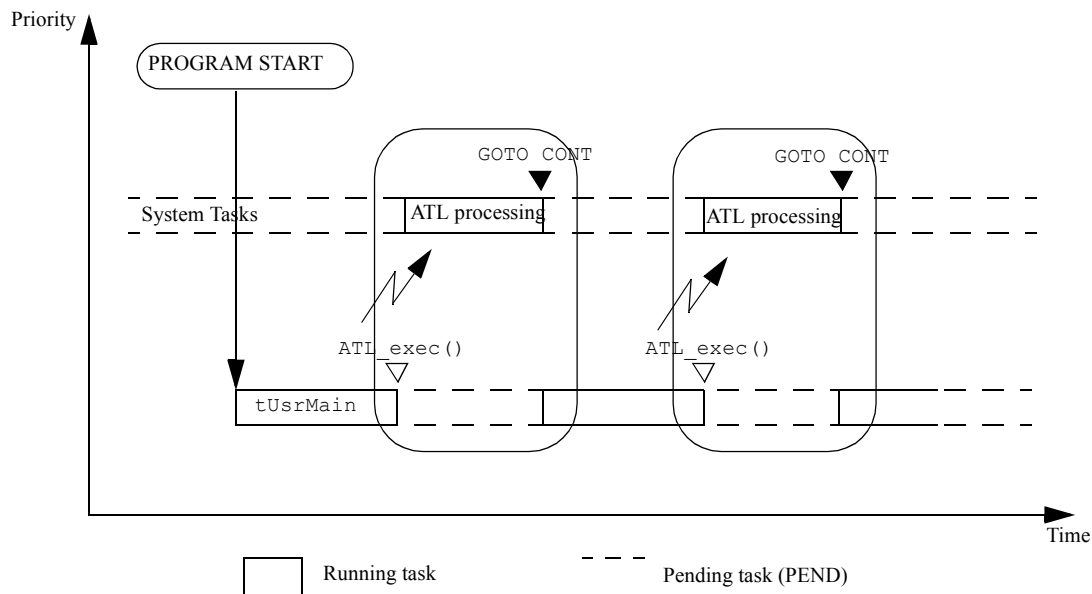
ATL programs used with the ATL-separated method must be compiled using one of the XATL/U-51 cross compilers shown below:

*XATL/U-51 (Product: PXTL51-00S***) REV 6.00 and later*

*XATL/U-51 (Product: PXTL51-36S***) REV 3.00 and later*

3. 4. 2. 2 Executing an ATL Program

By using the `ATL_exec()` function, a C program can execute an internal subroutine or TEST statement in the ATL test plan program loaded by `ATL_init()` or any ATL external subroutine program.

Figure 3-1 ATL Program Linkage at Execution (ATL_GOSUB Command)

The `ATL_exec()` function requests the system program to execute a specified ATL program. The task that called this function remains in the pending state until the system program completes the request or until the ATL program executes an instruction to return to the C program (refer to [Figure 3-1](#)). In multitask processing, `ATL_exec()` can be called from any task. During the execution of this function, however, other user tasks are preempted by the system task and become pending.

➔ For more information on multitasking, refer to [8. 4. 2 "Multitasking" on page 8-10](#) and ["VxWorks Programmer's Guide"](#).

The ATL program executed by the `ATL_exec()` function can execute any ATL statement. However, the `GOTO PRO` statement cannot branch control to another ATL test plan program. For this kind of branching, load an ATL test plan program on the C program side by using the `ATL_init()` function.

If a runtime error occurs during ATL program execution, a runtime error message is displayed on the VKT and the `ATL_exec()` function returns an error. When an `ON` statement is specified in the ATL program or the event handler has been set as described in [3. 5 "Event Processing" on page 3-28](#), the specified `ON` statement or the event handler is processed first. If a runtime error occurs, the `ATL_exec()` function and the group of library functions that access the tester are disabled.

The `ATL_exec()` function receives an argument *command* which specifies what should be executed within the ATL program and a variable number of arguments that are required to execute the specified *command*.

```
ATL_exec(command, ... /* arguments to command */);
```

The value that can be specified for the variable arguments depend on the argument *command*. The *commands* listed in [Table 3-4](#) can be specified for the *command* argument.

Table 3-4 ATL_exec() Command

Command	Explanation
ATL_GOSUB	Executes an internal subroutine in an ATL test program.
ATL_TEST	Executes a TEST statement in an ATL test program.
ATL_CALL	Executes an ATL external subroutine program.

ATL_GOSUB

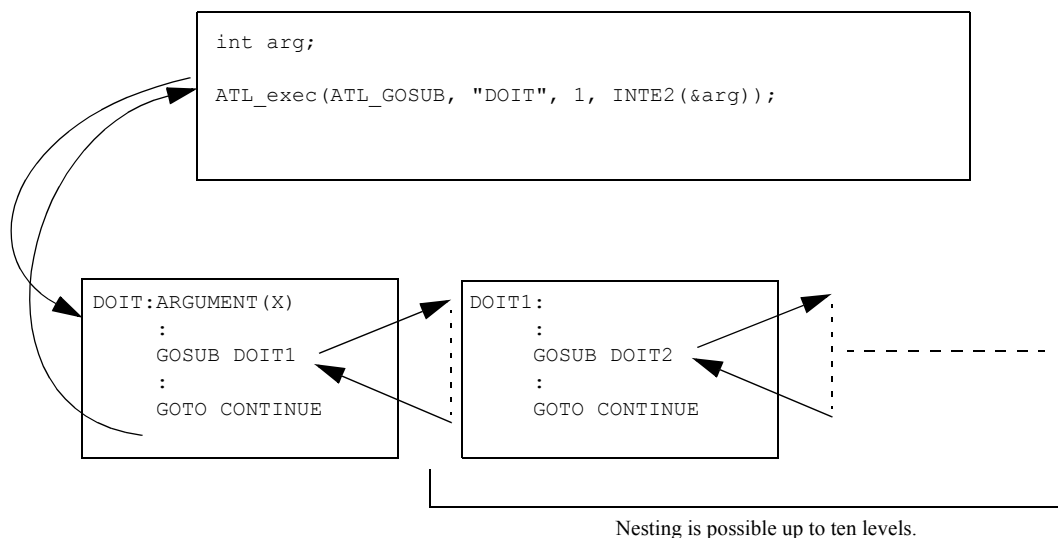
The ATL_GOSUB command executes an internal subroutine in the ATL test program loaded by ATL_init(). For the variable arguments, specify the label name of the internal subroutine to branch to, the number of arguments to be passed to the subroutine, and the specified number of arguments.

➔ For more information on how to pass arguments to a subroutine, refer to [3. 4. 5 "Passing Arguments to an ATL Subroutine" on page 3-21](#).

The function shown below is used to execute the ATL_GOSUB command.

```
ATL_exec(ATL_GOSUB, char *label, int nargs, ...);
```

In this command, the ATL_exec() function returns to the caller when the GOTO CONTINUE statement corresponding to the internal subroutine called by this function is executed in the ATL program. Like an ATL program that is not in hybrid mode, the ATL program can call up to ten internal subroutines using nesting.

Figure 3-2 Branching to Internal Subroutines and Return

ATL_TEST

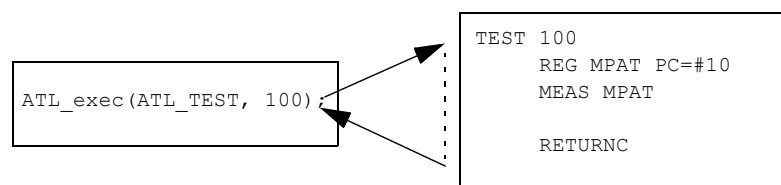
The ATL_TEST command executes a TEST statement in the ATL test program loaded by ATL_init(). For the variable argument, specify the test number for the TEST statement to branch to.

The function shown below is used to execute the ATL_TEST command.

```
ATL_exec(ATL_TEST, int testno);
```

In this command, the ATL_exec() function returns to the caller when a RETURNC statement is executed in the ATL program.

Figure 3-3 Branching to Internal Subroutine and Return



Like a GOTO TEST statement in an ATL program, the ATL_TEST command cannot branch to a TEST statement whose test number is specified by a variable.

ATL_CALL

The ATL_CALL command calls an external subroutine program. For variable arguments, specify an external subroutine program name, the number of arguments to be passed to the subroutine, and the specified number of arguments.

➔ For more information on how to pass arguments to a subroutine, refer to [3. 4. 5 "Passing Arguments to an ATL Subroutine" on page 3-21](#).

The function shown below is used to execute the ATL_CALL command.

```
ATL_exec(ATL_CALL, char *subr_name, int nargs, ...);
```

In this command, the ATL_exec() function returns to the caller when a RETURN statement is executed in the ATL program.

3. 4. 3 ATL-mixed Method

This section describes the ATL interface and system operation when the program is configured by using the ATL-mixed method.

3. 4. 3. 1 Overview

When the ATL-mixed method is used, ATL statements are described in the C program. The C Compiler (atcc) compiles the ATL language which is described in the C program by using the XATL/U-51 cross compiler and outputs ATL object codes into the C object file. By using this method, the C language and ATL language sources and object files can be merged together.

ATL processes are described by using the pragma function (#pragma) which is a preprocess instruction of the C and C++ languages, and can be called from the C process by using the ATL interface function.

Table 3-5 lists the ATL interface for the ATL-mixed program.

Table 3-5 ATL Interface for the ATL-mixed Program

Item	Description
Header file "AT/at1.h"	A file necessary when configuring the ATL-mixed program.
#pragma atl name	Specifies the program name that is displayed in the control box (C-BOX).
#pragma atl socket	Specifies the ATL socket program.
#pragma atl proc	Defines embedded ATL subroutines.
ATL_init2()	Prepares for device measurement and ATL execution.
ATL_proc()	Executes embedded ATL subroutines.



Tip

To compile an ATL-mixed program, one of the following XATL/U-51 cross compilers is necessary.

*XATL/U-51(Product: PXTL51-00S***) REV 6.02 and later*

*XATL/U-51(Product: PXTL51-36S***) REV 3.01 and later*

Figure 3-4 shows an example of an ATL-mixed program in which the driver output level of IN1 is set to 2.2 V for HI and 0.2 V for LOW.

Figure 3-4 ATL-mixed Program Example

```
1:  #include "AT/hybrid.h"
2:  #include "AT/atl.h"
3:
4:  #pragma atl name "SAMPLE"
5:  #pragma atl socket "SSOC"
6:  #pragma atl proc SET_IN(CH,LV(2)) "{
7:      IN(CH)=LV(1),LV(2)
8:  }"
9:
10: int test_main(void)
11: {
12:     int ch;
13:     struct atreal vin[2];
14:
15:     if(ATL_init2() != 0){
16:         AT_perror("ATL_init2()");
17:         exit(1);
18:     }
19:
20:     ch = 1;
21:     AT_strtorm(vin, 2, "2.2V", "0.2V");
22:     if(ATL_proc(SET_IN, 2, INTE2(&ch), DREAL(vin, 2)) != 0){
23:         AT_perror("SET_IN");
24:         exit(1);
25:     }
26:
27:     return(0);
28: }
```

The header file "AT/atl.h" in line 2 must be included in ATL-mixed programs.

Lines 4 through 8 are definitions which are related to ATL. The program name that is displayed in the control box (C-BOX) is SAMPLE and the ATL socket program that is used is SSOC. ATL processes are described between "{" and "}" after the keyword #pragma atl proc, which indicates embedded ATL subroutines. In this example, an embedded ATL subroutine named SET_IN is defined and the IN statement is executed in SET_IN.

ATL_init2() in line 15 prepares to execute ATL and loads the measurement conditions specified in the ATL socket program SSOC.

ATL_proc() in line 22 calls the embedded ATL subroutine SET_IN. As an argument, channel numbers and voltage levels which are specified in the IN statement are passed to SET_IN.

Thus, the ATL-mixed program can describe ATL processes in the C program and execute them.

3. 4. 3. 2 Program Syntax

Definitions which are related to ATL are described by using the pragma function in the ATL-mixed program. This section describes the pragma syntax related to ATL.

3. 4. 3. 2. 1 #pragma atl name

Function

Specifies the program name that is displayed in the control box (C-BOX).

Syntax

```
#pragma atl name "praname"
```

Description

This pragma specifies the program name that is displayed in the control box (C-BOX) when ATL_init2() in a source file in which #pragma atl name is described is executed. The program name is specified for *praname* by the same rule of specifying the ATL test plan program name. If two or more source files are used, separate names can be specified.

If this pragma is not described and another pragma, which is related to ATL, is described, the compiler automatically assigns a unique name, which consists of alphanumeric characters and starts with H, to the source file.

Limitation

1. This pragma cannot be specified in files included by the #include instruction.
2. This pragma cannot be specified more than once in a single source file.
3. The pragma, which is related to ATL, can be specified in up to 256 source files.
If more than 256 source files specify the pragma, the program cannot be loaded.

3. 4. 3. 2. 2 #pragma atl socket

Function

Specifies the ATL socket program.

Syntax

```
#pragma atl socket "socname"
```

Description

This pragma specifies the ATL socket program that is used to set device measurement conditions when ATL_init2() in the source file in which #pragma atl socket is described is executed. The ATL socket program name is specified for *socname*. If two or more source files are used, each source file can specify a different ATL socket program. If no program is specified, it is assumed that there is no ATL socket program which is related to device measurement.

Limitation

1. This pragma cannot be specified in files included by the `#include` instruction.
2. This pragma cannot be specified more than once in a single source file.
3. The pragma, which is related to ATL, can be specified in up to 256 source files.
If more than 256 source files specify the pragma, the program cannot be loaded.

3. 4. 3. 2. 3 #pragma atl proc

Function

Defines embedded ATL subroutines.

Syntax

```
#pragma atl proc name(args...) "{
    ....
    (ATL statements)
    ....
}"
```

Description

This pragma defines embedded ATL subroutines. An embedded ATL subroutine is equivalent to the internal subroutines in the ATL test plan program and can be executed by using the `ATL_proc()` function.

The subroutine name is specified for *name* by the same rule of specifying a label name in the ATL language. Arguments that are passed to the subroutine are specified for *args* by the same rule of specifying arguments in the ARGUMENT statement in the ATL language. However, even if there is no argument specified, parentheses after "*name*" cannot be omitted.

Any ATL statements can be described between "{ that indicates the beginning of a subroutine and }" that indicates the end of subroutine. Describe "{ indicating the beginning of the statement in the same line as `#pragma atl proc`. ATL statements are described in the lines which follow "{. Describe, }" indicating the end of subroutine in a new line after the last ATL statement.

Limitation

1. If character strings are described with an at mark (@) in several lines of an ATL statement, a double-quotation mark (") that encloses a character string must be escaped with a backslash (\).

```
#pragma atl proc XXX() "{
    WRITE \"THIS IS @
        SAMPLE\"
}"
```

2. Macros in the C and C++ languages are not applied to the portion of an embedded ATL subroutine.

3. Preprocessor instructions in the C and C++ languages can not be described in an embedded ATL subroutine.
4. This pragma is described in several lines, but this is supported by only the atcc that is a C compiler for the hybrid controller. To compile by using a different C compiler (for example; for offline debugging), whether or not the macro `__ATCC__` is defined must be checked.

```
#ifdef __ATCC__
#pragma atl proc SET_IN(CH, LV(2)) "{
    IN(CH)=LV(1), LV(2)
}"
#endif
```

5. The pragma, which is related to ATL, can be specified in up to 256 source files. If more than 256 source files specify the pragma, the program cannot be loaded.

3. 4. 3. 3 Preparing Device Measurement

The `ATL_init2()` function must be executed before measuring a device.

The `ATL_init2()` function sets measurement conditions, which are described in the ATL socket program, to the test system before measuring a device. This function also sets conditions which are required to execute the embedded ATL subroutines.

The `ATL_init2()` function has the following format.

```
#include "AT/hybrid.h"
#include "AT/atl.h"

int ATL_init2(void);
```

This function loads the measurement conditions from the ATL socket program specified in `#pragma atl socket` which is described in the source file where this function is called. If no ATL socket program is specified in the source file, it is assumed that no ATL socket program exists.

When `#pragma atl name` is specified in the source file where this function is called, the program name is displayed in the control box. If `#pragma atl name` is not specified, a program name that was assigned by the compiler is displayed in the control box.

When `ATL_init2()` is called twice or more, the program name that is displayed in the control box according to the `#pragma atl` specifications in the called source file and the number of devices for parallel testing are updated each time, while the measurement conditions and measurement results remain unchanged.

3. 4. 3. 4 Executing Embedded ATL Subroutines

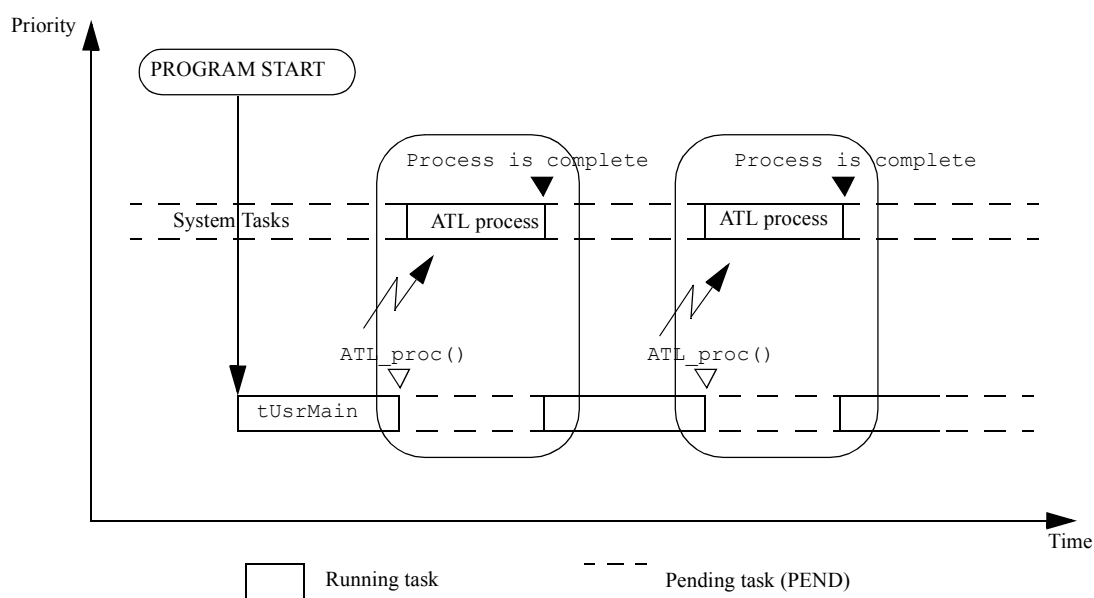
To execute embedded ATL subroutines, the `ATL_proc()` function is used.

The ATL_proc() function requests the system program to execute the specified embedded ATL subroutine and then the system program executes the ATL processes. Until the processes are complete, the task that called this function is suspended (pend) (Refer to [Figure 3-5](#)). This operation is equivalent to the ATL_exec() function in the ATL-separated method.

➔ For more information on the ATL_exec() function, refer to [3. 4. 2. 2 "Executing an ATL Program" on page 3-8](#).

If a runtime error occurs while executing the subroutine, a runtime error message is displayed on the VKT and the ATL_proc() function returns an error. However, if an ON statement is specified in the subroutine or if the event handler was set as described in [3. 5 "Event Processing" on page 3-28](#), the ON statement or the event handler is performed prior to the error process. Once a runtime error occurs, the ATL_proc() function always returns an error.

Figure 3-5 Executing Embedded ATL Subroutines and System Operation



The ATL_proc() function has the following format.

```
#include "AT/hybrid.h"
#include "AT/atl.h"

int ATL_proc(atlproc_t procaddr, int nargs, ...);
```

The embedded ATL subroutine address, which is defined in the source file where this function is called, is specified for the argument *procaddr*. The embedded ATL subroutine address is given as a *atlproc_t*-type variable that has the same name as the subroutine which is specified in `#pragma atl proc`. The embedded subroutine which is executed must be defined before this function is called.

The number of arguments to be passed to the subroutine is specified for the argument *nargs*. The *nargs* arguments, which is passed to the subroutine, is specified for the variable argument by the procedure described in [3. 4. 5 "Passing Arguments to an ATL Subroutine" on page 3-21](#).

An embedded ATL subroutine can call subroutines that are nested up to ten levels. Also, an embedded ATL subroutine which is defined in the same source file can be called as an internal subroutine. The ATL_proc function returns to the caller when all the processes in the subroutine are complete or when the nest level of the internal subroutine is 0 and the GOTO CONTINUE statement is executed. (Refer to [Figure 3-6](#), [Figure 3-7](#), and [Figure 3-8](#))

Figure 3-6 Execution and Return of an Embedded ATL Subroutine (1)

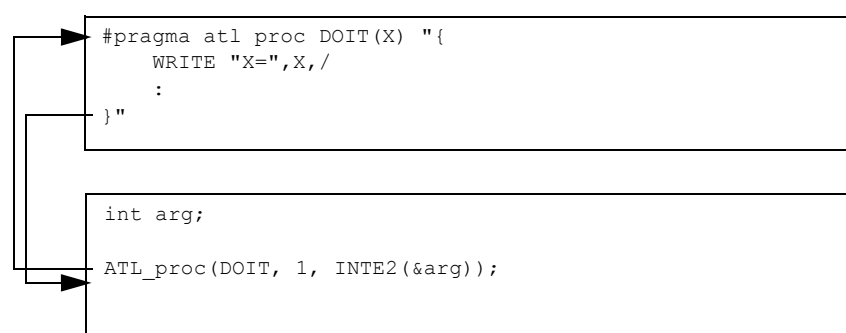


Figure 3-7 Execution and Return of an Embedded ATL Subroutine (2)

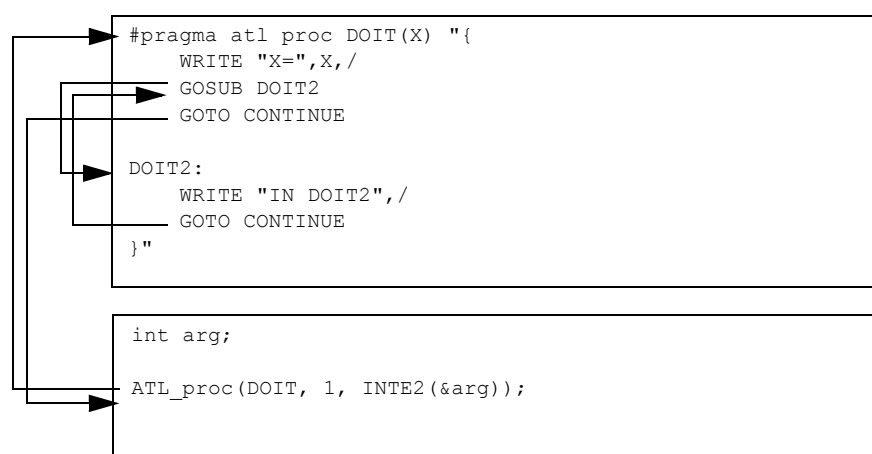
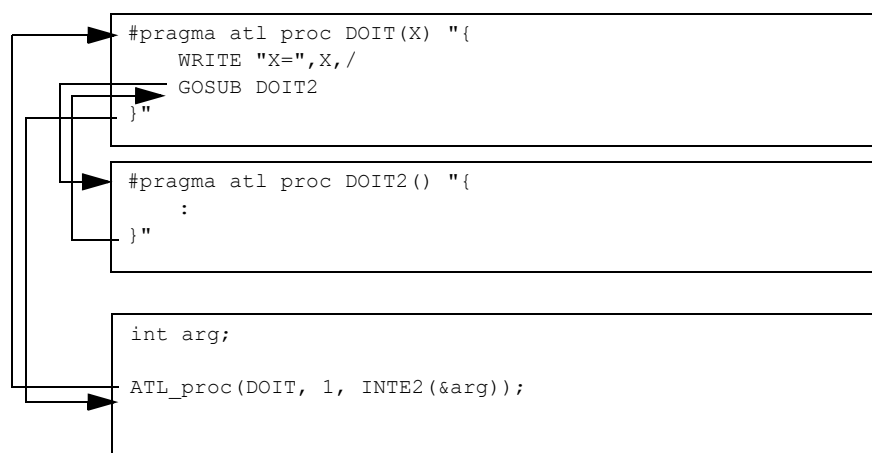


Figure 3-8 Execution and Return of an Embedded ATL Subroutine (3)

3. 4. 3. 5 Executing an ATL External Subroutine Program

To execute an ATL external subroutine program directly from the C program, the `ATL_CALL` command is used in conjunction with the `ATL_exec()` function in the same manner as the ATL-separated method. The ATL-mixed program can not use commands rather than the `ATL_CALL` command of the `ATL_exec()` function.

➔ For more information on the `ATL_exec()` function, refer to [3. 4. 2. 2 "Executing an ATL Program" on page 3-8](#).

3. 4. 3. 6 The Scope of Variables Declared in an Embedded ATL Subroutine

A variable, which is declared in an embedded ATL subroutine, is effective in the source file in which the variable is described and can be referred to, or changed from, the other embedded ATL subroutine which is in the same source file. The same variable name cannot be declared in different embedded ATL subroutines in a single source file. If the same variable name is specified in different source files, these variables indicate different settings.

3. 4. 3. 7 ATL Program Variable Area

When the ATL-mixed method is used, ATL object codes are included in the object file of the C program. The ATL object codes are loaded into the user memory area as a part of the C program. The variable area required for execution is also allocated from the user memory area.

➔ For more information on the memory layout of the tester processor, refer to [8. 2 "Tester Processor Configuration" on page 8-2](#).

**Tip**

To execute the ATL external subroutine program, variable area for the ATL program in the controller is used.

3. 4. 4 Expressing Fixed-Bit Width Integers and Floating-Point Numbers with Unit

The bit width for the integer type of the C and C++ languages is not defined as that of the ATL language. Therefore, the hybrid controller uses the data type for which the bit width is defined (refer to [Table 3-6](#)) to transfer data between C and ATL programs.

Table 3-6 Integers Defining Bit Width

Type	Explanation
int16_t	16-bit signed integer
int32_t	32-bit signed integer
uint16_t	16-bit unsigned integer
uint32_t	32-bit unsigned integer

Since the C program cannot handle floating-point numbers (real numbers) with units, these numbers are expressed with the structure `struct atreal`, which has a double-precision floating-point number and its unit as members.

```
struct atreal {
    double      value; /* Value */
    enum AT_UNIT unit; /* Unit */
};
```

The following enumerators are defined for the enumeration `AT_UNIT`:

Name	Explanation
UNIT_NONE	No unit
UNIT_A	Unit: Ampere (A)
UNIT_V	Unit: Volt (V)
UNIT_S	Unit: Second (S)

The `AT_strtor()` and `AT_strtorm()` library functions convert a character string of a floating-point number with a unit to the structure `struct atreal`. Using these library functions, the user can specify a floating-point number with a unit in a form that is visually easy to understand.

```
struct atreal x[2];

/* x[0]=0.3V, x[1]=2.6V */
AT_strtorm(x, 2, "0.3V", "2.6V");
```

3. 4. 5 Passing Arguments to an ATL Subroutine

The `ATL_exec()` and `ATL_proc()` functions can pass arguments to an ATL subroutine. Specify arguments by using the macro shown in [Table 3-7](#).

Table 3-7 Passing Arguments to an ATL Subroutine

Macro Name	Argument Type	Argument-passing Method
<code>INTE1(int16_t *p)</code> <code>INTE2(int32_t *p)</code> <code>DINTE1(int16_t *p, size_t sz)</code> <code>DINTE2(int32_t *p, size_t sz)</code>	16-bit signed integer 32-bit signed integer 16-bit signed integer array 32-bit signed integer array	Call by reference
<code>LOGI1(uint16_t *p)</code> <code>LOGI2(uint32_t *p)</code> <code>DLOGI1(uint16_t *p, size_t sz)</code> <code>DLOGI2(uint32_t *p, size_t sz)</code>	16-bit unsigned integer 32-bit unsigned integer 16-bit unsigned integer array 32-bit unsigned integer array	
<code>REAL2(double *p)</code> <code>REAL3(struct atreal *p)</code> <code>REAL4(double *p)</code> <code>DREAL2(double *p, size_t sz)</code> <code>DREAL3(struct atreal *p, size_t sz)</code> <code>DREAL4(double *p, size_t sz)</code>	Single-precision floating-point number without unit Single-precision floating-point number with unit Double-precision floating-point number without unit Single-precision floating-point number array without unit Single-precision floating-point number array with unit Double-precision floating-point number array without unit	
<code>STRING(char *p, size_t sz)</code>	Character string	
<code>VREAL2(double *p)</code> <code>VREAL3(struct atreal *p)</code> <code>VREAL4(double *p)</code> <code>DVREAL2(double *p, size_t sz)</code> <code>DVREAL3(struct atreal *p, size_t sz)</code> <code>DVREAL4(double *p, size_t sz)</code>	Single-precision floating-point number without unit Single-precision floating-point number with unit Double-precision floating-point number without unit Single-precision floating-point number array without unit Single-precision floating-point number array with unit Double-precision floating-point number array without unit	Call by value

Macro Name	Argument Type	Argument-passing Method
CINTE(int32_t <i>v</i>) CLOGI(uint32_t <i>v</i>) CREAL(double <i>v</i>)	32-bit signed integer 32-bit unsigned integer Single-precision floating-point number without unit	Call by value (Constant)

The argument-passing methods are classified into two types: call by reference and call by value. When using call by reference, and when any value is assigned to the dummy argument variable in the called ATL program, then the assignment result is reflected on the C program. When using call by value, a dummy argument variable is assigned locally; as a result, the assignment result is not reflected on the C program. When a constant is passed, no value can be assigned to a dummy argument variable.

For the argument *p* of the macros, specify a pointer to an object of the type corresponding to the macro. If the argument is an array, specify the number of elements of the array in argument *sz*. To pass a constant, specify its value in argument *v*.

The ATL subroutine can receive the specified argument as a real argument of the type shown in [Table 3-8](#).

Table 3-8 Real Argument Types in an ATL Subroutine

Macro Specifying Argument	Real Argument Types in an ATL Subroutine
INTE1 INTE2 DINTE1 DINTE2	INTE[1] type non-array variable INTE[2] type non-array variable INTE[1] type, one-dimensional and non-subscripted array variable (The number of elements is sz.) INTE[2] type, one-dimensional, and non-subscripted array variable (The number of elements is sz.)
LOGI1 LOGI2 DLOGI1 DLOGI2	LOGI[1] type non-array variable LOGI[2] type non-array variable LOGI[1] type, one-dimensional and non-subscripted array variable (The number of elements is sz.) LOGI[2] type, one-dimensional, and non-subscripted array variable (The number of elements is sz.)
REAL2, VREAL2 REAL3, VREAL3 REAL4, VREAL4 DREAL2, DVREAL2 DREAL3, DVREAL3 DREAL4, DVREAL4	REAL[2] type non-array variable REAL[3] type non-array variable REAL[4] type non-array variable REAL[2] type, one-dimensional and non-subscripted array variable (The number of elements is sz.) REAL[3] type, one-dimensional, and non-subscripted array variable (The number of elements is sz.) REAL[4] type, one-dimensional, and non-subscripted array variable (The number of elements is sz.)
STRING	STRING variable (The number of characters which can be assigned is sz.)
CINTE CLOGI CREAL	Precision integer constant Precision logical constant Precision real number without a unit

➔ For more information on the relationship between actual arguments and dummy arguments in a subroutine of an ATL program, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

When Floating-point number arguments are passed, the floating-point expression is converted because the C program and the ATL program have different expressions. However, this conversion might result in an error because the expression of floating-point numbers is not compatible. Therefore, there are two methods of passing arguments of floating-point numbers as variables: call by reference and call by value, as with REAL2 and VREAL2. When passing floating-point numbers, take into consideration the fact that this conversion affects the C program, and distinguish these passing methods appropriately.



About conversion of floating-point numbers

Converting floating-point numbers might result in an error, depending on whether or not the mantissa part is excess or deficiency for the expression of the floating-point number. In some cases, a floating-point number written in a C program cannot be correctly expressed in an ATL program.

➔ For more information, refer to ATL_exec() described in [ASX/U-51 Hybrid Controller Reference Manual\(No. : 8409600\)](#).

If a floating-point number is passed using call by reference, this conversion is applied even when argument values are not changed in an ATL program; as a result, the value of the variable in a C program may differ before and after it is passed as an argument. When argument values are not being changed in an ATL program, this kind of operation can be prevented by passing arguments with the call by value method.

An example of this conversion is shown below. Because, value 0.1 used in this example has an infinite fraction in a binary floating-point expression; an error occurs during conversion. When this occurs, the original value of the variable is changed even though no value is rewritten in the ATL program.

◆ Effect Due to Error at Conversion of Floating-point Number

- Example of value change (Call by reference)

(C program) <pre>double x = 0.1; AT_printf("x=%e\n", x); ATL_exec(ATL_GOSUB, "LB", 1, REAL2(&x)); AT_printf("x=%e\n", x);</pre>	(ATL program) <pre>LB: ARGUMENT(X) GOTO CONTINUE</pre>
(Output) <pre>x = 1.000000e-01 x = 9.999996e-02</pre>	

- Example of no value change (Call by value)

(C program) <pre>double x = 0.1; AT_printf("x=%e\n", x); ATL_exec(ATL_GOSUB, "LB", 1, VREAL2(&x)); AT_printf("x=%e\n", x);</pre>	(ATL program) <pre>LB: ARGUMENT(X) GOTO CONTINUE</pre>
(Output) <pre>x = 1.000000e-01 x = 1.000000e-01</pre>	

Tip

When the equivalence comparison of a floating-point number in an ATL program and that in a C program are performed, either value may be an approximate one so the program may not perform as expected. Also, when a magnitude comparison is performed, the value may be an approximate value. Accordingly consider these points when you make a program.

3. 4. 6 Data Exchange by System Variables

Data can be exchanged between a C program and an ATL program not only by subroutine arguments but also by system variables.

The hybrid controller uses the following common system variables among all stations.

Table 3-9 System Variables HBn

System Variable	Variable Type
HB0001 - HB0010	GENERAL[5] type
HB0011 - HB0999	GENERAL[3] type
HBCOMM	STRING type of 256 characters

In an ATL program, these system variables can be accessed by directly specifying their names in the program. In a C program, system variables are accessed using library functions. The following functions are available to access system variables:

Table 3-10 Functions Used to Access System Variables

Function	Explanation
ATL_set_inte()	Sets a integer value to a system variable.
ATL_get_inte()	Acquires a system variable as an integer.
ATL_set_intem()	Sets an integer array to multiple continuous system variables.
ATL_get_intem()	Acquires multiple continuous system variables as an integer array.
ATL_set_logi()	Sets a logical number to a system variable.
ATL_get_logi()	Acquires a system variable value as a logical number.
ATL_set_logim()	Sets a logical array to multiple continuous system variables.
ATL_get_logim()	Acquires multiple continuous system variables as a logical array.
ATL_set_real()	Sets a real number to system variable.
ATL_get_real()	Acquires a system variable as a real number.
ATL_set_realn()	Sets a real number array to multiple continuous system variables.
ATL_get_realn()	Acquires multiple continuous system variables as a real number array.
ATL_set_char()	Sets a character string to a system variable.
ATL_get_char()	Acquires a system variable as a character string.

For the ATL_set_real(), ATL_set_realn(), ATL_get_real() and ATL_get_realn() functions, keep in mind the fact that floating-point numbers are converted between ATL and C programs in the same way that arguments are passed to a subroutine.

When accessing a system variable from a C program, the target system variable must be specified in the argument of each function. The `ATL_set_inte()` function, for example, has the following format, where the first argument *svid* specifies an ID code representing the system variable.

```
ATL_set_inte(int svid, int32_t src);
```

The ID codes of system variables are defined as follows.

Table 3-11 ID Codes of System Variables

ID Code	System Variable	Remarks
SV_HB(int <i>n</i>)	HB0001-HB0999	Common to all stations <i>n</i> : Integer value from 1 to 999 indicating the system variable number
SV_HBCOMM	HBCOMM	Common to all stations
SV_DEVICE	DEVICE	For each station
SV_LOT	LOT	For each station
SV_SUBLOT	SUBLOT	For each station
SV_DUTIF	DUTIF	For each station
SV_OPERAT	OPERAT	For each station
SV_COMMEN	COMMEN	For each station
SV_WAFER	WAFER	For each station
SV_SWITCH	SWITCH	For each station
SV_GLOB(int <i>n</i>)	GLOB1-GLOB50	For each station <i>n</i> : Integer value from 1 to 50 indicating the system variable number

A system variable for each station refers to the corresponding variable of the current station.

The `ATL_get_XXX()` function converts data set in a target variable to the type of data to be acquired (XXX in the function name).

The example below sets 3.5 V from a C program to the system variable HB0011. An ATL program changes the value into a logical number and the C program reads the value.

◆ System Variable Access from C Program

```
struct atreal x;
uint32_t      lval;

/* set 3.5V in HB0011 */
ATL_set_real(SV_HB(11), AT_strtor(&x, "3.5V"));
:
/* change the value of HB0011 in ATL */
ATL_exec(...);
:
/* read as LOGICAL[2] */
ATL_get_logi(SV_HB(11), &lval);
```

The value of the system variable HBn is saved on a host disk automatically in the same manner as other system variables when the controller terminates. It is automatically restored when the controller starts.

3. 5 Event Processing

An event indicates a device test failure or other specific system symptom. Using an event processing function, a C program can activate any user-defined processing if an event occurs during program execution.

This section explains the event processing function.

3. 5. 1 Event Class

Events can be classified into a synchronous event class and an asynchronous event class. These classes have the following characteristics.

Synchronous event class

The synchronous event class is a class of events that occur in synchronization with program execution.

An event belonging to this class occurs during specific processing only. For example, a functional test failure event occurs in a functional test only. This kind of event can be considered as an event which occurs synchronously with program execution.

This class has the following characteristics:

- An event occurs during specific processing only.
- An event handler is executed in the context of a user task.
- There are almost no limits on the processing that can be performed by an event handler.
- No other synchronous events occur during the execution of the event handler.

Asynchronous event class

The asynchronous event class is a class of events that do not occur in synchronization with program execution. An event belonging to this class occurs irrespective of the status of program execution. For example, an MRA analysis termination event occurs at the end of an MRA analysis executed by the host system. This kind of event can be considered as an event which occurs asynchronously with program execution.

This class has the following characteristics:

- An event may occur at any time.
- An event handler is executed by interrupting the program.
- An event handler is executed in the context of a system task.
- Another asynchronous event may occur during the execution of the event handler.
- There are limits on the processing that can be performed by the event handler.

3. 5. 2 Event Handler

An event handler performs the processing executed when an event occurs. The operations and characteristics of event handlers are different for synchronous and asynchronous events. This section explains what is common to both types of event handlers.

Only one event handler can be set for each event. A default event handler is prepared for each event. If no event handler is set, the default event handler is executed. If the event handler is set to ignore an event, no processing is executed. The action of the default event handler is defined for each event. Refer to the section of each event class for more information.

The `AT_event()` function is used to set an event handler. This function has the following format.

```
int AT_event(
    int event,
    void (*handler)(int)
);
```

For the argument *event*, specify an integer value (event code) representing the target event. For the argument *handler*, specify a user-defined function to be registered as an event handler. An event handler function defined by the user must have the following format.

```
void handler(
    int event
);
```

The code of the event that caused the event handler call is passed to the argument *event* of the *handler()* function.

In addition to user-defined event handlers, the following predefined event handlers can be specified.

Table 3-12 Predefined Event Handlers

Event Handler	Explanation
ATEV_DFL	Specifies the default event handler.
ATEV_IGN	Ignores the event.

The event handler is reset to the settings of the default event handler `ATEV_DFL` just before its execution. To make the event handler effective continuously, the event handler has to be set each time it is executed.

3. 5. 3 Synchronous Event Class

3. 5. 3. 1 Synchronous Event

A synchronous event occurs in synchronization with program execution. [Table 3-13](#) lists the event codes for synchronous events.

Table 3-13 Event Codes for Synchronous Events

Event Code	Explanation
ATEV_DCTF	Failure in a DC parametric test
ATEV_FCTF	Failure in a functional test
ATEV_RSTE	Runtime or system error during ATL program execution

A synchronous event handler is managed with the branch destination of the ON statement in the ATL program. The handler settings are as follows:

- If the ATL_init() or ATL_init2() function is executed, the event handler is reset to ATEV_DFL.
- The event handler set in the C program remains invalid while the ATL_exec() function is executing an external subroutine program. Then the ON statement in the external subroutine becomes valid.
- If the AT_event() function and the ON statement in the ATL test plan program (when the ATL-separated method is used) or the embedded ATL subroutine (when the ATL-mixed method is used) are executed for the same event, the most recent setting takes effect.

[Table 3-14](#) explains the action of the default event handler ATEV_DFL for synchronous events.

Table 3-14 Default Event Handler (ATEV_DFL) Actions for Synchronous Events

Event Code	Default Operation
ATEV_DCTF	Ignored
ATEV_FCTF	Ignored
ATEV_RSTE	The ATL program is suspended and the ATL_exec() or ATL_proc() function returns an error to the caller. An error is sent to the handler or prober, and the error lamp in the control box (C-BOX) lights up.

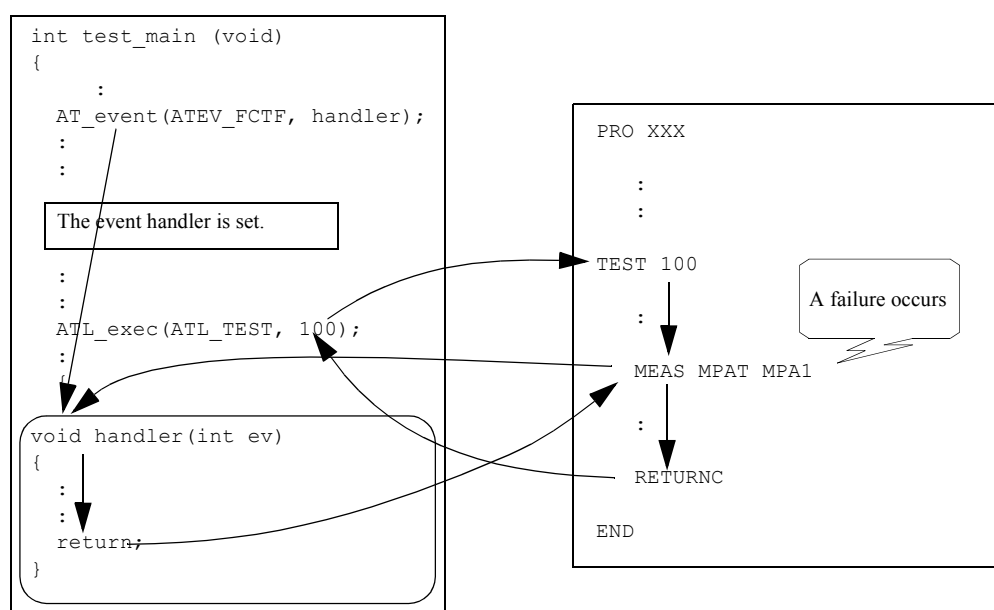
3. 5. 3. 2 Synchronous Event Handler

A synchronous event handler is executed when an event occurs. After termination of the event handler, the program restarts processing as follows.

Table 3-15 Operations After Terminating Synchronous Event Handler

Event Code	Explanation
ATEV_DCTF	Restarts processing from the status just before the event occurred.
ATEV_FCTF	Restarts processing from the status just before the event occurred.
ATEV_RSTE	Restarts processing from the next line of the location where the error occurred in the ATL program.

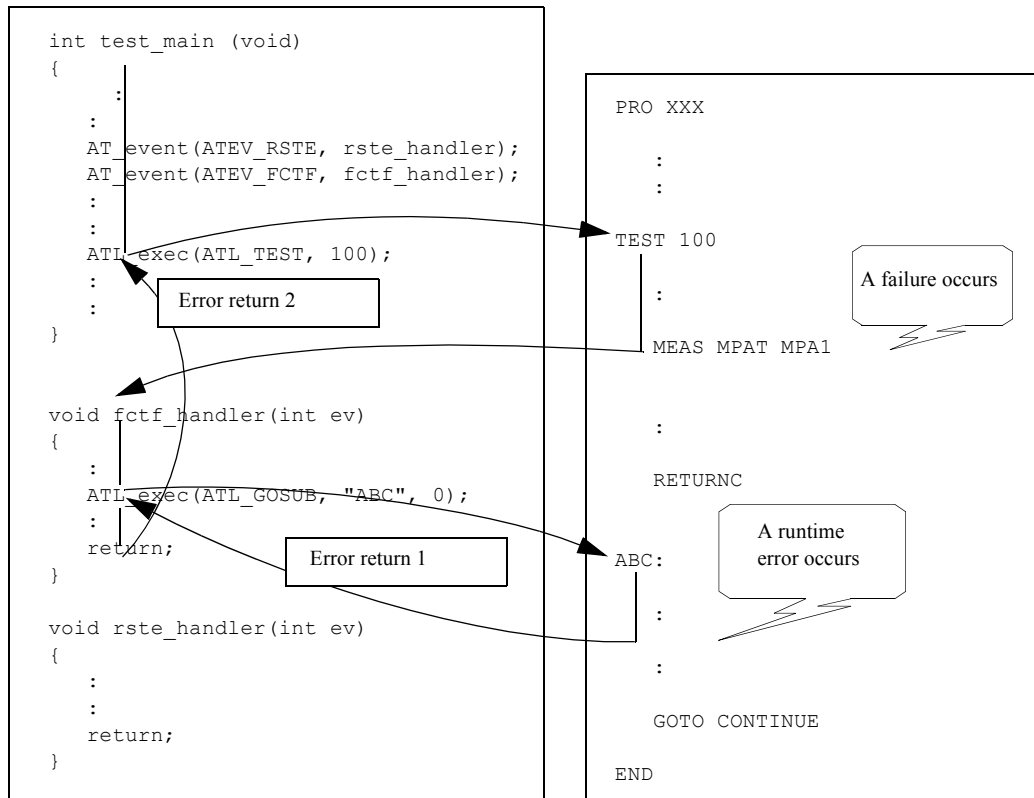
Figure 3-9 shows the execution status of a synchronous event handler.

Figure 3-9 Synchronous Event Handler Execution (1)

A synchronous event handler is executed in the context of the task that called the `ATL_exec()` or `ATL_proc()` function. Consequently, a synchronous event handler can execute ordinary processing except in the following cases:

Event processing during operation of a synchronous event handler

Synchronous event processing is disabled during the operation of a synchronous event handler. If a synchronous event other than `ATEV_RSTE` occurred by the ATL program execution in the synchronous even handler, that event is ignored. If an `ATEV_RSTE` event occurs, an error is returned by the associated `ATL_exec()` or `ATL_proc()` function. In addition, the process immediately before the event handler execution is discontinued and an error is returned by the original `ATL_exec()` or `ATL_proc()` function after the termination of the event handler. (Figure 3-10)

Figure 3-10 Synchronous Event Handler Execution (2)**Setting of a synchronous event handler during synchronous event handler operation**

The setting of a synchronous event handler during synchronous event handler operation takes effect after termination of the handler.

ONRTN statement in a synchronous event handler

If the ATL program is executed by using the ATL_exec() or ATL_proc() function from a synchronous event handler, the ONRTN statement, which is used while the ATL program is executed, is processed as No Operation (NOP).

Non-local jump from a synchronous event handler

The AT_evlongjmp() function is used for a non-local jump from a synchronous event handler. If the longjmp() function is used, the ATL program is regarded as processing a synchronous event and the subsequent synchronous event processing is disabled.

3. 5. 4 Asynchronous Event Class**3. 5. 4. 1 Asynchronous Event**

An asynchronous event is a real-time event that occurs irrespective of the program execution. [Table 3-16](#) lists the event codes for asynchronous events.

Table 3-16 Event Codes for Asynchronous Events

Object Function	Event Code	Explanation
MRA ^{*1}	ATEV_MRAANLY	End of analysis
	ATEV_MRARSLT	End of creation of repair analysis file
	ATEV_MRAUSRP	End of user process

**1 To combine the hybrid controller with MRA, a version of the MRA software compatible with this function must be used. For more information on whether the MRA software is compatible, contact Advantest Corporation.*

The program can control the following aspects regarding the reception of an asynchronous event:

- Blocking and unblocking the event
Using the AT_evmask() function, the program can block (or unblock) the delivery of an event. The blocked event, if one occurs, is added to the event queue until its delivery is unblocked. The event handler is disabled during this period.
- Waiting for an event
Using the AT_evwait() function, the program can wait for an event. This allows synchronous processing even though the event is an asynchronous event.

Table 3-17 explains the action of the default event handler ATEV_DFL for asynchronous events.

Table 3-17 Default Event Handler Actions for Asynchronous Events

Event Code	Default Operation
ATEV_MRAANLY	Ignored
ATEV_MRARSLT	Ignored
ATEV_MRARSLT	Ignored

3. 5. 4. 2 Asynchronous Event Handler

If an asynchronous event that is not blocked occurs, the execution of the current program is suspended and the event handler for the event is executed. An asynchronous event handler interrupts even a synchronous event handler. When the event handler returns, the program restarts processing from the interrupted position.

Limitations on asynchronous event handlers

An asynchronous event handler is executed in the context of the system task tUsrProc. The following limitations apply to an asynchronous event handler:

- The ATL_init(), ATL_exec(), ATL_init2(), ATL_proc and AT_evwait() functions cannot be used.
- The AT_event() function cannot be used to set a synchronous event handler.

- Since the asynchronous event handler interrupts the system program, the system cannot respond during execution of the asynchronous event handler. As a result, do not perform a process using the asynchronous event handler that causes the system to be in a wait state for a long time.
- The available function stack size for an asynchronous event handler is 1 MB. To prevent the function stack from being used up, avoid reserving a large area using an automatic variable and increasing the nesting level with recursive function calls.
- Do not use a function that changes the attribute of the tUsrProc task.
- Do not use a function, for example, exit() or abort(), that causes the tUsrProc task to terminate. If such a function is used, the system may stop responding.
- The SIGINT signal is reserved in the tUsrProc task. As a result, do not change the signal handler and signal mask for the SIGINT signal in the asynchronous event handler.

Queuing of asynchronous events

If another asynchronous event occurs during the execution of an event handler, the event is added to the event queue and processed after the current event handler returns. The size of the event queue is 32. Asynchronous events that occurred when the event queue was full are discarded. It is guaranteed that the delivery of an event is done in the order of its occurrence. However, the delivery of a blocked event is delayed until the event is unblocked.

Exceptions caused by asynchronous event handler

If the asynchronous event handler causes an exception, the controller performs the process described in [3. 7 "Hardware Exception" on page 3-36](#). However, when the asynchronous event handler interrupts the execution of an ATL program, the error is not processed until the execution of the ATL program ends.

3. 6 Implementation of Error Number errno

A library function fails if an error occurs. A failure of a library function is indicated by return of an unusual value from the function. Such values are -1 or the NULL pointer.

➔ For more information on the return values, refer to the reference manual for each function.

When an error occurs in a library function, the function assigns an error number to the errno external variable. When an error occurs, you can find the cause of the error in the program in detail by checking the errno value.

➔ For more information on the implementation of errno in VxWorks, refer to errnoLib described in ["VxWorks Reference Manual"](#).

In the hybrid controller, a library function processes the errno value in VxWorks using a unique error number that does not interfere with the error numbers defined in VxWorks. The error numbers unique to the hybrid controller are not recognized by the VxWorks errnoLib functions and C standard library functions. The hybrid controller includes the AT_printErrno(), AT_perror() and AT_strerror() library functions which recognize both VxWorks and unique error numbers. Use these library functions instead of VxWorks standard functions.

Some library functions of the hybrid controller assign an error number corresponding to an ATL error code to errno. The error message that corresponds to such an errno value has the following format:

```
S_atlerr_XXXX
```

An ATL error code (in octal numbers) is set for XXXX of the error message. To confirm the error contents, use the /ERR error code utility.

In the source program, the errno value can be referenced using the S_atlerr_ERRNO(v) macro. For the argument v in the macro, specify the value of an ATL error code.

A coding example using errno is shown below.

◆ Processing of errno

```
if(ATL_exec(ATL_GOSUB, "LB1", 0) != 0){
    /* check errno value */
    if(errno == S_atlerr_ERRNO(03051)){
        ... /* do something special for 3051 error */
    }else{
        ... /* other errors */
    }
}
```

3. 7 Hardware Exception

A hardware exception is a hardware trap that occurs if a program attempts an illegal operation. If a non-existent address is accessed or an illegal instruction is executed, an exception occurs and the program is usually disabled. If a user program is disabled, the controller immediately notifies the user of a reset.

3. 7. 1 Exception and Signal

A hardware exception is related to a signal prescribed by POSIX. If an exception occurs, a signal corresponding to the exception is delivered to the task that caused the exception. If a signal handler is set for the signal, it is activated. Since VxWorks ignores all signals by default, a signal handler must be explicitly set for the signal.

➔ For explanation of the relationship between exceptions and signals and the associated interface, refer to the description of sigLib in ["VxWorks Reference Manual"](#).

3. 7. 2 Exception Processing

The controller detects and processes the following exceptions generated by user programs.

Table 3-18 Exceptions Detected by the Controller

Exception Type	Explanation
Data access exception	Attempted to access data in an invalid address.
Instruction access exception	Attempted to access an instruction in an invalid address.
Alignment exception	The memory access does not satisfy the alignment requirements of the processor.
Program exception	Attempted to execute an invalid instruction.
Floating point unavailable exception	A floating-point operation instruction was executed when an floating-point operation is not permitted.

If either of the following conditions is satisfied when an exception occurs in a user program, all user tasks are suspended and an error message is displayed on the VKT:

- No signal handler is set for the exception.
- The signal handler has returned after processing was completed.

In this case, an error is sent to the handler or probe connected to the tester and the error lamp in the control box (C-BOX) goes on.

3. 7. 3 Exception Error Messages

If an exception occurs, the controller outputs a message useful for debugging to the VKT. Exception error messages are output in the following format.

```
Error: 1          (a)
Task : tUsrMain  (b)
Data access exception.    (c)
Your program was suspended.
Hit [CTRL-C] to abort your program.

Exception current instruction address: 0xffffffff (d)
                                   in function: exc_trigger()

==== Stack Trace ====

(Contents of stack frame)    (e)
```

- (a) Error number
- (b) Name of the task which caused the exception
- (c) Message corresponding to the exception type
- (d) Address and function of the instruction where the exception occurred
- (e) Trace output from the stack frame of the task which caused the exception

[Table 3-19](#) lists the error numbers and exception types with their corresponding messages.

Table 3-19 Exception Types and Messages

Error No.	Exception Type	Message
1	Data access exception	Data access exception.
2	Instruction access exception	Instruction access exception.
3	Alignment exception	Alignment exception.
4	Program exception	Program exception.
5	Floating point unavailable exception	Floating point unavailable exception.

Function call relationships are output to the contents of stack frame sequentially from the earliest function call of the task where the exception occurred. The format is as follows.

```
[address] [symbolic address]: [function]
```

- address
The address of the instruction that is calling the function indicated by "function" is displayed.
- symbolic address
Name of the symbol nearest "address" and the offset value from the symbol to "address"

- function

Name of the function called at "address". If the function is a local one, its address and the offset value from the nearest symbol are displayed.

The following is a display example.

```
Error: 1
Task : tUsrMain
Data access exception.
Your program was suspended.
Hit [CTRL-C] to abort execution.

Exception current instruction address: 0xdffff48
                                   in function: exc_trigger()

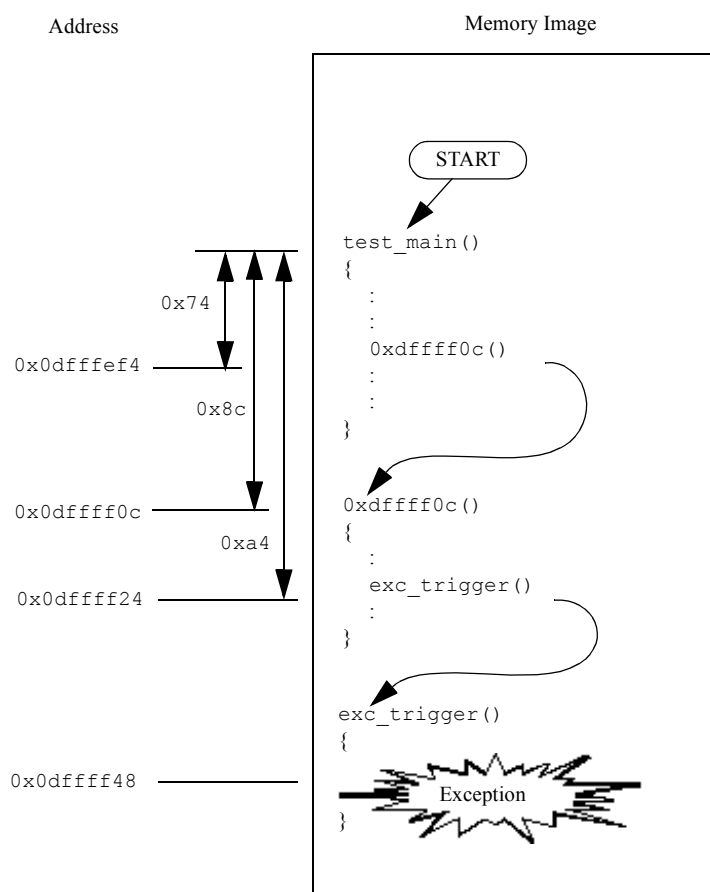
==== Stack Trace ====
0x0dffff24 test_main          + 0xa4: exc_trigger()
0x0dffffef4 test_main          + 0x74: 0xdffff0c() (@ test_main + 0x8c)
0x001b2bc8 vxTaskEntry        + 0x5c: test_main()
```

This example indicates that a data access exception occurred in the task tUsrMain. The exception occurred at the address 0xdffff48 in the exc_trigger() function. According to the contents of the stack frame, the exception occurred in the following series of function calls.

```
test_main()  →  0xdffff0c()  →  exc_trigger()
```

The 0xdffff0c() function is located at 0x8c bytes forward from test_main() and is called at 0x74 bytes forward from test_main(). By tracking the stack frame in this way, the function call relationship can be visualized as the locations in memory, as shown in [Figure 3-11](#). This image enables a more detailed analysis of the cause of the exception.

Figure 3-11 Tracing and Analyzing the Stack Frame



3. 7. 4 Return from Exception

If an exception occurs, the program stops at the location where the exception occurred. To terminate the program and return the controller to the command wait state, enter [CTRL]-[C] from the VKT or press the **[RESET]** button on the control box (C-BOX).

3. 8 File I/O

3. 8. 1 File

The controller contains three file system devices: an ASX file device, an NFS device, and a non-NFS network device. The user can open, read, and write files on these file system devices in a C program.

Use the file system devices after considering their features. If any of the devices will do, use the ASX file system device. Because this device has a function to automatically close the opened files when the program terminates, your program will be safe.

➔ For more information, refer to [8. 5 "I/O System" on page 8-15](#).

3. 8. 2 Standard I/O and VKT

In the hybrid controller, the standard input, output, and error output of a C program are assigned to the console terminal of VxWorks. As a result, when a standard I/O function such as printf() is used, its output is not displayed on VKT. To perform I/O operation on the VKT, use the dedicated functions listed in Table 3-18.

Table 3-20 I/O Functions for VKT

Function	Explanation
AT_printf()	Formatted output
AT_scanf()	Formatted input
AT_puts()	One-line output
AT_gets()	One-line input
AT_putchar()	One-character output
AT_getchar()	One-character input

These functions are named by adding AT_ prior to ANSI-C standard functions and are used to perform I/O operations for the VKT with the same arguments used for the corresponding ANSI-C standard functions.

➔ For more information on the low-level VKT I/O, refer to [8. 5. 5 "Controller Terminal \(VKT\) I/O" on page 8-18](#).

3. 9 Added or Modified ATL Statements

The following is a description of added or modified ATL statements. These ATL statement actions depend on the controller operating mode.

3. 9. 1 Added ATL Statements

This section explains ATL statements added to the hybrid controller.

3. 9. 1. 1 RETURNC Statement

In the hybrid mode, this statement returns the program control from the ATL program currently being executed by the ATL_TEST command of the ATL_exec() function to the C program. When the ATL mode is selected, this statement is ignored. (In other words, no operation (NOP) is assumed.)

RETURNC

3. 9. 2 Modified ATL Statements

This section explains how ATL statements are modified. The modifications explained in this section only take effect in hybrid mode. In the ATL mode, operation occurs as described in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498 \)](#).

3. 9. 2. 1 GOTO PRO Statement

A runtime error occurs if a GOTO PRO statement is executed in an ATL program called from a C program in the hybrid mode.

3. 9. 2. 2 GOTO CONTINUE Statement

The operation of this statement varies according to the operating mode as follows:

- When no internal subroutine is nested within the ATL program
Program control is returned to the caller of the ATL_exec() or ATL_proc() function in the hybrid mode. When the ATL mode is selected, this statement is ignored. (In other words, no operation (NOP) is assumed.)
- When an internal subroutine is nested within the ATL program
Program control is always returned to the caller of the internal subroutine regardless of the operating mode.

3. 9. 2. 3 ONRTN Statement

The ONRTN statement in an ATL program called from the synchronous event handler is ignored in the hybrid mode, (In other words, no operation (NOP) is assumed.

3.10 Control Box and O Command

This section explains the operations of the control box and the O command in hybrid mode by notifying differences from ATL mode.

- ➔ For more information on the control box, refer to [T5500 Series Memory Test System Control Box/Trigger Panel User's Manual\(No. : 8256298\)](#).
- ➔ For more information on the O command, refer to [T5500 Series Controller Commands Reference Manual\(No. : 8256297\)](#).

3.10. 1 Program Name and Test Number

In the hybrid mode, a program to be executed by the start instruction is set by the /HYBRID utility program, not by the control box or the O command.

The ATL-separated program displays the loaded ATL test plan program name in the control box when the C program executes the ATL_init() function. The ATL-mixed program displays the program name that was specified in #pragma atl name or the program name that was automatically assigned by the compiler (if no program name is specified in #pragma atl name) in the control box when the ATL_init2() function is executed.

If a TEST statement is executed in an ATL program that has been executed by using the ATL_exec() or ATL_proc() function, the test number is also displayed in the control box.

3.10. 2 Test Mode

The settings of the test mode are processed in the same way as for the ATL mode (refer to [Figure 3-12](#) and [Figure 3-13](#)).

However, for the ATL_TEST command of the ATL_exec() function, when the program control is returned to the C program by the RETURNC statement before the next TEST statement is executed, the MANUAL mode and MANUAL RECYCLE mode executed by the TEST statement have no effect. To perform the MANUAL mode test or the MANUAL RECYCLE mode test, write a dummy TEST statement right before the RETURNC statement as shown in [Figure 3-14](#).

Figure 3-12 Operations in MANUAL Mode

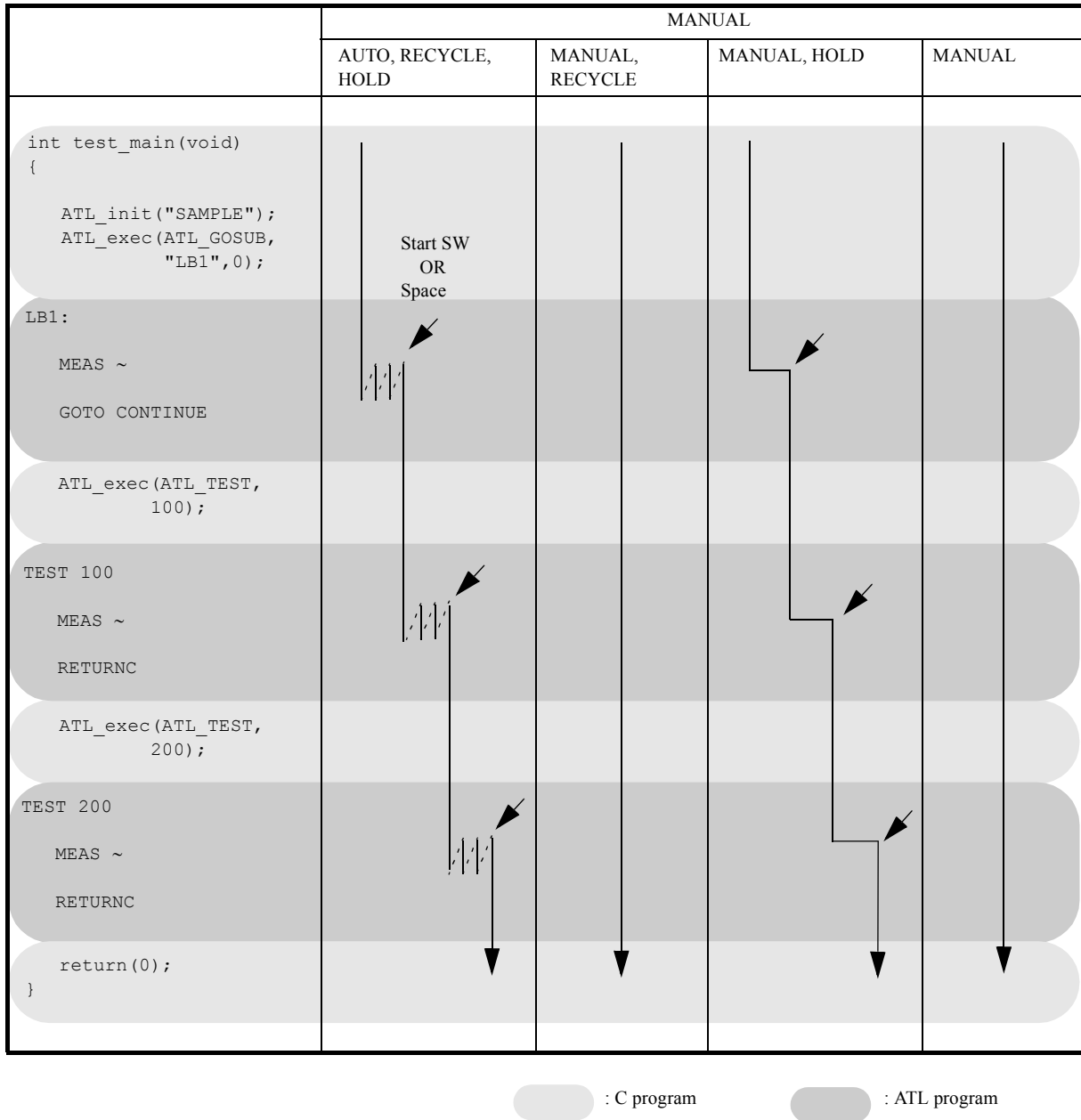


Figure 3-13 Operations in the AUTO Mode

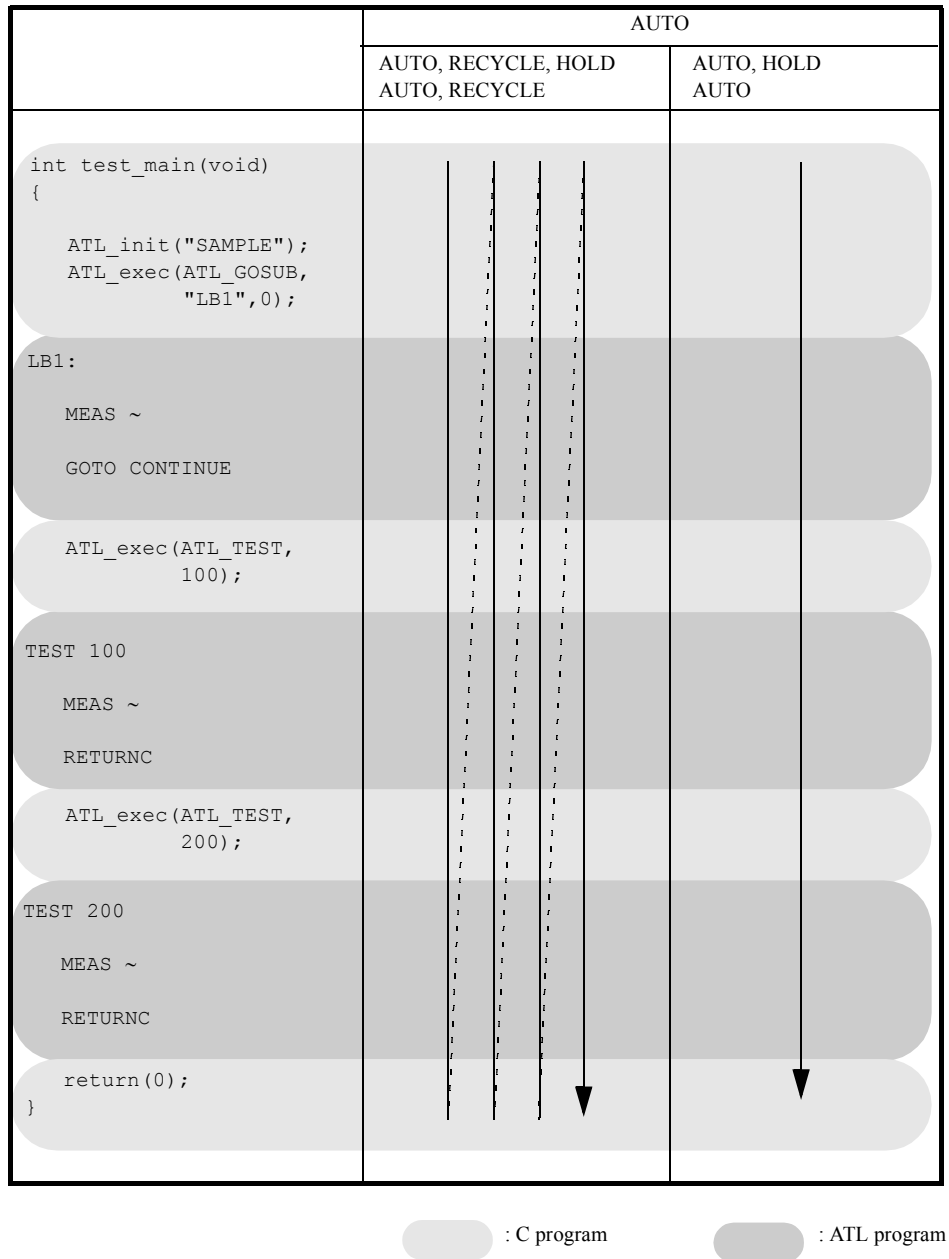
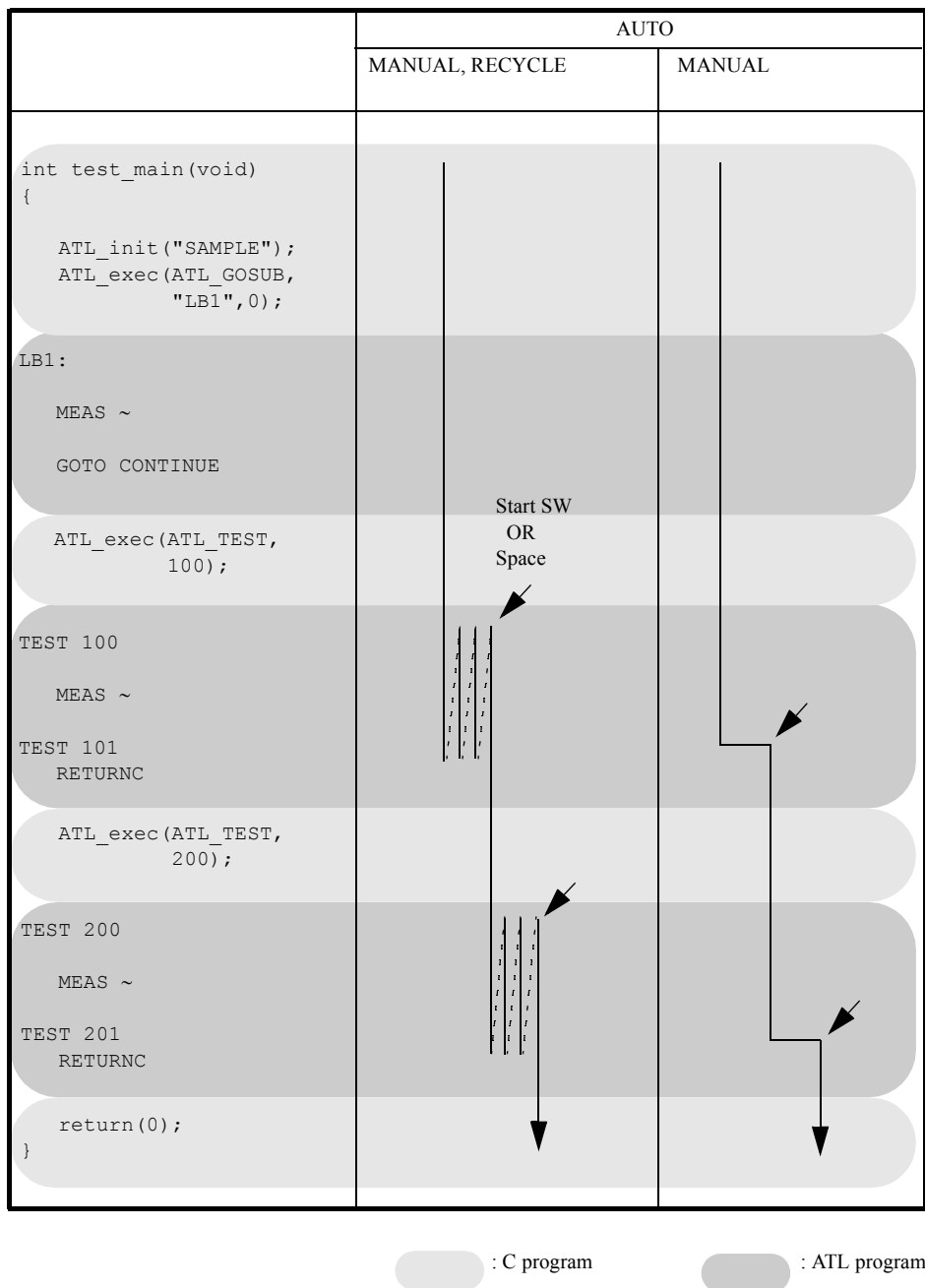


Figure 3-14 Tests in the MANUAL and MANUAL RECYCLE Modes



4. */HYBRID - Setting the Controller Operating Mode*

This chapter describes the /HYBRID utility program used to set the controller operating mode.

4. 1 Overview

The hybrid controller uses two operating modes (an ATL mode and a hybrid mode). The controller executes an ATL test plan program as the main program in the ATL mode and a C program as the main program in the hybrid mode. The /HYBRID utility program is used to set the operating mode and has the following functions:

- Setting the operating mode of the controller.
- Setting and updating C program for hybrid mode.

This utility program allows all conditions to be set directly using command arguments or to be set interactively.

4. 2 Syntax

When the controller is active, enter the command from the VKT as follows.

```
/HYBRID[Station number][Operating mode][Module name],
```

[]: Omissible

Station number

Specify the target station number to set the conditions. If this specification is omitted, the interactive setting screen is displayed. One of the following values can be specified after the delimiter '/':

1	Sets conditions for Station 1.
2	Sets conditions for Station 2.
A	Sets conditions for all stations.

Operating mode

Set the operating mode by specifying whether to enable hybrid mode or not. If this specification is omitted, the interactive setting screen is displayed. One of the following values can be specified after the delimiter '/':

E	(Enable) Enables hybrid mode.
S	(Enable and Suspend) Enables hybrid mode. This mode is used to debug a C program and to stop (break) execution at the beginning of the program.
D	(Disable) Disables hybrid mode (sets the ATL mode).

Module name

If E or S is specified as the operating mode, specify the object file name (module name) of a C program to execute. If this specification is omitted, the interactive setting screen is displayed. If D is specified as the operating mode, the module name specification is ignored.

A maximum of 128 characters can be used to specify a module name. A module name can include a file system device name and a path name.

➔ For more information on the file system device, refer to [8. 5 "I/O System" on page 8-15](#).

Up to 98 characters out of the 128 characters can be used to specify a file name other than the file system device name and the path name in the module name. The module names is case sensitive.

It must be the same file name as the one used on the host system, including the extension. However, commas, back slashes '\', and blank characters such as spaces or tabs cannot be used for the module name. Blank characters before a module name are ignored.

The following module name is reserved by the system and cannot be specified:

```
stest.o, clink.o, vxwrapper.o
```

4. 3 Function

The /HYBRID utility program sets the operating mode for the controller. If hybrid mode is enabled, it also sets a C program to be executed. The operating mode and the module name can be set for each station. The user can specify the station number, the operating mode, and the module name by either using command arguments or interactively. An explanation to specify interactively is described later.

If A is specified for a station number directly using a command argument, the operating mode and the module name become common for all stations. To change the settings for each station, set the arguments individually for each station or specify A for station number in the interactive setting screen.

If E or S is specified as the operating mode, the C program module specified for the module name is loaded from the host system disk into the tester processor.

Tip

If a file system device name is not specified in the module name setting, the module is searched from the NFS-mounted file system of the host. Therefore, the directory or file system containing the module should be NFS-exported to the tester processor.

➔ For more information, refer to [8. 5. 3 "NFS Devices" on page 8-16](#).

The module already loaded on the tester processor is unloaded. The test_main() function should be defined in the specified C program module. Operating mode S is used to debug the C program with a debugger. When execution starts, the program stops (breaks) at the beginning of the test_main() function.

➔ For more information on the method of debugging a C program, refer to [7. "C Program Development Environment" on page 7-1](#).

If an error occurs, an error message is displayed and the specified conditions are not validated.

The values set by this utility are saved on the host system disk and automatically recovered the next time that the controller starts. If the host system disk has no information about this utility (that is, it has never been used before), the controller is set to the ATL mode.

4. 4 Interactive Setting

If you do not specify arguments for the commands required for the /HYBRID utility program, you can set unspecified items in an interactive manner. [Figure 4-1](#) shows the configuration of the setting screen.

Figure 4-1 Configuration of the Setting Screen

HYBRID CONTROLLER SETUP UTILITY

STATION NO. = 1 (a) Station number specification area

ORIGINAL SETTING

HYBRID MODE SWITCH : D (b) Current setting display area

OBJECT MODULE NAME : (none)

NEW SETTING

HYBRID MODE SWITCH : (c) New setting area

OBJECT MODULE NAME :

- | | | |
|-----|------------------------------------|--|
| (a) | Station number specification area: | Area where the number of the station to be set up is specified or displayed. |
| (b) | Current setting display area: | Area where current settings are displayed |
| (c) | New setting area: | Area for setting new conditions |

Station number setting and display

If a station number is not specified in the command argument, the following input screen is displayed. Enter a target station number at the cursor position.

Figure 4-2 Setting Screen of Station Number

HYBRID CONTROLLER SETUP UTILITY

STATION NO. (-)

Cursor position

The value 1, 2, or A can be specified as the station number. When A is specified, "ALL" is displayed. Once the station number has been set, the current setting display area and the new setting area are displayed.

If a station number is specified in the command argument, this area displays the specified station number and does not accept input.

Current setting display

The original setting display area displays the current operating mode and module name. If the operating mode is set to D, "(none)" is displayed in the module name area.

Figure 4-3 Setting Screen of Current Setting

ORIGINAL SETTING		
HYBRID MODE SWITCH	: D	 Operating mode
OBJECT MODULE NAME	: (none)	 Module name

Operating mode setting and display

If an operating mode is not specified in the command argument, the following operating mode input screen is displayed in the new setting area. Enter an operating mode at the cursor position and press the Return key.

Figure 4-4 Setting Screen of Operating Mode

NEW SETTING		
HYBRID MODE SWITCH	: 	Cursor position
E:Enable S:Enable & Suspend(for Debug) D:Disable		

The value E, S, or D can be specified as the operating mode. If the Return key is pressed without making any changes, the current operating mode setting is used. Once the operating mode has been determined, the module name setting area is displayed.

If an operating mode is specified in the command argument, this area displays the specified operating mode and does not accept input.

Module name setting and display

If D is specified as the operating mode, "(none)" is displayed in the module name area and the setup is completed.

If the value E or S is specified as operating mode but the module name is not specified in the command argument, the following module name input screen is displayed. Enter a module name at the cursor position and press the Return key.

Figure 4-5 Setting Screen of Module Name

NEW SETTING		
HYBRID MODE SWITCH	: E	
OBJECT MODULE NAME	: 	Cursor position

The module name specification method is the same as the specification method of the command argument.

➔ For more information, refer to [4. 2 "Syntax" on page 4-2](#).

If the Return key is pressed without making any changes, the current module setting is used.

If a module name is specified by a command argument, this area displays the specified module name and does not accept input.

Once the module name has been determined, the setup completes.

Settings when all stations are specified

If A is specified for the station number, a setting screen is displayed for Station 1 and then for Station 2.

Figure 4-6 Setting Screens When All Stations Are Specified

```
HYBRID CONTROLLER SETUP UTILITY
STATION NO.= ALL

*** STATION NO.=1 SETTING ***

ORIGINAL SETTING
  HYBRID MODE SWITCH   : D
  OBJECT MODULE NAME   : (none)
-----
NEW SETTING
  HYBRID MODE SWITCH   : E
  OBJECT MODULE NAME   : sample1.o:

*** STATION NO.=2 SETTING ***

ORIGINAL SETTING
  HYBRID MODE SWITCH   : D
  OBJECT MODULE NAME   : (none)
-----
NEW SETTING
  HYBRID MODE SWITCH   : S
  OBJECT MODULE NAME   : sample2.o
```

Each station is set in the same way as when a station number is set individually.

4. 5 Error Message

If an error occurs during the /test_main utility program execution, an error message is displayed. [4. 5. 1 "/HYBRID Error Messages and Causes" on page 4-8](#) lists the error messages and their causes.

4. 5. 1 /HYBRID Error Messages and Causes

MEMORY MODULE FOR HYBRID IS NOT INSTALLED

The required memory has not been added to the tester processor.
Add the required memory to the tester processor.

NOT AVAILABLE ON THIS TP SYSTEM

This system cannot execute the hybrid controller.
Check the system operating conditions.

ETHERNET CONNECTION PROBLEM

The Ethernet connection between the host system and the tester processor is faulty.
Check the system operating conditions.

LIB HYBRID MUST BE CALLED FROM VKT

This utility program cannot be called from a program.
Execute from VKT.

BAD HYBRID MODE SWITCH

The operating mode specification is not correct.
Check the settings of the operation mode.

BAD OBJECT MODULE NAME

The module name specification is not correct.
Check the specification conditions for the module name.

FAILED TO UNLOAD PREVIOUS MODULE

The pre-loaded module could not be unloaded.
The loaded module may be using a function that is not allowed.

➔ For more information refer to [8. 4. 4 "Limitations: Restricted VxWorks Functions" on page 8-12](#).

FAILED TO LOAD SPECIFIED MODULE

The specified module could not be loaded.
Make sure the specified file exists.

SYMBOL 'test_main' NOT FOUND

The test_main() function is not defined in the specified module.
A program with no test_main() function cannot be executed.

FAILED TO CHECKOUT LICENSE

A license has not been given to the Hybrid Controller.

CANNOT EXECUTE ON TESTING

This utility program cannot be executed during testing.

FAILED TO LOAD USER LIBRARY

Library module specified in the main module could not be loaded.

5. /test_main - Program Execution

This chapter describes the /test_main utility program used to execute the C program without using a program execution start instruction ^{*1}.

5. 1 Syntax

When the controller is active, enter the command from the VKT as follows.

```
/test_main␣      (a)  
/test_main/1␣    (b)  
/test_main/2␣    (c)
```

These input characters are not case sensitive. Thus, for example, /TEST_MAIN is also effective.

(a) Executes a C program on the current station ^{*2}.

(b) Executes a C program on Station 1.

(c) Executes a C program on Station 2.

**1 A program execution start instruction is pressing the space key from the VKT, pressing the START switch from the control box (C-BOX), or making a start request from the GPIB (handler or prober).*

**2 The current station refers to the station used by the most recent program. If no programs have been executed, station 1 is treated as the current station.*

5. 2 Function

When the operating mode for the specified station is in hybrid mode, this utility executes the C program set for the station.

Before executing this utility, set the target station to hybrid mode using the /HYBRID utility program.

5. 3 Error Message

If an error occurs during the utility program execution, an error message is displayed. The following is a list of the error messages and their causes:

COMMAND ERROR !!

The station number specification is incorrect.

COMMAND ERROR !!

SET /HYBRID ENABLE

The target station is not in hybrid mode.

Set the operating mode to hybrid mode using the /HYBRID utility program.

/test_main or /TEST_MAIN MUST BE CALLED FROM VKT

This utility program cannot be called from a program.

Execute from VKT.

/test_main or /TEST_MAIN CANNOT EXECUTE ON TESTING

This utility program cannot be executed during testing.

6. Programming

This chapter explains the information required to write a C program for the hybrid controller, provides programming advice and notes, and then shows examples of the device test program.

6. 1 Header File

When the "AT/hybrid.h" header file is included in a C source program, the hybrid controller functions can be used. The header file, which is related to the hybrid controller, is located in the system directory which is specified by the AT_ASYSTEM environment variable.

The header file "AT/hybrid.h" excludes header files for the ATL-mixed program which are described later and includes all header files that are related to the hybrid controller. Therefore, most hybrid controller functions can be used by describing the following preprocessor instruction at the beginning of a C source program.

```
#include "AT/hybrid.h"
```

In addition, to configure a program by using the ATL-mixed method, "AT/at1.h" must be included.

```
#include "AT/at1.h"
```

Only the header files which are required to use the hybrid controller functions are indirectly included with the above header files. ANSI C (C++) standard header files, POSIX header files, and VxWorks header files are still unavailable because they are unrelated to the hybrid controller functions. To use these header files, individually set them, as required. Note that the head file names must be enclosed in double quotation marks ("), not enclosed in '<' and '>'. This allows the the C compiler to search for the specified file from the VxWorks Include directory, not the host system Include directory.

For example, to use strcmp() in the C standard library, specify the ANSI C standard header file string.h as shown below.

```
#include "AT/hybrid.h"    /* hybrid controller standard header */  
#include "string.h"       /* ANSI string header */
```

6. 2 Entry Point

In the hybrid controller, a process of a C program always begins with the `test_main()` function. This function must be defined in a user C program. It is used instead of the `main()` function in an ordinary C program, and follows the syntax shown below.

```
int test_main(void);
```

The return value of this function is interpreted by the system program and used for the next program.

➔ For more information, refer to [3. 3. 2 "Program Termination" on page 3-4](#).

6. 3 Data Initialization

The initial values assigned to static and external variables in a user C program are applied when the C program is loaded in the memory of the tester processor. Since the C program is not loaded again for each execution, however, the last assigned value is used for the second and subsequent executions. For example, assume the program shown below.

◆ External Variables That Are Not Initialized

```
#include "AT/hybrid.h"

static int var = 0;

int test_main(void)
{
    AT_printf("var = %d\n", var);
    var++;

    return(0);
}

(Execution result)
H> var = 0
H> var = 1
H> var = 2
:
```

Each time this program is executed, the displayed variable var is incremented, and the initial value coded in the C program is assigned at the first execution only.

As shown in this example, data to be initialized each time a C program is executed must be explicitly initialized in the program code; this data cannot be initialized simply by assigning the initial value at declaration of the variable.

6. 4 Recommending the Use of Function Prototypes

According to C regulations, a function prototype does not need to be contained in the function declaration. However, when a function prototype is used, the matching of argument types is checked for each function call at compilation; accordingly, coding errors can be detected at an early stage.

For example, in the coding titled ["Not Using the Function Prototype \(No Errors Occur.\)" on page 6-4](#), the `foo()` function is written to receive two arguments. And the function expects that the type of the arguments is `double`. However, `test_main()` calls the `foo()` function with only a single `int`-type argument. Nevertheless, this code develops no errors during compilation. This is because the code is correct in syntax when the function prototype is not used. In addition, when this program is executed, the argument values acquired by the `foo()` function become undefined.

◆ Not Using the Function Prototype (No Errors Occur.)

```
#include "AT/hybrid.h"

static void foo();

int test_main(void)
{
    foo(1);
    return(0);
}

static void foo(arg1, arg2)
double arg1;
double arg2;
{
    AT_printf("arg1=%d\n", arg1);
    AT_printf("arg2=%d\n", arg2);
}
```

When the function prototype is used, this code causes an error at compilation; as a result, the coding error can be detected before the program is executed. Moreover, when the type of an argument does not match the function prototype, the argument is converted to a type declared in the function prototype to be transferred. In this example, `int`-type data assigned to the arguments is converted to the `double` type and transferred to the `foo()` function.

◆ Using the Function Prototype (A Compilation Error Can Be Detected.)

```
#include "AT/hybrid.h"

static void foo(double, double);

int test_main(void)
{
    foo(1);
    return(0);
}

static void foo(double arg1, double arg2)
{
    AT_printf("arg1=%d\n", arg1);9
    AT_printf("arg2=%d\n", arg2);
}
```

6. 5 Library Functions and Error Processing

Most library functions use a return value to report the occurrence of an error to the caller. If the error is ignored without a justifiable reason and the process is continued, the program may not function correctly. When using a library function that may return an error, be sure to evaluate the return value of the function. However, the return value of the function can be ignored when it is certain that no error will occur in the function or that an error will not affect the execution of the program.

For example, if a runtime error or system error occurs while an ATL program is being executed with the ATL_exec() function, an error message is displayed on VKT, but the execution of the C program does not stop. In this case, the ATL_exec() function returns with an error; accordingly, evaluate the return value in the program and decide whether program execution is to be continued.

◆ Error Processing of a Function

```
#include "AT/hybrid.h"

int test_main(void)
{
    if(ATL_init("SAMPLE") != 0){
        AT_printf("Error: ATL_init\n");
        exit(1);
    }
    if(ATL_exec(ATL_GOSUB, "LB1", 0) != 0){
        AT_printf("Error: ATL_exec: LB1\n");
        exit(2);
    }
    :
    :
    return(0);
}
```

6. 6 Problems Particular to VxWorks

For programs being run under VxWorks, the user must take into account some points that don't need to be considered when using an ordinary UNIX program. This section describes these points.

6. 6. 1 Releasing Resources

As described in [3. "System Basis" on page 3-1](#), "SYSTEM BASICS," the hybrid controller executes a user program as an independent task of VxWorks. Differently from the concept of the UNIX process, system resources reserved by the VxWorks task are not released even after the task ends, except in the cases described below. The term "system resources" refers here to files, that is, file descriptors linked to a specific device via `open()`, and dynamically allocated memory. This also includes files and memory reserved indirectly by other functions. For example, `fopen()` and `socket()` use system resources because they call `open()` internally. Also, VxWorks functions with `Create` (such as by `semBCreate()`) use system resources because they dynamically allocate memory in most cases.

The system resources reserved directly using one of the following functions within the user program are released by the controller after the user program ends:

<code>open()</code>	<code>creat()</code>	<code>fopen()</code>	<code>freopen()</code>	<code>socket()</code>
<code>malloc()</code>	<code>calloc()</code>	<code>realloc()</code>	<code>memalign()</code>	<code>operator new(C++)</code>
All library functions of the hybrid controller				

The system resources reserved using a function other than the above functions must be released on the responsibility of the user program. System resources are limited; if resources are used but then never released, the user program and system program might malfunction.

To securely release system resources, use the `AT_exit()` function to register a function that is called at program end and releases all system resources. The registered function is called even if the program is forcibly terminated.

➔ For more information, refer to [3. 3. 4 "Releasing Resources" on page 3-5](#).

6. 6. 2 The Scope of Symbols with External Linkage

In VxWorks, symbols with external linkage defined in a program (external symbol), that is, external variables and external functions not declared as static, are registered in a system symbol table when the program is loaded, and their scope spans the entire system. As a result, external symbols defined in a user program may be in contention with those defined in the system. In this case, the system program may reference the external symbols defined in a user program, and may cease to function normally.

For example, when a function having the same name as `printf()` of the C standard library is defined in a user program, the system program using the `printf()` function may reference the `printf()` function defined by the user. If the latter `printf()` function performs an unspecified process, the system program ceases to function normally.

To prevent this problem, create the program taking note of the following:

- External symbol naming rule particular to the controller

One of the prefixes shown below is always applied to an external symbol defined by the tester processor system program. As a result, do not define an external symbol with any of these prefixes in a user program.

`tp, tbus, St, stest, at, AT`

This rule does not apply to external symbols defined by VxWorks. Whether or not the same external symbol as a VxWorks external symbol is defined can be checked with the symbol check command described below.

- Using the symbol check command (`atsymchk`) for the preliminary check

This command checks whether an external symbol defined in a user program duplicates one defined by the system (VxWorks or tester processor system program). Execute this command when the program is being compiled.

➔ For more information on the symbol check command, refer to [7. "C Program Development Environment" on page 7-1](#).

6. 6. 3 Handling Uninitialized External Variables

In C, an initial value need not be assigned to external variables. Also, external variables with the same name may exist in multiple files without having been initialized nor declared as external. In the hybrid controller, however, external variables without external declaration cannot exist in multiple files.

6. 6. 4 Recommended Standard and Compatibility

We cannot guarantee that VxWorks will continue to be used in the future for the hybrid controller. As a result, the user must consider compatibility between source programs in the programming stage. Users are requested to write a program in such a way that the source program can be executed even after migrating to a different platform by simply recompiling the source program for the new platform.

For this reason, you should only use C (C++) standard libraries and POSIX system interfaces, without using the proprietary VxWorks interfaces. In addition, the syntax of the C language (C++ language) should conform to the ANSI C language (C++ language) standard. Do not use any extension functions that are unique implementations of the compiler.

6. 7 Program Example

In this section, a program example for 64 device simultaneous measurement of 128 MB SDRAM using the T5375 memory test system is described. One item each for DC parametric test and functional test is used in the program to simplify the test.

The program is configured with the four programs.

- main64.c
C program
- s64samp.asc
Socket program.
- p64samp.asc
Test plan program.
- mpsdram.asc
Pattern program.

◆ C Program Overview

TEST_COND is a structure to store the test conditions for each test item. It is used as a static variable in a test program, because each test condition is not changed during program execution. The test conditions for a DC parametric measurement is stored into a variable dct_item of structure TEST_COND. The test conditions for a functional test are stored into variable fct_item of structure TEST_COND.

On the function testDC() call, each condition specified in dct_item is set in loop and the test will be executed. If the DUT is failed, then it is eliminated from the target tests in the event handler.

On the function testFunc() call, each condition specified in fct_item is set in loop and the test will be executed. The function test result is read out from the AFM, and if the fail count of the DUT is larger than 100, then it is eliminated from the target tests.

Program main64.c

```

/* main64.c
 *
 * This is a sample program for Hybrid Controller to measure 128M SDRAM
 * device on T5375 memory test system.
 *
 * The measurement flow is as follows;
 *     1. Contact check
 *     2. DC parametric test and fail DUT rejection.
 *     3. Dynamic functional test, read results from AFM, and reject
 *        DUT with fail count more than 100.
 */
#include "AT/hybrid.h"
#include "string.h"
#include "errno.h"

/*****
 * Data types
 */

/* calibration condition */
typedef struct {
    char      *calfile;           /* calibration data filename */
    char      *cond;             /* condition */
} CALB_COND;

/* AFM conditions */
typedef struct {
    int          modcnt;          /* number of AFM action mode */
    int          pgssel;         /* pgssel */
    struct fm_rwparam  fmparam;  /* static AFM parameters */
} AFM_COND;

/* test conditions */
typedef struct {
    char      *vivo;             /* VI/VO conditions */
    char      *pin;             /* PIN conditions */
    char      *timing;          /* timing conditions */
    CALB_COND *calb;            /* calibration condition */
    int       testno;           /* test no. (for DC test) */
    char      *label;          /* label name (for FUNC test) */
    AFM_COND  *afm_cond;       /* AFM setting (for FUNC test) */
} TEST_COND;

```

```

/* obtaining number of elements in an array */
#define ARRAY_SZ(p)      (sizeof(p)/sizeof(p[0]))

/*****
/ * Static test specifications
/ *
/ */

/* ATL program name */
static char ATL_PRONAME[] = "P64SAMP";

/* misc subroutines */
static char PNLDEF[]      = "PNLDEF";      /* defining pinlist */
static char CNTCK[]      = "CNTCK";      /* contact check */
static char TINT[]       = "TINT";        /* first calibration */

/* subroutine for random R/W test */
static char TRNDRW[]     = "TRNDRW";

/* calibration conditions */
static CALB_COND calb1_norm = { "ClMA", "NORMAL" };
static CALB_COND calb1_xfer = { "ClMA", "XFER" };
/* AFM condition for RANDOM R/W test */
static AFM_COND afm_cond1 = {
    1, 1, { { FM_APLINK_XY, 0x0, 0x3fff, 0, 0x1ff }, 0, FM_ORDATA }
};

/* subroutines to set VI/VO conditions */
static char VLFT[]       = "VLFT";
static char VILO[]       = "VILO";

/* subroutines to set PIN conditions */
static char PLFT[]       = "PLFT";
static char PILO[]       = "PILO";

/* subroutines to set timing conditions */
static char TLFT[]       = "TLFT";
static char TILO[]       = "TILO";

/* DC parametric test items */
static TEST_COND dct_item[] = {
    /* ILI */
    { NULL, NULL, NULL, NULL, 100 },
    /* ILO */
    { VILO, PILO, TILO, &calb1_xfer, 200 }
};

```

```

/* Dynamic functional test items */
static TEST_COND fct_item[] = {
    /* RANDOM R/W */
    { VLFT, PLFT, TLFT, &calb1_xfer, 0, TRNDRW, &afm_cond1 }
};

/* for contact check output */
static char dut1_up[] = "66666555555555544444444443333333 <--- DUT NO.";
static char dut1_lo[] = "43210987654321098765432109876543 <--- DUT NO.";
static char dut2_up[] = "333222222222211111111111000000000 <--- DUT NO.";
static char dut2_lo[] = "21098765432109876543210987654321 <--- DUT NO.";

/*****
 * Forward declarations
 *
 */
static int      contactCheck(void);
static int      testDC(int, TEST_COND *);
static void     DCTF_rejection(int);
static int      testFunc(int, TEST_COND *);
static int      setViVo(TEST_COND *);
static int      setPin(TEST_COND *);
static int      setTiming(TEST_COND *);
static int      calibration(TEST_COND *);
static char *   uint32toBinString(uint32_t, char *);

/*****
 * Main program
 *
 */

/* test_main() - the main entry of the test program */
int
test_main(void)
{
    /* load ATL program */
    if(ATL_init(ATL_PRONAME) != 0){
        AT_printf("ATL_init(): %s: %s\n", ATL_PRONAME, AT_strerror(errno));
        return(1);
    }

    /* import PINLIST definitions */
    if(ATL_exec(ATL_GOSUB, PNLDEF, 0) != 0){
        AT_printf("ATL_exec(): %s: %s\n", PNLDEF, AT_strerror(errno));
        return(1);
    }
}

```

```

/* contact check */
if(contactCheck() != 0)
    return(1);

/* collect calibration data */
if(ATL_exec(ATL_GOSUB, TINT, 0) != 0){
    AT_printf("ATL_exec(): %s: %s\n", TINT, AT_strerror(errno));
    return(1);
}

if(ATL_exec(ATL_CALL, "CALB", 2,
    STRING(calb1_norm.calfile, strlen(calb1_norm.calfile)),
    STRING(calb1_norm.cond, strlen(calb1_norm.cond))) != 0){
    AT_printf("ATL_exec(): CALB: %s\n", AT_strerror(errno));
    return(1);
}

/* DC parametric test */
if(testDC(ARRAY_SZ(dct_item), dct_item) != 0)
    return(1);

/* Dynamic functional test */
if(testFunc(ARRAY_SZ(fct_item), fct_item) != 0)
    return(1);

return(0);
}

/* contactCheck() - perform contact check */
static int
contactCheck(void)
{
    uint32_t    gfdut[4];
    char        bc[33];
    int         stat;

    AT_printf("CONTACT CHECK\n\n");

    if(ATL_exec(ATL_GOSUB, CNTCK, 0) != 0){
        AT_perror("ATL_exec(): CNTCHK");
        return(-1);
    }

    /* read fail dut */
    if(AT_read_gfdut(4, gfdut) != 0){
        AT_perror("AT_read_gfdut()");
        return(-1);
    }
}

```

```

/* display results */
if(gfdut[0] != 0 || gfdut[2] != 0){
    AT_printf("\n");
    AT_printf("%38c%s\n", ' ', dut1_up);
    AT_printf("%38c%s\n", ' ', dut1_lo);
    AT_printf("\tMISS CONTACT !\t\tGFDUT %s\n\n",
        uint32toBinString(gfdut[2], bc));
    AT_printf("%38c%s\n", ' ', dut2_up);
    AT_printf("%38c%s\n", ' ', dut2_lo);
    AT_printf("\tMISS CONTACT !\t\tGFDUT %s\n\n",
        uint32toBinString(gfdut[0], bc));

    /* reject fail DUTs */
    stat = AT_set_rejectionm(1, ARRAY_SZ(gfdut), gfdut);
    if(stat < 0){ /* error */
        AT_perror("AT_set_rejectionm()");
        return(-1);
    }else if(stat == 1){ /* all rejected */
        AT_printf("ALL DUT REJECTED!\n");
        exit(0);
    }
}

return(0);
}

/* testDC() - DC parametric test */
static int
testDC(int nitem, TEST_COND *tcond)
{
    int i;
    TEST_COND *tc;
    AT_printf("DC TEST\n\n");

    /* DC test fail event handler */
    AT_event(ATEV_DCTF, DCTF_rejection);

    for(i = 0, tc = tcond; i < nitem; i++, tc++){
        AT_printf("\tTEST %d\n", tcond->testno);
        if(setViVo(tc) != 0)
            return(-1);
        if(setPin(tc) != 0)
            return(-1);
        if(setTiming(tc) != 0)
            return(-1);
        if(calibration(tc) != 0)
            return(-1);
    }
}

```

```

        /* measure */
        if(ATL_exec(ATL_TEST, tcond->testno) != 0){
            AT_printf("ATL_exec(): TEST %d: %s\n",
                      tcond->testno, AT_strerror(errno));
            return(-1);
        }
    }

    /* reset event handler */
    AT_event(ATEV_DCTF, ATEV_DFL);

    return(0);
}

/* DCTF_rejection() - reject fail DUTs on DC test failure */
static void
DCTF_rejection(int evcode)
{
    int          stat, testno;
    uint32_t     fdut[4];
    char         bc[33];

    /* read fail DUTs */
    if(AT_read_fdut(ARRAY_SZ(fdut), fdut) != 0){
        AT_perror("AT_read_fdut()");
        exit(1);
    }
    if(fdut[0] != 0 || fdut[2] != 0){
        testno = AT_read_test();
        AT_printf("\n");
        AT_printf("%38c%s\n", ' ', dut1_up);
        AT_printf("%38c%s\n", ' ', dut1_lo);
        AT_printf("\t\tFDUT  %s\n\n",
                  testno, uint32toBinString(fdut[2], bc));
        AT_printf("%38c%s\n", ' ', dut2_up);
        AT_printf("%38c%s\n", ' ', dut2_lo);
        AT_printf("\t\tFDUT  %s\n\n",
                  testno, uint32toBinString(fdut[0], bc));
    }

    /* reject fail DUTs */
    stat = AT_set_rejectionm(1, ARRAY_SZ(fdut), fdut);
    if(stat < 0){ /* error */
        AT_perror("AT_set_rejectionm()");
        exit(1);
    }else if(stat == 1){ /* all rejected */
        AT_printf("ALL DUT REJECTED!\n");
        exit(0);
    }
}

```

```
/* set event handler */
AT_event(ATEV_DCTF, DCTF_rejection);
return;
}

/* testFunc() - Dynamic Functional test */
static int
testFunc(int nitem, TEST_COND *tcond)
{
    int            i;
    TEST_COND      *tc;
    uint32_t        n_fmgrp, n_fmdu;
    uint32_t        confno;
    struct fm_rwparam fmparam;
    uint32_t        dutno[64], j;
    uint32_t        failcnt;

    AT_printf("FUNCTION TEST\n\n");

    for(i = 0, tc = tcond; i < nitem; i++, tc++){

        AT_printf("\tTEST NAME: %s\n\n", tc->label);

        if(setViVo(tc) != 0)
            return(-1);
        if(setPin(tc) != 0)
            return(-1);
        if(setTiming(tc) != 0)
            return(-1);
        if(calibration(tc) != 0)
            return(-1);

        /* read number of DUT groups */
        if(AT_read_afm_confcnt(tc->afm_cond->modcnt, tc->afm_cond->pgsel,
                               1, &n_fmgrp) != 0){
            AT_perror("AT_read_afm_confcnt()");
            return(-1);
        }
        /* read number of DUTs/group */
        if(AT_read_afm_confcnt(tc->afm_cond->modcnt, tc->afm_cond->pgsel,
                               2, &n_fmdu) != 0){
            AT_perror("AT_read_afm_confcnt()");
            return(-1);
        }
    }
}
```



```

/* set AFM access parameters */
memcpy(&fmparam, &tc->afm_cond->fmparam, sizeof(fmparam));

for(confno = 1; confno <= n_fmgrp; confno++){
    /* measure */
    if(ATL_exec(ATL_GOSUB, tcond->label, 1, INTE(&confno)) != 0){
        AT_printf("ATL_exec(): %s: %s\n",
            tcond->label, AT_strerror(errno));
        return(-1);
    }

    /* read assigned dut no. for this AFM config no. */
    if(AT_read_afm_confdat(tc->afm_cond->modcnt,
        tc->afm_cond->pgsel,
        confno, 1, ARRAY_SZ(dutno), dutno) != 0){
        AT_perror("AT_read_afm_confdat()");
        return(-1);
    }
    for(j = 0; j < n_fmddut; j++){
        /* read fail count of dutno[j] */
        fmparam.dutno = dutno[j];
        if(AT_read_afm_failcnt(FM_FTA, &fmparam, 1, &failcnt) != 0){
            AT_perror("AT_read_afm_failcnt()");
            return(-1);
        }
        /* if fail count > 100, reject it */
        if(failcnt > 100){
            AT_printf("\tdUT %2d: FAIL\n", dutno[j]);
            if(AT_set_rejection(0, dutno[j]) != 0){
                AT_perror("AT_set_rejection()");
                return(-1);
            }
        }
    }
}

return(0);
}

```

```
/* setViVo() - setting VI/VO conditions */
static int
setViVo(TEST_COND *tcond)
{
    if(tcond->vivo){
        if(ATL_exec(ATL_GOSUB, tcond->vivo, 0) != 0){
            AT_printf("ATL_exec(): %s: %s\n",
                      tcond->vivo, AT_strerror(errno));
            return(-1);
        }
    }
    return(0);
}

/* setPin() - setting pin conditions */
static int
setPin(TEST_COND *tcond)
{
    if(tcond->pin){
        if(ATL_exec(ATL_GOSUB, tcond->pin, 0) != 0){
            AT_printf("ATL_exec(): %s: %s\n",
                      tcond->pin, AT_strerror(errno));
            return(-1);
        }
    }
    return(0);
}

/* setTiming() - setting timing conditions */
static int
setTiming(TEST_COND *tcond)
{
    if(tcond->timing){
        if(ATL_exec(ATL_GOSUB, tcond->timing, 0) != 0){
            AT_printf("ATL_exec(): %s: %s\n",
                      tcond->timing, AT_strerror(errno));
            return(-1);
        }
    }
    return(0);
}
```

```
/* calibration() - call calibration */
static int
calibration(TEST_COND *tcond)
{
    if(tcond->calb){
        if(ATL_exec(ATL_CALL, "CALB", 2,
                    STRING(tcond->calb->calfile,
                           strlen(tcond->calb->calfile)),
                    STRING(tcond->calb->cond,
                           strlen(tcond->calb->cond))) != 0){
            AT_printf("ATL_exec(): CALB: %s\n", AT_strerror(errno));
            return(-1);
        }
    }
    return(0);
}

/* uint32toBinString() - Convert uint32_t to a string in binary patterns */
static char *
uint32toBinString(uint32_t data, char *buff)
{
    register int      i;
    register char     *p;

    for(i = 0, p = buff; i < 32; i++, p++)
        *p = (char)((data >> (31 - i)) & 0x1) + 0x30;
    *p = '\0';

    return(buff);
}
```

Program s64samp.asc

```

;;=====
;;|| SOCKET S64SAMP ||
;;|| || ||
;;|| SAMPLE SOCKET PROGRAM FOR 128M SDRAM MEASUREMENT ON T5375 ||
;;|| MEMORY TEST SYSTEM. [FULL-TH,64DUT/STN] ||
;;|| || ||
;;=====
SOCKET S64SAMP

; Pin-name Pin-no Tester-channel Pin-no
; VCC = VS1 = VS1 ; 1
; DQ0 = P33 = 33 ; 2
; VCCQ = VS2 = VS2 ; 3
; DQ1 = P34 = 34 ; 4
; DQ2 = P35 = 35 ; 5
; VSSQ = GND = GND ; 6
; DQ3 = P36 = 36 ; 7
; DQ4 = P37 = 37 ; 8
; VCCQ = VS2 = VS2 ; 9
; DQ5 = P38 = 38 ; 10
; DQ6 = P39 = 39 ; 11
; VSSQ = GND = GND ; 12
; DQ7 = P40 = 40 ; 13
; VCC = VS1 = VS1 ; 14
; LDQM = PD15 = 15 ; 15
; WE = PD16 = 16 ; 16
; CAS = PD17 = 17 ; 17
; RAS = PD18 = 18 ; 18
; CS = PD19 = 19 ; 19
; BA0 = PD13 = 13 ; 20
; BA1 = PD14 = 14 ; 21
; A10 = PD11 = 11 ; 22
; A0 = PD1 = 1 ; 23
; A1 = PD2 = 2 ; 24
; A2 = PD3 = 3 ; 25
; A3 = PD4 = 4 ; 26
; VCC = VS1 = VS1 ; 27
; VSS = GND = GND ; 28
; A4 = PD5 = 5 ; 29
; A5 = PD6 = 6 ; 30
; A6 = PD7 = 7 ; 31
; A7 = PD8 = 8 ; 32
; A8 = PD9 = 9 ; 33
; A9 = PD10 = 10 ; 34
; A11 = PD12 = 12 ; 35

```

```

;      NC      = PD20 = 20      ; 36
      CKE      = PD21 = 21      ; 37
      CLK      = PD22 = 22      ; 38
      UDQM     = PD23 = 23      ; 39
;      NC      = PD24 = 24      ; 40
;      VSS     = GND  = GND     ; 41
      DQ8      = P41  = 41      ; 42
;      VCCQ    = VS2  = VS2     ; 43
      DQ9      = P42  = 42      ; 44
      DQ10     = P43  = 43      ; 45
;      VSSQ    = GND  = GND     ; 46
      DQ11     = P44  = 44      ; 47
      DQ12     = P45  = 45      ; 48
;      VCCQ    = VS2  = VS2     ; 49
      DQ13     = P46  = 46      ; 50
      DQ14     = P47  = 47      ; 51
;      VSSQ    = GND  = GND     ; 52
      DQ15     = P48  = 48      ; 53
;      VSS     = GND  = GND     ; 54

;***** T5375 STD-TYPE TEST HEAD *****

      DEFINE MULTIPLE 64DUT 1STN
      BIT = 16
      PPS = 2
      PIN MODE = A
      CHTYPE11      ; xxx I/O

      DUT# = VS1      VS2      PD1-24      P33-48
      DUT1 = PPS97    PPS33    PD1-24E1    P33-48E1
      DUT2 = PPS98    PPS34    PD1-24E2    P97-112E1
      DUT3 = PPS65    PPS1     PD1-24A1    P33-48A1
      DUT4 = PPS66    PPS2     PD1-24A2    P97-112A1
      DUT5 = PPS73    PPS9     PD1-24C1    P33-48C1
      DUT6 = PPS74    PPS10    PD1-24C2    P97-112C1
      DUT7 = PPS105   PPS41    PD1-24G1    P33-48G1
      DUT8 = PPS106   PPS42    PD1-24G2    P97-112G1
      DUT9 = PPS99    PPS35    PD1-24E3    P33-48E3
      DUT10 = PPS100  PPS36    PD1-24E4    P97-112E3
      DUT11 = PPS67   PPS3     PD1-24A3    P33-48A3
      DUT12 = PPS68   PPS4     PD1-24A4    P97-112A3
      DUT13 = PPS75   PPS11    PD1-24C3    P33-48C3
      DUT14 = PPS76   PPS12    PD1-24C4    P97-112C3
      DUT15 = PPS107  PPS43    PD1-24G3    P33-48G3
      DUT16 = PPS108  PPS44    PD1-24G4    P97-112G3
      DUT17 = PPS101  PPS37    PD1-24E5    P33-48E5
      DUT18 = PPS102  PPS38    PD1-24E6    P97-112E5
      DUT19 = PPS69   PPS5     PD1-24A5    P33-48A5
      DUT20 = PPS70   PPS6     PD1-24A6    P97-112A5

```

```

DUT21 = PPS77 PPS13 PD1-24C5 P33-48C5
DUT22 = PPS78 PPS14 PD1-24C6 P97-112C5
DUT23 = PPS109 PPS45 PD1-24G5 P33-48G5
DUT24 = PPS110 PPS46 PD1-24G6 P97-112G5
DUT25 = PPS103 PPS39 PD1-24E7 P33-48E7
DUT26 = PPS104 PPS40 PD1-24E8 P97-112E7
DUT27 = PPS71 PPS7 PD1-24A7 P33-48A7
DUT28 = PPS72 PPS8 PD1-24A8 P97-112A7
DUT29 = PPS79 PPS15 PD1-24C7 P33-48C7
DUT30 = PPS80 PPS16 PD1-24C8 P97-112C7
DUT31 = PPS111 PPS47 PD1-24G7 P33-48G7
DUT32 = PPS112 PPS48 PD1-24G8 P97-112G7
DUT33 = PPS113 PPS49 PD1-24F1 P33-48F1
DUT34 = PPS114 PPS50 PD1-24F2 P97-112F1
DUT35 = PPS81 PPS17 PD1-24B1 P33-48B1
DUT36 = PPS82 PPS18 PD1-24B2 P97-112B1
DUT37 = PPS89 PPS25 PD1-24D1 P33-48D1
DUT38 = PPS90 PPS26 PD1-24D2 P97-112D1
DUT39 = PPS121 PPS57 PD1-24H1 P33-48H1
DUT40 = PPS122 PPS58 PD1-24H2 P97-112H1
DUT41 = PPS115 PPS51 PD1-24F3 P33-48F3
DUT42 = PPS116 PPS52 PD1-24F4 P97-112F3
DUT43 = PPS83 PPS19 PD1-24B3 P33-48B3
DUT44 = PPS84 PPS20 PD1-24B4 P97-112B3
DUT45 = PPS91 PPS27 PD1-24D3 P33-48D3
DUT46 = PPS92 PPS28 PD1-24D4 P97-112D3
DUT47 = PPS123 PPS59 PD1-24H3 P33-48H3
DUT48 = PPS124 PPS60 PD1-24H4 P97-112H3
DUT49 = PPS117 PPS53 PD1-24F5 P33-48F5
DUT50 = PPS118 PPS54 PD1-24F6 P97-112F5
DUT51 = PPS85 PPS21 PD1-24B5 P33-48B5
DUT52 = PPS86 PPS22 PD1-24B6 P97-112B5
DUT53 = PPS93 PPS29 PD1-24D5 P33-48D5
DUT54 = PPS94 PPS30 PD1-24D6 P97-112D5
DUT55 = PPS125 PPS61 PD1-24H5 P33-48H5
DUT56 = PPS126 PPS62 PD1-24H6 P97-112H5
DUT57 = PPS119 PPS55 PD1-24F7 P33-48F7
DUT58 = PPS120 PPS56 PD1-24F8 P97-112F7
DUT59 = PPS87 PPS23 PD1-24B7 P33-48B7
DUT60 = PPS88 PPS24 PD1-24B8 P97-112B7
DUT61 = PPS95 PPS31 PD1-24D7 P33-48D7
DUT62 = PPS96 PPS32 PD1-24D8 P97-112D7
DUT63 = PPS127 PPS63 PD1-24H7 P33-48H7
DUT64 = PPS128 PPS64 PD1-24H8 P97-112H7

```

END

Program p64samp.asc

```

;;=====
;;|| PRO P64SAMP                                     ||
;;||                                     ||
;;|| THIS IS A SAMPLE TEST PLAN PROGRAM FOR HYBRID CONTROLLER TO ||
;;|| MEASURE 128M SDRAM ON T5375 MEMORY TEST SYSTEM.           ||
;;||                                     ||
;;|| FILES:                                                     ||
;;||      S64SAMP.ASC:    FULL-TH,64DUT 1STN SOCKET PROGRAM    ||
;;||      MPSPDRAM.ASC:   PATTERN PROGRAM                      ||
;;||                                     ||
;;=====
PRO P64SAMP S64SAMP

;;=====
;; PINLIST DEFINITIONS
;;=====
PNLDEF:

    PINLIST ADD  = A0,A1,A2,A3,A4,A5,A6,A7,A8,A9
    PINLIST INPN = WE,CAS,RAS,CS,A11,A10,ADD,CKE,CLK,@
                  LDQM,UDQM,BA0,BA1
    PINLIST DQ   = DQ0,DQ1,DQ2,DQ3,DQ4,DQ5,DQ6,DQ7,DQ8,@
                  DQ9,DQ10,DQ11,DQ12,DQ13,DQ14,DQ15
    PINLIST ALPN = INPN,DQ

GOTO CONTINUE

;;=====
;; CONTACT CHECK
;;=====
CNTCK:

    ALPN      = OPEN
    PCON      = 0
    LCON      = 0

    TIME1     = 5.00MS:VS1,VS2
    TIME2     = 5.00MS:IN
    TIME3     = 5.00MS:OTHERS
    SETTLING TIME = 5MS,5MS

    VS1       = OPEN
    VS2       = OPEN
    ISVM      = -10UA,R80UA,M(-6V)
    LIMIT DC  = -0.1V,-2.0V
    MEAS/T DC (ALPN)

GOTO CONTINUE

```

```

;;=====
;; CONDITIONS FOR INITIAL CALIBRATION
;;=====
TINT:

    SEND MPAT MPSDRAM
    PCON      = 3

    TIME1     = 5.00MS:VS1,VS2
    TIME2     = 5.00MS:IN
    TIME3     = 5.00MS:OTHERS

    ALPN      = OPEN

GOTO CONTINUE

;;=====
;; DC PARAMETRIC TEST
;;=====

;***** ILI TEST (INPUT LEAK CURRENT) *****

TEST 100

    VS1       = 3.6V
    VS2       = 3.6V
    IN1       = 3.6V,0.0V

    VSIM      = 3.6V,R16V,M80UA
    LIMIT DC  = 5UA,-5UA

    INPN      = IN1, FIXL
    DQ        = OPEN

    SETTLING TIME = 2MS,0S
    MEAS/T DC (INPN)

RETURNC

```



```

;***** ILO TEST (OUTPUT LEAK CURRENT) *****

TEST 200

    VSIM          = 2.000V,R16V,M80UA
    LIMIT DC      = 5UA,-5UA

    SEND MPAT MPSDRAM
    REG MPAT XT = #02A
    REG MPAT PC = #40

    START MPAT *                      ; MAKE DQ-PIN HI-Z
    WAIT TIME 5.000MS
    MEAS/T DC (DQ)
    STOP MPAT

RETURNC

;;=====
;; DYNAMIC FUNCTIONAL TEST
;;=====

;***** RANDOM R/W TEST *****

TRNRDW:

    ARGUMENT (CFGN)

    IF CFGN=1 THEN ENTRY
        RESET AFM ALL
        AFM ADDRESS = X0-13 Y0-8
    EXIT

    AFM MULTIMODE = CONFIG (CFGN)
    AFM ACTION    = FT>FMX
    RESET AFM FT ALL

TEST 300

    SEND MPAT MSDRAM
    REG MPAT XT = #031                ; CL=3 BL=2
    REG MPAT PC=#0
    MEAS MPAT

GOTO CONTINUE

```

```

;***** ILO TEST (OUTPUT LEAK CURRENT) *****
TEST 200

      VSIM          = 2.000V,R16V,M80UA
      LIMIT DC      = 5UA,-5UA

      SEND MPAT MPSDRAM
      REG MPAT XT = #02A
      REG MPAT PC = #40

      START MPAT *                               ; MAKE DQ-PIN HI-Z
      WAIT TIME 5.000MS
      MEAS/T DC (DQ)
      STOP MPAT

RETURNC

;;=====
;; DYNAMIC FUNCTIONAL TEST
;;=====

;***** RANDOM R/W TEST *****
TRNDRW:
      ARGUMENT(CFGN)

      IF CFGN=1 THEN ENTRY
          RESET AFM ALL
          AFM ADDRESS = X0-13 Y0-8
      EXIT

      AFM MULTIMODE = CONFIG(CFGN)
      AFM ACTION    = FT>FMX
      RESET AFM FT ALL

TEST 300

      SEND MPAT MSDRAM
      REG MPAT XT = #031                               ; CL=3 BL=2
      REG MPAT PC=#0
      MEAS MPAT

GOTO CONTINUE

```

```

;;=====
;; VOLTAGE CONDITIONS
;;=====
VLFT:
    VS1      = 3.3V,R5V
    VS2      = 3.3V,R5V
    PCON     = VS1,VS2
    LCON     = 0
    IN1      = 3.3V,0.0V
    OUT1     = 1.5V,1.5V
    VT1      = 1.5V
    IL1      = -2.0MA,2.0MA
GOTO CONTINUE
VILO:
    VS1      = 3.6V
    VS2      = 3.6V
    PCON     = 0
    LCON     = 0
    IN1      = 3.6V,0.0V
    OUT1     = 2.4V,0.4V
GOTO CONTINUE

;;=====
;; PIN CONDITIONS
;;=====
PLFT:
    CLK      = IN1,  RZX,    BCLK3,CCLK3
    CKE      = IN1,  /RZO,   BCLK4,CCLK4    <C0>
    CS       = IN1,  /RZO,   BCLK6,CCLK6    <C1>
    RAS      = IN1,  /RZO,   BCLK6,CCLK6    <C2>
    CAS      = IN1,  /RZO,   BCLK6,CCLK6    <C3>
    WE       = IN1,  /RZO,   BCLK6,CCLK6    <C4>
    ADD      = IN1,  XOR,ACLK1,BCLK1,CCLK1,PSM, <X0-9,Y0-9>
    A10      = IN1,  XOR,ACLK1,BCLK1,CCLK1,PSM, <X10,C6>
    A11      = IN1,  XOR,ACLK1,BCLK1,CCLK1,    <X11>
    BA0      = IN1,  XOR,ACLK1,BCLK1,CCLK1,    <X12>
    BA1      = IN1,  XOR,ACLK1,BCLK1,CCLK1,    <X13>
    LDQM     = IN1,  NRZB,    BCLK7,        HINIT, <C5>
    UDQM     = IN1,  NRZB,    BCLK7,        HINIT, <C9>
    DQ       = IN1,  XOR,ACLK2,BCLK2,CCLK2,IOC,STRB1,OUT1,@
              EXPDLY2,DRE1,DRENZ,IL1,VT1,<D0-15>
GOTO CONTINUE

```

```

PILO:
    CLK      = IN1,   RZX,      BCLK3,CCLK3
    CKE      = IN1,   /RZO,      BCLK4,CCLK4      <C0>
    CS       = IN1,   /RZO,      BCLK5,CCLK5      <C1>
    RAS      = IN1,   /RZO,      BCLK6,CCLK6      <C2>
    CAS      = IN1,   /RZO,      BCLK6,CCLK6      <C3>
    WE       = IN1,   /RZO,      BCLK6,CCLK6      <C4>
    ADD      = IN1,   XOR,ACLK1,BCLK1,CCLK1,PSM, <X0-9,Y0-9>
    A10      = IN1,   XOR,ACLK1,BCLK1,CCLK1,PSM, <X10,C6>
    A11      = IN1,   XOR,ACLK1,BCLK1,CCLK1,      <X11>
    BA0      = IN1,   XOR,ACLK1,BCLK1,CCLK1,      <X12>
    BA1      = IN1,   XOR,ACLK1,BCLK1,CCLK1,      <X13>
    LDQM     = IN1,   NRZB,      BCLK7,          HINIT, <C5>
    UDQM     = IN1,   NRZB,      BCLK7,          HINIT, <C9>
    DQ       = OPEN

GOTO CONTINUE

;;=====
;; TIMING SETS
;;=====
TLFT:    ; LOOSE FUNCTION
    RATE    = 20.0NS:2,30NS
    BCLK3    = 10.0NS:3                ; CLK
    CCLK3    = 20.0NS:3
    BCLK4    = 5.0NS:3                 ; CKE
    CCLK4    = 12.5NS:3
    BCLK6    = 5.0NS:3                 ; RAS/CAS/WE/CS
    CCLK6    = 12.5NS:3
    ACLK1    = 0.0NS:3                 ; ADDRESS
    BCLK1    = 5.0NS:3
    CCLK1    = 12.5NS:3
    BCLK7    = 0.0NS:3                 ; DQM
    ACLK2    = 0.0NS:3                 ; I/O
    BCLK2    = 5.0NS:3
    CCLK2    = 12.5NS:3
    DREL1    = 0.0NS:3                 ; (DRE)
    DRET1    = 0.0NS:3
    STRB1    = 5.0NS+20NS:2            ; (CPE) STRB + RATE
GOTO CONTINUE

```

```
TILO:
    RATE          = 20.0NS:2
    BCLK3          = 10.0NS:2          ; CLK
    CCLK3          = 20.0NS:2
    BCLK4          = 5.0NS:2          ; CKE
    CCLK4          = 12.5NS:2
    BCLK5          = 5.0NS:2          ; CS
    CCLK5          = 12.5NS:2
    BCLK6          = 5.0NS:2          ; RAS/CAS/WE
    CCLK6          = 12.5NS:2
    ACLK1          = 0.0NS:2          ; ADDRESS
    BCLK1          = 5.0NS:2
    CCLK1          = 12.5NS:2
    BCLK7          = 5.0NS:2          ; DQM
    CCLK7          = 12.5NS:2
    ACLK2          = 0.0NS:2          ; I/O
    BCLK2          = 5.0NS:2
    CCLK2          = 12.5NS:2
    DREL1          = 0.0NS:2          ; (DRE)
    DRET1          = 0.0NS:2
    STRB1          = 5.0NS:2          ; (CPE)
GOTO CONTINUE

END
```

Program mpsdram.asc

```

;;=====
;;|| MPAT MPSDRAM ||
;;|| || ||
;;|| SAMPLE PATTERN PROGRAM FOR 128M SDRAM ||
;;|| || ||
;;=====
MPAT MPSDRAM

;;=====
;; SDEF MNAME      CKE CS  RAS CAS WE  DQM A10
;;=====
SDEF MRS      =      C1 C2  C3  C4              ;MODE REGISTER SET
SDEF REFR     =      C1 C2  C3                  ;AUTO REFRESH
SDEF PRE      =      C1 C2      C4              ;SINGLE BANK PRECHARGE
SDEF PALL     =      C1 C2      C4      C6      ;ALL BANK PRECHARGE
SDEF ACT      =      C1 C2                    ;BANK ACTIVE
SDEF WR       =      C1      C3  C4            ;WRITE
SDEF RD       =      C1      C3              ;READ
SDEF MASK     =              C5      C9      ;DATA MASK/OUTPUT DISABLE

;;=====
;; ADDRESS GENERATION
;;=====
SDEF INIT     = XB<0 YB<0 YC<0 XCS<0 TP<TPH D4<D4B D3<D3B ;INITIALIZE
SDEF COUT     = X<XC Y<YC                          ;CURRENT REGISTER
SDEF TOUT     = XT SCROFF                          ;MODE REGISTER SET
SDEF XINC     = XC<XC+1                          ;ROW ADDRESS INCREMENT
SDEF YINC     = YC<YC+1                          ;COLUMN ADDRESS INCREMENT
SDEF YDI4     = YC<YC+D4                          ;COLUMN ADDRESS (+4) INCREMENT
SDEF YADR     = M2                                ;COLUMN ADDRESS

;;=====
;; REGISTERS
;;=====
REGISTER
XMAX          = #3FFF
YMAX          = #1FF
IDX1          = #0
IDX2          = #3FFD
IDX3          = #1FE
IDX4          = #0
IDX5          = #0
TPH           = #0
D2            = #800
DRE1          = W
CPE1          = R

```

```

;;=====
;; RANDOM READ/WRITE (BL=2,CL=3)
;;=====
MODULE BEGIN

REGISTER
    XMAX      = #3FFF
    YMAX      = #1FF
    IDX1      = #6
    IDX2      = #FE ; (YMAX+1)/2-2
    IDX3      = #3FFE ; (XMAX+1)-2
    IDX4      = #FE ; (YMAX+1)/2-2
    YMASK     = #1FE
    CCD       = #1
    D4B       = #2

START #0      NOP      YADR      MASK INIT      TS1
      NOP      YADR      MASK COUT      TS1
      NOP      YADR PALL  MASK COUT      TS1 ;ALL BANK PRECHARGE
      NOP      YADR      MASK COUT      TS1
      NOP      YADR      MASK COUT      TS1
      L0:      NOP      YADR REFR      COUT      TS1 ;AUTO REFRESH
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      JN11 L0  YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR MRS    TOUT      TS1 ;MODE REGISTER SET
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      ;--- ALL 0 WRITE ---
      L1:      NOP      ACT      COUT      TS1 ;BANK ACTIVE
      NOP      YADR      COUT      TS1
      NOP      YADR WR    W      COUT      TS1 FP1 ;WRITE
      JN12 .-1 YADR      W      COUT      YDI4 TS1 FP1 ;WRITE
      NOP      YADR      COUT      TS1
      NOP      YADR      COUT      TS1
      NOP      YADR PRE    COUT XINC TS1 ;SINGLE BANK PRECHARGE
      JN13 L1  YADR      COUT      TS1

```

```

;--- 0 READ / 1 WRITE ---
L2:   NOP          ACT          COUT          TS1          ;BANK ACTIVE
      NOP          YADR          COUT          TS1
L3:   NOP          YADR RD      R      COUT          TS1 FP1      ;READ
      NOP          YADR          R      COUT          TS1 FP1
      NOP          YADR          COUT          TS1
      NOP          YADR          COUT          TS1
      NOP          YADR          COUT          TS1
      NOP          YADR WR      W      COUT          TS1 FP1 /D    ;WRITE
      NOP          YADR          W      COUT          TS1 FP1 /D
      NOP          YADR          COUT          TS1
      NOP          YADR          COUT          TS1
      JN14 L3      YADR          COUT          YDI4 TS1
      NOP          YADR PRE      COUT XINC TS1      ;SINGLE BANK PRECHARGE
      JN13 L2      YADR          COUT          TS1
;--- 1 READ ---
L4:   NOP          ACT          COUT          TS1          ;BANK ACTIVE
      NOP          YADR          COUT          TS1
      NOP          YADR RD      R      COUT          TS1 FP1 /D    ;READ
      JN12 .-1     YADR          R      COUT          YDI4 TS1 FP1 /D ;READ
      NOP          YADR          COUT          TS1
      NOP          YADR          COUT          TS1
      NOP          YADR PRE      COUT XINC TS1      ;SINGLE BANK PRECHARGE
      JN13 L4      YADR          COUT          TS1
;--- REVERSE ---
      NOP          YADR          COUT          TS1
      JZD L1       YADR          COUT          TS1
      STPS
MODULE END

;=====
;; OUTPUT LEAK
;=====
MODULE BEGIN

START #40
      NOP          YADR          MASK INIT      TS1
      NOP          YADR          MASK COUT      TS1
      NOP          YADR PALL     MASK COUT      TS1      ;ALL BANK PRECHARGE
      NOP          YADR          MASK COUT      TS1
      NOP          YADR          MASK COUT      TS1
      NOP          YADR          MASK COUT      TS1

```



```
L0:    NOP        YADR REFR        COUT        TS1        ;AUTO REFRESH
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      JN11 L0     YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        MRS            TOUT        TS1        ;MODE REGISTER SET
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      NOP        YADR            COUT        TS1
      JMP  .-1    YADR            MASK COUT    TS1
      STPS

MODULE END

END
```

7. C Program Development Environment

This chapter describes the C program development environment for the hybrid controller. The development environment contains the compiler and debugger required after a C or C++ source program has been created. The following environments are in the order listed.

1. C (C++) Compiler
Describes a C (C++) compiler for the hybrid controller.
2. Linker
Describes how to link multiple object files.
3. Automatic compilation
Describes the make command that is used to automate compilation and link process.
4. Tornado Development Tool
Describes how to debug programs using the Tornado debugger and how to use other Tornado development tools.
5. Other Development Tools
Describes other tools that can be used to support program development.

7. 1 C (C++) Compiler

To compile a C or C++ language source program, use the atcc compiler driver for the hybrid controller. The compiler driver compiles a C or C++ source program and generates an object program. The command atcc can be executed on the host system; however, the object program generated by atcc is applicable only for the tester processor.

A source program may be divided into multiple files. In this case, the respective source files are compiled and object files are generated. All the generated object files are linked in the manner described in [7. 2 "Linker" on page 7-7](#) and merged into a single object file. As described above, when there are multiple source files, use GNU make or make(1S) to automate the compilation for the sake of processing efficiency.

➔ For more information on automatic compilation, refer to [7. 4 "Automatic Compilation - make -" on page 7-10](#).

The atcc is a compiler to which functions have been added and is included in the Tornado development tool. The Tornado compiler is based on GNU^{*1} CC, and its functions are the same as those of GNU CC.

➔ For more information on GNU CC, refer to "Using GNU CC" in ["GNU ToolKit User's Guide"](#).

 Tip

The `atcc` uses the `XATL/U-51` cross compiler to compile `ATL`-mixed programs. The following `XATL/U-51` cross compiler is required.

`XATL/U-51(Product: PXTL51-00S***) REV 6.02 and later`

`XATL/U-51(Product: PXTL51-36S***) REV 3.01 and later`

*1 `GNU`(`GNU's Not UNIX!`) is one of the projects in `FSF` (Free Software Foundation). The software products distributed by `FSF`, which are protected by copyright, conform to the `GNU General Public License` (`GPL`). The `GPL` is described in "[GNU ToolKit User's Guide](#)".

7. 1. 1 Environment Variables Required for Execution

To use the `atcc` compiler driver for the hybrid controller, environment variables must be set as shown in [Table 7-1](#).

Table 7-1 Environment Variables Required to Run the C Compiler

Environment Variable	Explanation
<code>WIND_BASE</code>	This variable specifies the directory where Tornado has been installed.
<code>WIND_HOST_TYPE</code>	This variable specifies the type of host environment used by Tornado. Always set <code>sun4-solaris2</code> for the hybrid controller.
<code>AT_ASYSTEM</code>	This variable specifies the system directory in which the Advantest products have been installed. The value is automatically determined by executing the <code>ASX/U-51</code> tester environment setup script.
<code>PATH</code>	This variable specifies a shell command search path. <code>\$WIND_BASE/host/sun4-solaris2/bin</code> and <code>\$AT_ASYSTEM</code> need to be added to this path.
<code>LD_LIBRARY_PATH</code>	This variable specifies a dynamic link library search path. <code>\$WIND_BASE/host/sun4-solaris2/lib</code> needs to be added to this path.

7. 1. 2 Compiling C (C++) Source Program

Basic compilation command

The C compiler determines the programming language by checking the extension added to the source file name. The extension of a C language source file name must be `'c'`, the extension of a C++ language source file must be `'C'` (uppercase letter), `'cc'`, or `'cpp'`. In this section, these extensions are referred to as the `'language_id'`.

The most basic command line, which is required to compile a source file with the base name^{*1} `"basename,"` is as follows.

```
% atcc -c basename.language_id
```

When this command is executed, an object file named "basename.o" is created. The -c option is used to create a relocatable object file without editing the link setting.

To compile a source file for the hybrid controller, the -c option must be specified.

For the C compiler, the user can specify a variety of options. The following paragraphs introduce the typical options and the options automatically added when using the atcc command.

**1 The base name refers to a file name without an extension.*

Warning options

The options below are used to control warning messages at compilation.

Table 7-2 Major C Compiler Warning Options

Option	Explanation
-Wall	Displays warning information about portions inappropriate and to be avoided in usage.
-W	Makes program-logic related checks not performed by the -Wall option.

Debug option

The option below is specified to debug the program using the debugger. An additional information for the debugger is output to an object file.

Table 7-3 Major C Compiler Debug Option

Option	Explanation
-g	Outputs debug information in stabs format.

Optimization option

The options below are used to specify an optimization level at compilation.

Table 7-4 Major C Compiler Optimization Options

Option	Explanation
-O	Performs standard optimization
-O2	Optimized at a higher level than with the -O option

XATL/U-51 cross compiler option

The option below specifies an option to pass to the XATL/U-51 cross compiler when the ATL-mixed program is compiled.

Table 7-5 XATL/U-51 Cross Compiler Options

Option	Explanation
<code>-Wxt,option</code>	Passes the options specified in "<i>option</i>" to the XATL/U-51 cross compiler. Multiple options can be specified in "<i>option</i>" by separating them with commas. The following options can be specified in "<i>option</i>." <code>C=condition</code> Specifies the conditions of the ATL conditional compilation. This is equivalent to specifying the <code>-C condition</code> for the XATL/U-51 cross compiler. <code>S=socket</code> Specifies the ATL socket program which is used for compilation. Specify the ATL socket program file name for "<i>socket</i>". This specification is processed before the <code>#pragma atl socket</code> specification.

Default options

The atcc applies the following options by default. All of these options are necessary to compile programs that run on the hybrid controller.

Table 7-6 Default Compilation Options

Option	Explanation
<code>-ansi</code>	Compiles, assuming an ANSI C (C++) program.
<code>-nostdinc</code>	Searches for a header file only in the current directory and the directory specified by the <code>-I</code> option.
<code>-fno-builtin</code>	Does not apply any built-in function of the compiler to functions that do not begin with two underscores.
<code>-fno-for-scope</code>	Sets the application range of a variable declared in the for statement to the end of the for statement in a C++ program.
<code>-fvolatile</code>	Assumes memory reference via a pointer to be volatile.
<code>-fno-common</code>	Assigns an uninitialized external variable to a BSS section, not COMMON section.
<code>-mstrict-align</code>	Does not permit memory references in unaligned boundary.
<code>-mlongcall</code>	Support branches to a 32-bit address space.
<code>-DCPU=PPC604</code>	Required to compile for VxWorks.
<code>-D_GNU_TOOL</code>	Required to compile for VxWorks.
<code>-D__ATCC__</code>	A macro which indicates that the source program is being compiled by the atcc.
<code>-I\$(WIND_BASE)/target/h</code>	Includes the directory containing VxWorks header files in the header file search directory.
<code>-I\$(AT_ASYSYSTEM)</code>	Includes the ASX/U-51 system directory in the header file search directory.

7. 1. 3 Atcc-specific Pragma

The atcc supports the following pragma.

Pragma	Overview
#pragma load	Dynamic link for library module
#pragma atl name	A pragma for the ATL-mixed program. ➔ For more information, refer to 3. 4. 3 "ATL-mixed Method" on page 3-11.
#pragma atl socket	
#pragma atl proc	

7. 1. 3. 1 #pragma Load - Dynamic Link for Library Module

◆ Function

This pragma is used to load the library module dynamically to the tester processor and link it to the main module^{*1}. By using this function, the module used in common among multiple programs is not required to be linked statically to the program, and maintainability is improved.

^{*1} It shows a module specified in the /HYBRID utility program.

◆ Syntax

```
#pragma load(prio) "filename"
```

◆ Argument

filename

Specifies the names of library module (object file name). Library module name is specified by the same rules as the module name specified in the /HYBRID utility program.

prio

"prio" specifies the precedence to load the specified library module. When multiple library modules are specified, prio is used to determine the precedence to load them.

Precedence is specified by using an integer, 0 to 99, with precedence increasing in descending order of values. Specification of *prio* with round brackets () can be omitted. When specification is omitted, 50 is automatically specifies as the precedence.

◆ Example of Description

```
#pragma load      "lib1.o"
#pragma load(20)  "lib2.o"
```

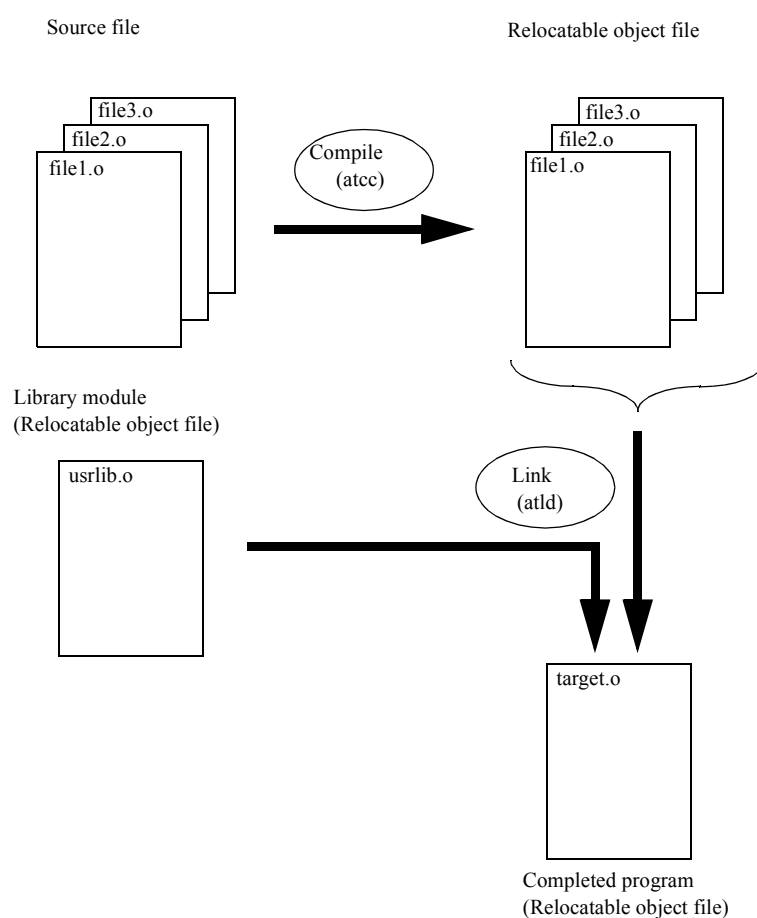
This pragma is effective only for the specified C program to be main module. Therefore the pragma specified in the library module becomes invalid and ignored. Library modules are loaded in sequence from the highest priority before the main module is loaded. For the modules having same precedence, the precedence among modules is not determined to be loaded. When same library modules are specified several times, only the module specified with highest precedence is effective. Unresolved symbols in the module will be solved by linking one after another when loading. Therefore the precedence has important meaning when there is dependence among modules. For example, when the module A depends on the module B, the module B needs to be loaded before the module A is loaded. In that case, the precedence of module B must be specified higher than that of module A.

7. 2 Linker

When a C (C++) source program consists of some files, the files must be compiled to object files and merged onto one object file using a process called link editing.

The hybrid controller uses relocatable object files (ordinarily, with a .o extension) used for the link editing, not an executable object file with the link setting. Therefore, a process referred to as a partial link is performed to merge more than one relocatable object files into a new relocatable object file.

Figure 7-1 Link Editing for Object Files



Tip

Library module must be relocatable type.

➔ For more information on how to link dynamically the library module, refer to [7. 1. 3. 1 "#pragma Load - Dynamic Link for Library Module" on page 7-5.](#)

Unresolved external reference symbols for the C standard library and others included in the object files are resolved by linking them to symbols of the target system when the program is loaded.

Link editing is performed using the atld linker, as shown below.

```
% atld -r -o target_object object1 object2 ....
```

The linker has the following options.

Table 7-7 Major Linker Options

Option	Explanation
-r	Generates a relocatable object file. This option is required.
-X	Deletes temporary local symbols (generated by the compiler).
-S	Deletes symbol information for the debugger.
-s	Deletes all symbol information that can be deleted.
-Map <i>mapfile</i>	Outputs a link map (diagnostic information about the position of symbols mapped by the linker) to <i>mapfile</i> .
-o <i>target_object</i>	Sets an object file name generated by the linker to <i>target_object</i> . When this option is omitted, the default file name is <i>a.out</i> .
<i>object1, object2, ...</i>	Specify the names of object files to be linked.

In the example in [Figure 7-1](#), the link command line is specified as follows:

```
% atld -r -o target.o file1.o file2.o file3.o usrlib.o
```

As GNU CC also has a function to invoke the linker, link editing can be performed using the atcc compiler driver.

➔ For more information on how to start the linker from the compiler, refer to "Using GNU CC" in ["GNU ToolKit User's Guide"](#).

The command atld internally invokes the linker included in the Tornado development tool with the same arguments specified in the command arguments, so that the user does not need to take the platform into consideration. The Tornado linker is based on the GNU linker, and its functions are the same as those of the GNU linker.

➔ For more information on GNU linker, refer to "Using ld, the GNU linker" in ["GNU ToolKit User's Guide"](#).

7. 3 Developing C++ Programs

The hybrid controller also supports C++ program development.

All VxWorks standard functions and hybrid controller library functions can also be accessed from a C++ program.

To develop a C++ program that can be executed with VxWorks, a procedure called munching is necessary after all source programs have been compiled and linked. This procedure enables VxWorks to call constructors and destructors for static objects in the correct order.

The example below shows a command that creates the munched object file `hello.out` from object file `hello.o`.

```
% nmppc hello.o | wtxtcl $WIND_BASE/host/src/hutils/munch.tcl \  
-asm ppc > ctdt.c  
  
% atcc -c ctdt.c  
  
% atld -r hello.o ctdt.o -o hello.out
```

In actual munching, replace the object names `hello.o` and `hello.out` in this example with the actual object file names.

➔ For more information on C++ development, refer to ["VxWorks Programmer's Guide"](#).

7. 4 Automatic Compilation - make -

When there are multiple source files, manual compilation of source files and linkage do little to enhance processing efficiency. Even when one source file is corrected, it takes time to check the dependency of the corrected source file to other source files and to recompile all the related files.

The make command automatically performs the related operations. There are two types of make commands: GNU make included in Tornado and make(1S) included in the SUNWspot or SUNWxcu4t packages of Solaris. These commands have almost the same function, with the exception of a few features. The user can choose any one from those commands.

➔ For more information on GNU make, refer to ["GNU Make User's Guide"](#).

This section outlines the make command and explains a sample Makefile for the hybrid controller.

7. 4. 1 Outline of the make Command

The make command automatically judges which parts in a large program need to be compiled, executes commands to compile those parts, and (re)constructs a program.

To use the make command, it is necessary to create a file named Makefile^{*1} that contains the information on how to construct the program. The program to be constructed is ordinarily updated by an object file, and the object file is created by compiling a source file. Accordingly, describe dependency relationships and commands required for each phase to construct the program in the Makefile.

**1 The file can also be named makefile. This file is loaded by default when the make command is executed without the '-f' option. Using the '-f' option, the user can specify another file instead of Makefile.*

The functions of the make command are explained using the following sample Makefile.

```
# Sample Makefile

# source files, object files, and the target
SRCS  = foo1.c foo2.c
OBJS  = $(SRCS:%.c=%.o)
TARGET = sample.o

# rule to make target
$(TARGET): $(OBJS)
    $(LD) -r $(OBJS) -o $@

# rule to compile each source
%.o: %.c
    $(CC) -c $<

# dependency rules
$(OBJS): foo.h
```

Variables can be used in Makefile. Each variable is expressed using an equal sign (=) and the variable on the left side of the equal sign is set to the value on the right side of the equal sign. In the example above, the variable SRCS is set to `foo1.c` and `foo2.c`.

The variable `var` can be referenced by `$(var)`. In addition to simple references like this, make also supports substitution references. In this example, the substitution reference `$(SRCS:%.c=%.o)` is performed to set the variable OBJS. '%' specifies a pattern matching. Using this symbol, all '.c' in the variable SRCS are substituted with '.o,' and the variable OBJS is set to the resulting value. In other words, the variable OBJS is set to `foo1.o` and `foo2.o`.

A line containing a colon (:) indicates a rule. The rule describes when and how an objective file (target) should be created. The left side of the colon (:) indicates the target. The right side, also referred to as a dependency file, lists the files required to create the target. The command to create the target is described on the next line. The command description must begin with a tab. After updating all dependency files, or after applying every rule in which its respective dependency file is set as a target, the make executes this command.

In the rule of `$(TARGET)` in this example, the dependency file is `$(OBJS)`. This rule instructs make to create `$(OBJS)` before creating `$(TARGET)`. The '\$@' element in the command is referred to as an automatic variable or dynamic macro that is substituted with the target.

Although there are no rules in which `$(OBJS)` is set as a target, the files contained in `$(OBJS)` are processed by the next rule, `%.o:%.c`. The character '%' appearing in the rule indicates pattern matching. This rule performs pattern matching for files with the suffix '.o.' The '<' element of the command is an automatic variable that is substituted for the first file name in the dependency files.

The last rule specifies only the `$(OBJS)` dependency file. If this rule is not included when `$(OBJS)` depends on `foo.h`, `$(OBJS)` is not updated even if `foo.h` is updated. This is because the dependency relationship of `foo.h` is not described in any other rules. This rule notifies make that `$(OBJS)` depends on `foo.h`, ensuring that `$(OBJS)` is updated when `foo.h` is updated.

7. 4. 2 Makefile Sample

Figure 7-2 shows a sample Makefile which can be used to make ATL programs and C programs for the hybrid controller.

Figure 7-2 Makefile Sample

```

# Sample Makefile for Hybrid Controller
# -----
# Don't forget to escape a newline character with a backslash '\\' to
# concatenate two lines.
#
# -----
# Put C/C++ source files and the target object file here.
#
CSRCS    = csrcl.c csrc2.c ...
C++SRCS  = cppsrc1.cc cppsrc2.cc ...
CTARGET  = target.o

# -----
# Put ATL source and object files here.
#
ATLSRCS  = atlsrcl.asc atlsrcl2.asc ...
ATLOBJS  = atlsrcl.pro atlsrcl2.soc ...
# -----
# C/C++ compilation flags and linker flags.
# CFLAGS: C/C++ compiler flags. Use '-g' instead for debug.
# LDFLAGS: Linker flags. The '-r' flag is always required.
#
CFLAGS   = -O2
LDFLAGS  = -r

# -----
# make rules: You will not need to edit below.
# -----
# target architecture

ARCH     = ppc

# C/C++ compiler, linker and utilities
CC       = atcc
LD       = atld
NM       = nm$(ARCH)
MUNCH    = wtxtcl $(WIND_BASE)/host/src/hutils/munch.tcl -asm $(ARCH)
SYMCHK   = atsymchk

# C/C++ object filename generation
COBJS    = $(CSRCS:%.c=%.o)
C++OBJS1 = $(C++SRCS:%.cc=%.o)
C++OBJS2 = $(C++OBJS1:%.C=%.o)
C++OBJS  = $(C++OBJS2:%.cpp=%.o)

# - continued on the following page -

```

Figure 7-3 Sample Makefile (Cont'd)

```
# - continued from the preceding page -

# ATL compiler
TRANS = $(AT_SYSTEM)/at\\$exec_xatl51su trans51

# ATL filename suffixes
ATLSUFFIXES = %.pro %.soc %.sub %.lib %.mpa %.scr

# target build rule starts here
all: $(CTARGET) $(ATLOBJS)

# C/C++ linking phase
$(CTARGET): $(COBJS) $(C++OBJS)
    @ $(RM) $@ ctdt.c
    $(NM) $(C++OBJS) | $(MUNCH) > ctdt.c
    $(CC) -fdollars-in-identifiers $(CFLAGS) -c ctdt.c
    $(LD) $(LDFLAGS) $^ ctdt.o -o $@
    @ $(RM) ctdt.c ctdt.o
    $(SYMCHK) $@

# C compilation rule
%.o: %.c
    $(CC) $(CFLAGS) -c $<

# C++ compilation rules
%.o: %.cc
    $(CC) $(CFLAGS) -c $<
%.o: %.C
    $(CC) $(CFLAGS) -c $<
%.o: %.cpp
    $(CC) $(CFLAGS) -c $<

# ATL compilation rule
$(ATLSUFFIXES): %.asc
    $(TRANS) $<

# remove all generated files
clean::
    $(RM) $(CTARGET) $(COBJS) $(C++OBJS) $(ATLOBJS)
    $(RM) *~ core ctdt.c ctdt.o
```

When using this sample, set the following variables to appropriate values. The base name of an ATL source file and that of the object file must be the same.

- CSRCS
Specify the C source file names.
- C++SRCS

Specify the C++ source file names.

- CTARGET

Specify the final object file name to be created.

- ATLSRCS

Specify the ATL source file names.

- ATLOBJS

Specify the ATL object file names.

To generate object file names to be set in ATLOBJS from ATL source file names, it is necessary to open each ATL source file and check the title at the head of the file. However, this process can only be performed using a method that depends on the implementation of make; Thus, this variable is specified explicitly in this example.

For example, if only GNU make is used, the ATL object file names can be generated as follows.

```
# ATL object filename generation
ATLOBJS = \
    $(foreach file, $(ATLSRCS), \
        $(shell /usr/bin/nawk '\
            /^[ \t]* (PRO|SOCKET|SUBR|LIB|MPAT|SCRAM) / { \
                printf("%.8s.%.3s", tolower($$2), tolower($$1)); \
                exit; \
            }' $(file) \
        ) \
    )
```

- CFLAGS

Specify options to be transferred to the C (C++) compiler.

- LDFLAGS

Specify options to be transferred to the linker.

The rule can be reinforced by adding the dependency relationship of a source file for each object file to this Makefile. The dependency relationship of a C program can automatically be generated by specifying the '-M' option of the C compiler.

➔ For more information, refer to GNU CC described in ["GNU Make User's Guide"](#) and ["GNU ToolKit User's Guide"](#).

7. 5 Tornado Development Tool

The Tornado development tool, which is a product of Wind River Systems bundled to the ASX/U-51 system software, provides an integrated cross development environment which allows you to develop a program used for the target system under the host system. The Tornado development tool consists of the following components:

- Launcher

Launcher is an integrated target management tool. This tool is a central control panel of Tornado which manages the connection with a target and starts other tools.

- CrossWind

CrossWind is a debugger with the extensions to the GNU source level debugger (GDB) which has an easy-to-understand graphical user interface. This debugger can be used to debug a program for the target system on the host system.

- Browser

Browser, a tool for monitoring a system object in the target system, displays the state of the target system in an easy-to-understand form.

- WindSh

WindSh is a command shell to control the target from the host. This tool is mainly used for diagnosis of the system and programs, and therefore users usually do not have to use it.

This section describes the procedure to debug a C program using these tools. However, it does not explain each tool in detail.

➔ For more information on each tool, refer to ["Tornado User's Guide"](#).

The Tornado environment bundled with ASX/U-51 is customized to provide simpler interfaces suitable for use with the hybrid controller. Therefore, the environment partially conflicts with the descriptions found in ["Tornado User's Guide"](#). The differences are explained in this section.

7. 5. 1 Starting Tornado

This section explains how to start Tornado.

7. 5. 1. 1 Preparation

The environment variables listed in the table below need to be set before using Tornado.

Table 7-8 Environment Variables Required for Executing Tornado

Environment Variable	Explanation
WIND_BASE	This variable specifies the directory where Tornado has been installed.
WIND_HOST_TYPE	This variable specifies the type of host environment used by Tornado. Always set <code>sun4-solaris2</code> for the hybrid controller.

Environment Variable	Explanation
WIND_REGISTRY	Specify the name of the host where the Tornado registry will be executed or is being executed (in other words, the host name of the ADVANsite system).
PATH	This variable specifies a shell command search path. Specify the following directory as an additional directory. \$WIND_BASE/host/sun4-solaris2/bin
LD_LIBRARY_PATH	This variable specifies a dynamic link library search path. Specify the following directory as an additional directory. \$WIND_BASE/host/sun4-solaris2/lib /usr/dt/lib /usr/openwin/lib

These variables do not need to be set each time if they are set in the shell startup script (`~/ .cshrc` for C shell, `~/ .profile` for Bourne shell).

7. 5. 1. 2 Starting Tornado Registry

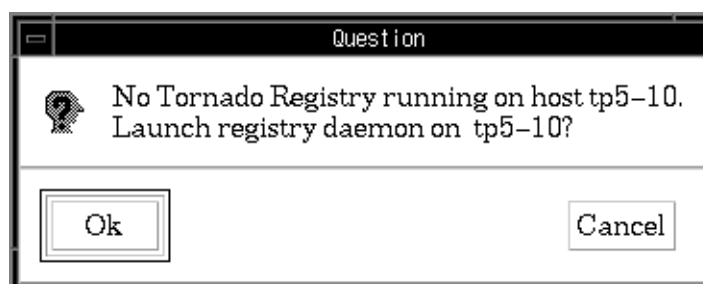
A Tornado registry is a daemon that manages usable targets. A Tornado tool searches for a target by means of the registry daemon. Therefore, the registry daemon must always be executed before Tornado is used.

Only one Tornado registry should run on the host. The registry need not be started each time it is required. The simplest way to start the registry daemon is to use the Launcher. Because the registry daemon must be executed on the ADVANsite environment processor (host system), start the Launcher on the same environment processor. Use the following command to start the Launcher.

```
% launch &
```

If the Tornado registry has not been executed, a dialog box asking whether or not to start a registry daemon is displayed as shown below.

Figure 7-4 Dialog Box Asking Whether or Not to Run the Registry Daemon



If you click the **[OK]** button, the registry daemon is started and the Launcher main window opens. The first time the registry daemon is started, some dialog boxes may be opened before the main window opens. Click the **[OK]** button in each of these dialog boxes.

After the Launcher main window opens, it can be used as the starting point for accessing all Tornado functions.

7. 5. 2 Launcher

The Launcher, which is the main Tornado control panel, can be used to manage the connection between a tool and the target and to start other Tornado tools. This section describes basic Launcher operations.

➔ For more information, refer to "Launcher" described in ["Tornado User's Guide"](#).

7. 5. 2. 1 Starting and Terminating the Launcher

The Tornado registry must be executed before Tornado is used. If the Tornado registry is not found, Launcher attempts to start the registry on the host where the Launcher was started.

➔ For more information on how to start the registry, refer to [7. 5. 1 "Starting Tornado" on page 7-15](#).

Use the following command to start the Launcher.

```
% launch &
```

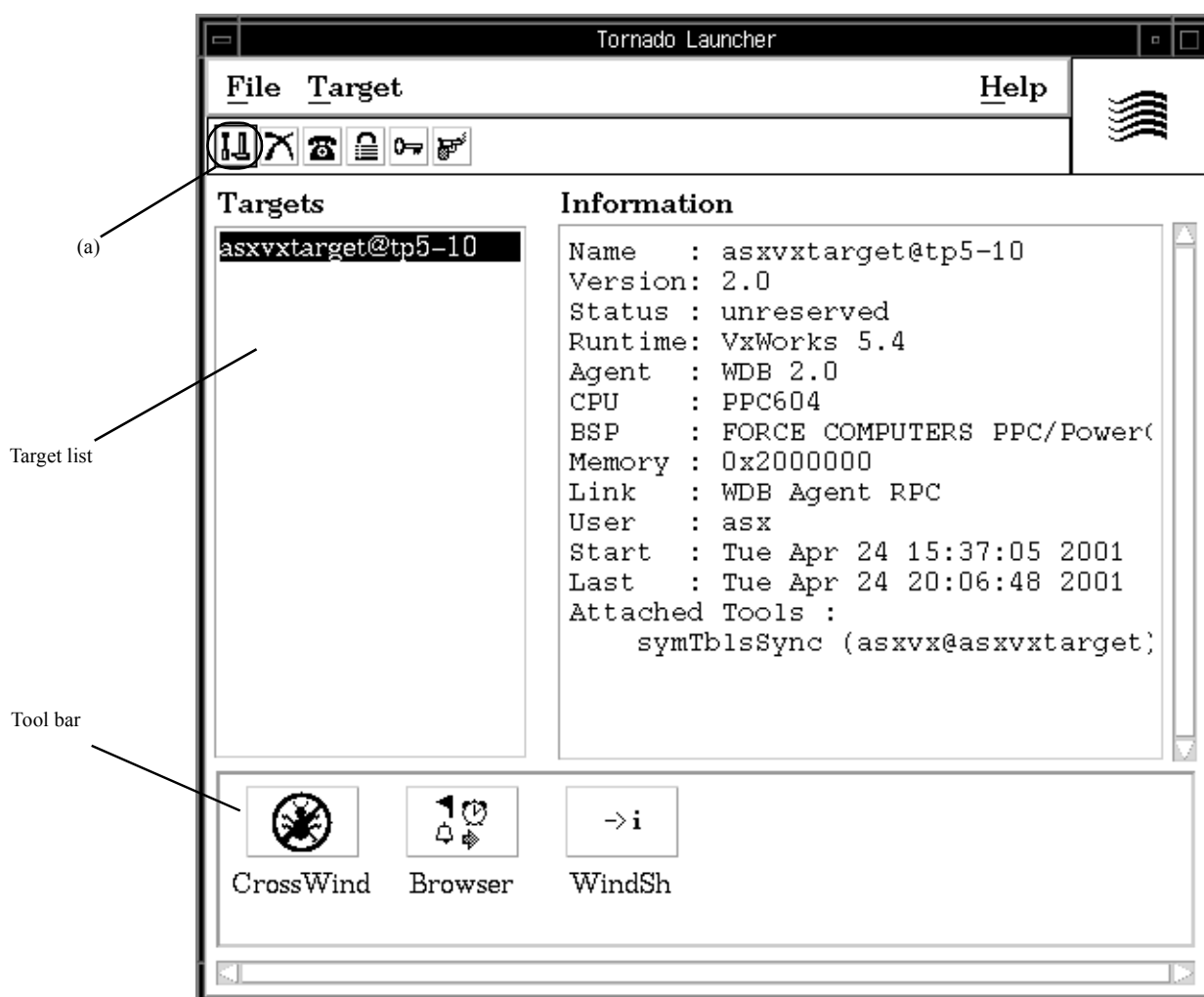
The Launcher main window opens when this command is executed. Some dialog boxes to confirm execution conditions may be opened before the main window opens. If so, check the messages in the dialog boxes, then click the **[OK]** button.

To terminate the Launcher, select **[Quit]** from the **[File]** menu in the main window.

7. 5. 2. 2 Window Configuration

[Figure 7-5](#) shows the Launcher main window configuration. Target and tool information occupies most of the window.

Figure 7-5 Launcher Main Window



The target list displays usable targets and target servers. A target server is a program that communicates with a target and responds to requests from each Tornado tool. A target server should exist for each target. When a target on the target list is selected with the mouse, the tool is connected to the target. The first time the Launcher is started, nothing is displayed in the target list. In this case, start the target server, referring to [7. 5. 2. 3 "Starting a Target Server" on page 7-19](#). The target server for the tester processor is `asxvxtarget@hostname`. To debug a C program for the hybrid controller, select this target server in the target list.

The toolbar has buttons that are linked to the following Tornado tools.

- CrossWind
C (C++) source-level debugger with graphic interface
- Browser
Viewer for checking and monitoring target system objects

- WindSh

Tornado shell

Buttons linked to the tools Project, WindView, VxSim, and WindNav are not displayed on the tool bar because they cannot be used, even though they are described in the *"Tornado User's Guide"*. The **[Support]**, **[Admin]**, **[Info]** and **[About]** menus described in Tornado manuals are also not displayed on the menu bar for the same reason.

7. 5. 2. 3 Starting a Target Server

To use the Tornado tools, it is necessary to connect the tool to the target server. A target server should exist for each target.

You can start a target server by selecting **[Create...]** from the **[Target]** menu or pressing the button just below the menu bar ((a) shown in [Figure 7-5](#)). The Create Target Server form is displayed.

Input data as shown in [Figure 7-6](#), and the target server will be started with the proper configuration when the **[Launch]** button is pressed. The started target server is displayed in the target list in the main window.

The parameters set in the Create Target Server form are saved on a host disk. For subsequent target server activation, you can start the target server under the same conditions as the first activation by simply selecting the target server listed in the Saved Configurations field in the upper-left of the form.

Figure 7-6 Settings in the Create Target Server Form

The screenshot shows the 'Create Target Server' dialog box with the following settings:

- Saved configurations:** (Empty list)
- Target name or IP address:** asxvxtarget
- Target Server Options:**
 - Target server name:** asxvxtarget
 - Authorized users file:** (Browse button)
 - Object module format:** default (dropdown menu with 'a.out' below it)
 - Core file:** (Browse button)
 - Console display:** (Browse button)
 - Checkboxes:**
 - ☐ Target I/O Redirect
 - ☐ Shell I/O Redirect
 - ☐ Virtual console
 - ☐ No symbols
 - ☒ All symbols
 - ☒ Target/Host symbol tables synchronization
 - ☐ Use port mapper
 - ☐ Verbose
 - ☐ Locked
 - ☐ TSFS Read/Write
 - Memory cache size:** default
 - TSFS root directory:** (Browse button)
- Backend list:** default (dropdown menu with 'loopback' below it)
- Serial line speed:** default (dropdown menu with '1200' below it)
- Serial line device:** (Browse button)
- Backend timeout:** (Browse button)
- Backend resend:** (Browse button)
- Backend Log file:** (Browse button)
- Backend Log file max size:** (Browse button)
- WTX Log file:** (Browse button)
- WTX Log file max size:** (Browse button)
- WTX Log file filter:** (Browse button)
- Target server launch command:** tgtsvr asxvxtarget -A -s -n asxvxtarget
- Buttons:** Help..., Delete, Launch, Quit

7. 5. 3 Tornado Online Manual

Tornado contains a hypertext collection of all Tornado and VxWorks manuals in HTML format. These online manuals can be viewed from your present Web browser.

A tornado online manual cannot be opened unless the Web browser search path is set in the PATH environment variable. Set the search path in the variable before activating a Tornado tool.

After making the setting, start the Launcher and select **[Manuals index]** in the **[Help]** menu in the main window. The Web browser is started and the Tornado online manuals are displayed. The Tornado online manual can also be opened from the **[Help]** menu of each Tornado tool in the same way.

7. 5. 4 Debugging Using CrossWind

CrossWind is a C (C++) source-level debugger that uses a graphical user interface. When it is used, a program running on a target can be debugged on a host. Because the CrossWind debugger is based on the GNU source-level debugger (GDB), it also accepts GDB commands .

This section describes how to debug a C program, which is used by the hybrid controller, by using CrossWind.

➔ For more information on CrossWind and GDB, refer to the debugger described in "[Tornado User's Guide](#)" and "[GDB User's Guide](#)", respectively.

Tip

A hybrid controller program, usually consists of processes which are described in the C (C++) language and processes which are described in the ATL language. CrossWind debugs only processes which are described in the C (C++) language. To debug processes which are described in the ATL language, use ATLworks or the ATL debugger.

ATLworks can debug efficiently because it is capable of identifying the language that was used to describe the executed program and is equipped with a function which debugs processes that are described in both the C (C++) language and the ATL language in the same window.

Limitation

To debug processes which are described in the ATL language in a ATL-mixed program, use ATLworks. In this case, processes which are described in the ATL language cannot be debugged by using the ATL debugger (/DEBUG).

The ATLworks debugger cannot be used if an ATL-mixed program is being debugged by using CrossWind.

7. 5. 4. 1 Debugging Procedure

To debug a hybrid controller program, follow the procedure described below.

1) Compiling and linking

Compile the modules of the C program to be debugged with the debugging option '-g' and if other modules exist, link them with the target modules.

➔ For more information, refer to [7. 1 "C \(C++\) Compiler" on page 7-1](#) and [7. 2 "Linker" on page 7-7](#).

Compile the ATL program using the XATL/U-51 cross compiler.

2) Starting the controller and debugger

If the controller is inactive, start it. Then start the debugger (ATLworks, ATL debugger, or CrossWind) appropriate for the type of program to be debugged. Make any other preparations that are necessary before using the debugger.

➔ For more information on how to start CrossWind, refer to [7. 5. 4. 2 "Starting and Terminating CrossWind" on page 7-23](#).

3) Loading the C program

Load the C program by specifying the operation mode S with the /HYBRID utility program.

➔ For more information, refer to [4. "/HYBRID - Setting the Controller Operating Mode" on page 4-1](#).

4) Executing the program

Start the C program using the procedure described in [3. 3. 1 "Program Start-up" on page 3-3](#). The message shown below is displayed on the VKT when the program starts in operating mode S.

```
H>  
Break at 0xdfffe58 : test_main()      task : tUsrMain
```

This message informs you that program execution has stopped at the beginning of the test_main() function and indicates that the program can be attached to CrossWind.

5) Debugging

Continue the program execution after setting a breakpoint, and verify if the flow or data is correct.

➔ For more information on the functions and operation of CrossWind, refer to [7. 5. 4. 3 "Outline of CrossWind Functions" on page 7-23](#) and [7. 5. 4. 4 "CrossWind Operating Procedure" on page 7-29](#).

6) Correcting the Program

If a bug is detected in the program, correct the source program and restart from step 1). Repeat these steps until no more bugs are detected.

7. 5. 4. 2 Starting and Terminating CrossWind

To start CrossWind, select `asxvxtarget@hostname` from the Launcher target list and press the **[CrossWind]** button on the toolbar.

The CrossWind main window opens.

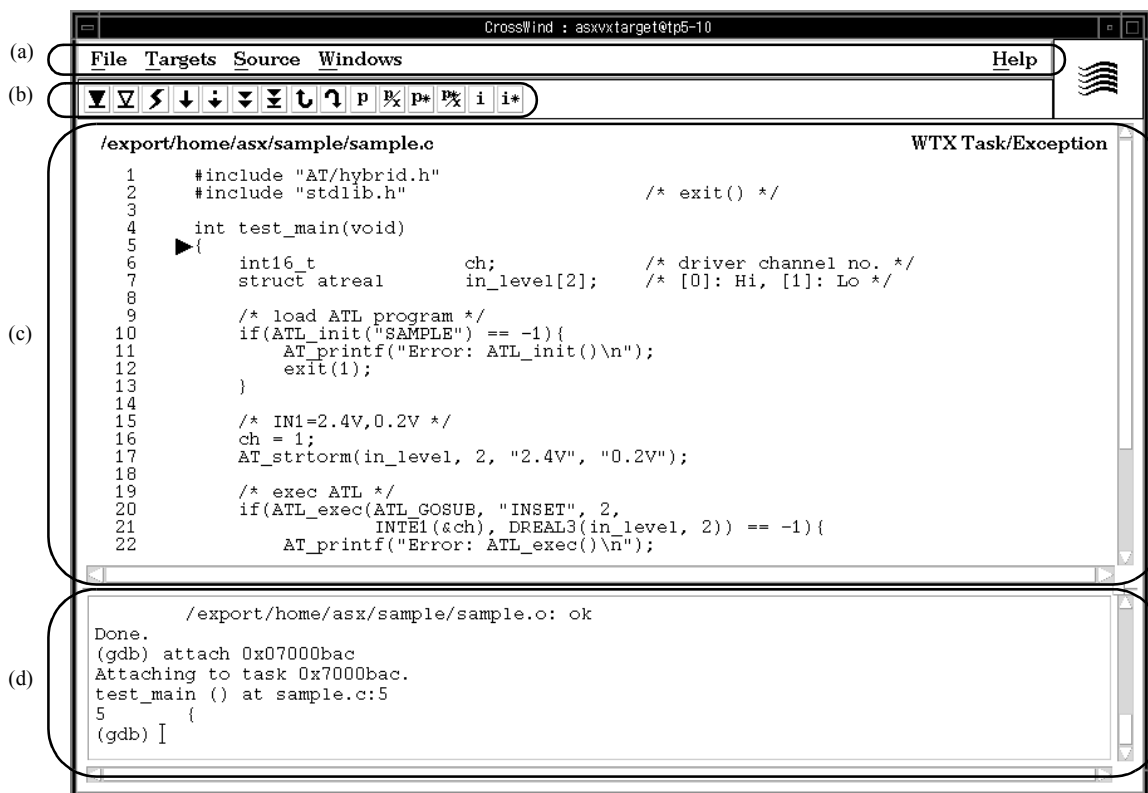
However, CrossWind cannot be started and an error occurs if a C program is being debugged using ATLworks or another CrossWind is connected to `asxvxtarget`.

To quit CrossWind, select **[Quit]** from the **[File]** menu in the CrossWind main window.

7. 5. 4. 3 Outline of CrossWind Functions

Figure 7-7 shows the CrossWind main window configuration.

Figure 7-7 CrossWind Main Window



(a) Menu bar (b) Button bar (c) Program display panel (d) Command panel

(a) The menu bar provides access to control facilities of CrossWind. (b) The button bar contains buttons for quick access to often used debugger commands with the mouse. (c) The program display panel displays the program and breakpoint information. (d) GDB commands can be executed directly from the command panel.

The menu bar, button bar, and command panel are described below. Items marked with * indicate functions contained only in CrossWind bundled with ASX/U-51. Items marked with ** indicate standard functions that are being changed.

Menu Bar

CrossWind control facilities can be accessed from the menus on the menu bar.

1. [File] menu

This menu is mainly used to control information about the program being debugged. It consists of the following items.

- **[Update Symbol Files]***

This command loads the debugging information of the C program loaded in the target into the debugger so that it can be used in a debugging session ^{*1}.

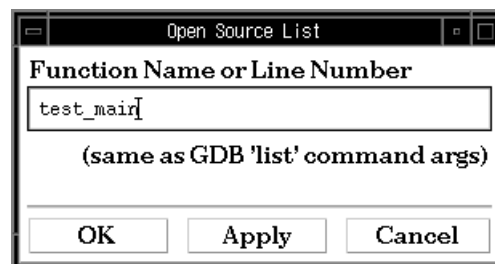
When a C program is freshly loaded using the/HYBRID utility program, the debugging information for the C program has not been loaded into the debugger. In this case, select this command to load the debugging information into the debugger.

**1 A debugging session covers the entire debugging process using the CrossWind debugger.*

- **[Open Source List...]***

This command specifies the location in the source program to be displayed in the program display panel by using a function name or a line number. When this command is selected, the Open Source List dialog box shown in [Figure 7-8](#) appears.

Figure 7-8 Open Source List Dialog Box



Specify the location to be displayed in the text box in the same format as the GDB list command arguments. [Table 7-9](#) shows main specification formats.

Table 7-9 Main Specification Formats for list Command

Specification Format	Explanation
<i>linenum</i>	The source program is displayed with the <i>linenum</i> line of the current source file positioned at the center.
<i>function</i>	The source program is displayed with the beginning of the <i>function</i> "function" positioned at the center.
<i>filename:linenum</i>	The <i>linenum</i> line of source file <i>filename</i> is displayed.

Specification Format	Explanation
<i>filename: function</i>	The beginning of the <i>function</i> "function" in the source file <i>filename</i> is displayed.

If the **[OK]** button is pressed, the source program is displayed in the program display panel and the dialog box closes. If the Apply button is pressed, the specification is accepted but the dialog box does not close. If the **[Cancel]** button is pressed, the dialog box closes and the specification is ignored.

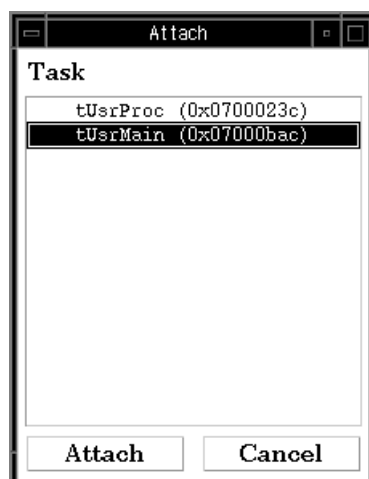
- Quit
This function closes CrossWind.

2. **[Targets]** menu

This menu controls the connection between a target task and a debugging session. It consists of the following items:

- **[Attach Task...]****
This command attaches the debugging session to a task that is already running on the target. When this command is selected, the Attach Task dialog box shown in [Figure 7-9](#) opens.

Figure 7-9 Attach Task Dialog Box



Tasks that can be attached are displayed in the scroll list in the dialog box. Select the task you want to debug and press the Attach button to attach the task to the debugging session. The following two tasks are generally targeted for debugging in the hybrid controller^{*1}.

^{*1} In the hybrid controller, system tasks cannot be attached to a debug session. Therefore, no system tasks are displayed in the scroll list of the dialog box.

Table 7-10 Tasks Targeted for Debugged

Task Name	Explanation
tUsrProc	Select this task when debugging a termination function registered by AT_atexit() or asynchronous event handler.
tUsrMain	Generally, select this task where the test_main() function is being executed.

When the selected task is attached to the debugging session, the debugger suspends the task and displays the corresponding source program where the execution stopped in the program display panel. If there is no corresponding debugging information for this location, the assembly listing is displayed. In this case, check the compile command because the C program may have been compiled without any debug options.

- **[Detach Task]****

This command detaches the debugging session from a task and continues the execution of the task.

- **[Kill Task]****

This command deletes the task attached to the debugging session from the target. This operation cannot be applied to a tUsrProc task. When you need to halt the processing of the tUsrProc task, input [CTRL]-[C] from the VKT and terminate the program.

- **[Connect Target Servers...]**

This command, which displays a dialog box for target server selection, is not used with the hybrid controller.

3. **[Source]** menu

The source program display method can be switched from the **[Source]** menu. When **[C/C++]** is selected from this menu, source programs written in the C or C++ language are displayed. Note that the source programs are displayed only if they are compiled with debugging option -g. Otherwise the program is displayed in the assembly code.

4. **[Windows]** menu

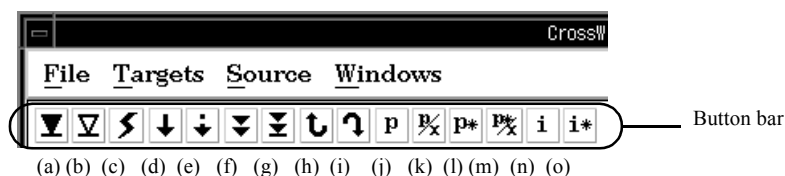
This menu displays an auxiliary debugger screen. When **[Backtrace]** is selected from this menu, a window that shows the function calling relationships (stack trace) is displayed.

5. **[Help]** menu

This menu is used to start a viewer that displays the Tornado online manuals and GDB command usage.

Button bar

Debugger functions that are often used can be executed from this bar by clicking the appropriate button. The buttons on this bar are described below:

Figure 7-10 Button Bar**(a) Setting and removing a breakpoint**

This button sets a breakpoint on the current line or at a location selected in the program display panel. Also, when the right mouse button is clicked in the program display panel, a breakpoint is set at that location.

To delete a breakpoint, right-click the breakpoint mark displayed in the left margin in the program display panel.

(b) Setting and removing a temporary breakpoint

This button functions almost the same way as the button (a). This button differs from the button (a) in that it stops the program only once. If the program stops at that location, the debugger disables the temporary breakpoint.

To delete a temporary breakpoint, right-click the temporary breakpoint mark displayed in the left margin in the program display panel.

(c) Interrupt

This button sends an interrupt to the task that the debugger is controlling. The interrupt halts the program and control is returned to the debugger. The debugger becomes able to operate by this command, but it cannot respond while the program is running.

(d) Steps to the next line

This button executes the current line and steps to the next line. If a function is called on the current line, the program suspends at the beginning of the function.

(e) Steps to the next line

This button executes the current line and steps to the next line. If a function is called on the current line, the program suspends on the next line in the program display panel after the function is executed.

(f) Continue execution

This button continues the program execution from where it was suspended.

(g) Complete current function

This button continues the execution of the program until the current function is completed and then suspends the program.

(h) Move the stack frame one level up

Since the debugger generally has the same scope as the function being executed, the local variables of a function that is the caller of the function being executed cannot be examined from the location where the program stopped. When this button is selected, the scope of the function that is the caller of the function being executed can be assigned to the debugger, allowing the local variables of the caller to be examined.

(i) Move the stack frame one level down

This button functions in the opposite manner of the button (h).

(j) Symbol value display

This button displays the symbol value of a variable in the command panel. When this button is pressed after a symbol in the program display panel is selected by left-clicking, the value of the symbol is displayed in the command panel.

(k) Symbol value display*

This button has the same function as the button (j) except that values are always displayed in hexadecimal integers.

(l) Pointer reference destination value display

This button assumes that the selected symbol is a pointer and displays the value of the object being pointed to. This button operates in the same way as the button (j).

(m) Pointer reference destination value display*

This button has the same function as the button (l) except that the values are always displayed in hexadecimal integers.

(n) Symbol value monitoring

This button opens a window for monitoring of a symbol value. The displayed symbol value is updated each time control is returned to the debugger. This button operates in the same way as the button (j).

(o) Pointer reference destination value monitoring

This button assumes that the selected symbol is a pointer and opens a window so that the value of an object being referenced can be monitored. This button operates in the same way as the button (j).

Command panel

This panel is a GDB command interpreter. You can execute a GDB command by directly inputting it in this panel.

7. 5. 4. 4 CrossWind Operating Procedure

To use the CrossWind debugger to debug a C program that operates with the hybrid controller, start the program as explained in [7. 5. 4. 1 "Debugging Procedure" on page 7-21](#), and then operate CrossWind as explained in this section.

➔ For more information on the specific operation of CrossWind, refer to [7. 5. 4. 3 "Outline of CrossWind Functions" on page 7-23](#) and "Debugger" in the *"Tornado User's Guide"*.

1) Loading debugging information

Select **[Update Symbol Files...]** from the **[File]** menu to load the debugging information of the C program into the debugger. This operation must be performed when a C program is freshly loaded in the tester processor or when a C program is updated.

2) Establishing the debugging session

Select **[Attach Task...]** from the **[Targets]** menu to select a task to be debugged. Generally, select the **[tUsrMain]** task. The source program is displayed in the program display panel. If it is not displayed correctly, check the compile command because the C program may have been compiled without debug options.

3) Setting a break point

Select **[Open Source List...]** from the **[File]** menu to display a location for setting a breakpoint. Set a breakpoint in the program display panel for the displayed program.

4) Continuing the program

Press the button (f) on the button bar to continue execution of the program. The program stops at the breakpoint and returns control to the debugger. This enables the debugger to run again.

Use various debugger functions to verify that the data contents and flow are correct.

5) Terminating debugging

To terminate debugging, complete the execution of the program or select **[Detach Task]** from the **[Targets]** menu.

7. 5. 5 Browser

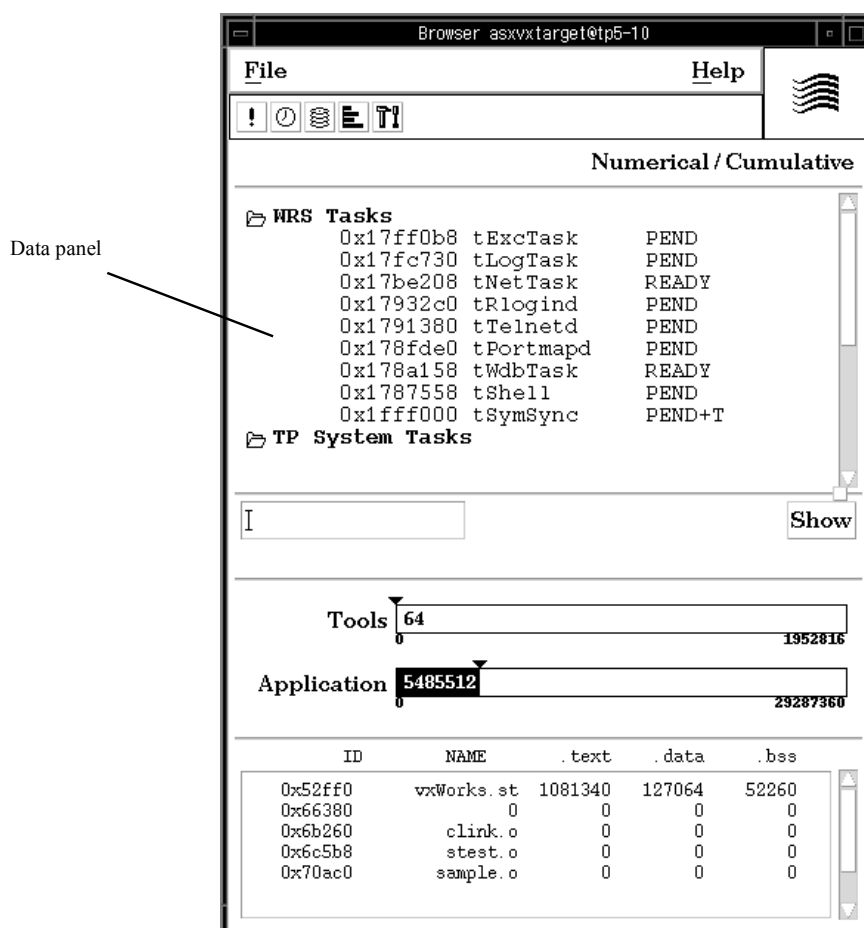
This tool graphically displays VxWorks system objects and system statuses.

To start Browser, select `asxvxtarget@hostname` from the Launcher target list and press the **[Browser]** button on the toolbar.

To terminate the Browser, select **[Quit]** from the **[File]** menu in the main window.

[Figure 7-11](#) shows the Browser main window.

Figure 7-11 Browser Main Window



This browser, which is bundled with ASX/U-51, differs from a standard browser in the hierarchy used to display a task list in the data panel. The hierarchies for this browser are described below.

[WRS Tasks]

Information about VxWorks operating system tasks is displayed.

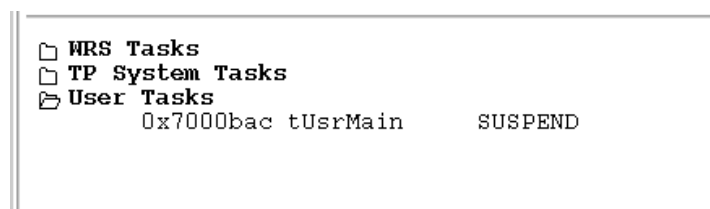
[TP System Tasks]

Information about tester processor system tasks is displayed.

[User Tasks]

Information about user tasks is displayed.

These hierarchies are classified into the folders of the same name listed above and displayed in the data panel. Clicking one of these folders shows or hides its contents.

Figure 7-12 Task List Folder in the Data Panel

➔ For more information on the Browser, refer to Browser in *"Tornado User's Guide"*.

7. 5. 6 WindSh

Windsh is a command shell used to control the target from the host. Because it is mainly used for diagnosing systems and programs, the user generally does not need to use this shell. The hybrid controller can develop a program without using this shell.

➔ For more information on WindSh, refer to Shell in *"Tornado User's Guide"*.

7. 6 Other Development Tools

This section describes C program development support tools:

7. 6. 1 Symbol Checker - `atsymchk` -

Function: This tool checks whether an external symbol defined in a user program duplicates one defined by the system.

Syntax

```
atsymchk file1 file2 ...
```

Functional Description

As described in [6. 6. 2 "The Scope of Symbols with External Linkage" on page 6-7](#), any external symbols defined in a user program must not match an external symbol defined in the system (VxWorks and tester processor system programs). Therefore, execute this command when compiling the source program to check for such external symbols.

Specify the name of the object file to be checked as an argument. If an external symbol defined in the system is detected in the specified object file, a symbol indicating the type of symbol and the name of the symbol are written to the standard output.

```
% atsymchk sample.o
Redefined reserved system symbols
sample.o:
  T fopen
  O Stc_Sym
```

The following symbols are used to indicate symbol types.

- T
Text symbol. This symbol indicates a text symbol and corresponds to a function name.
- O
Other data symbols. These are symbols such as DATA, BSS, and COMMON. These correspond to data object names.

Each external symbol is compared to external symbols of the same type in the above description. Text symbols and data symbols are not compared.

When two or more files are specified, the results for each file are displayed as shown below:

```
% atsymchk sample1.o sample2.o
Redefined reserved system symbols
sample1.o:
    T NG_symbol1
    T NG_symbol2
    :
    :
sample2.o
    T NG_symbolx
    O NG_symboly
    :
    :
```

Termination Status

0 is returned if external symbols defined in the system are not detected. A value of 1 or more is returned if an external symbol defined in the system is detected or if an error occurs.

Diagnosis

If a file with a format other than the object file format for the tester processor is specified, the following error message is displayed.

```
% atsymchk foo1.o foo2.o foo3.o
nmppc: foo1.o: File format not recognized
nmppc: foo3.o: File format not recognized
Redefined reserved system symbols
foo2.o:
    T foo
    O var
```

The formats of files `foo1.o` and `foo3.o` are incorrect in this example.

7. 6. 2 Tester Processor Reset - `tpreset` -

Function: This tool forces a reset of the tester processor.

Syntax

```
tpreset
```

Functional Description

This command forcibly resets the tester processor.

A function might be used incorrectly or a system program area might be destroyed during development of a C program. As a result, the system program might be placed in an unrecoverable state or the tester processor might cease to respond.

If either situation occurs, use this command. The tester processor is initialized and restored to the normal state. It takes from about 20 seconds to several minutes to reset the tester processor. While the processor is being resetting, an indicator is displayed to show the command is in progress.

Termination Status

Zero (0) is returned when the tester processor is reset normally. A value of 1 or more is returned if an error occurs.

8. System Configuration

This chapter explains the implementation of the tester processor and related system specifications from the viewpoint of the C-program execution environment in the hybrid controller, and provides information necessary for system programming.

8. 1 Overview

**Tip**

The tester processor specifications given in this chapter may seem inconsistent with the general specifications of the test system or other specifications. The test system specifications are described from the viewpoint of the ATL program execution environment, and thus the viewpoint differs from that which is in this chapter. The TP5/TP6 tester processor supported by the hybrid controller is equipped with a general-purpose microprocessor, and keeps upper compatibility by emulating the old tester processor on the microprocessor. Therefore, the ATL program execution environment is not different from the contents of the test system specifications.

The tester processor runs under VxWorks, a real-time multi-task operating system. The host system, however, runs under Solaris, which is an implementation of UNIX. The sections in this chapter explains the following items regarding these systems.

- Tester Processor Configuration
- ADVANsite Network
- VxWorks configuration
- System Tasks
- File system
- Host system access and security

**Tip**

System programming requires an understanding of advanced programming techniques and a full understanding of the contents of this chapter and the VxWorks system. If you attempt to use programs created without a fully understanding them, there may be serious adverse effects on the test system.

➔ For more information on VxWorks, refer to "[VxWorks Programmer's Guide](#)".

8. 2 Tester Processor Configuration

The TP5/TP6 tester processor is a unit equipped with a PowerPC microprocessor as the CPU and its configuration is shown in [Table 8-1](#).

Table 8-1 Tester Processor Configuration (TP5/TP6)

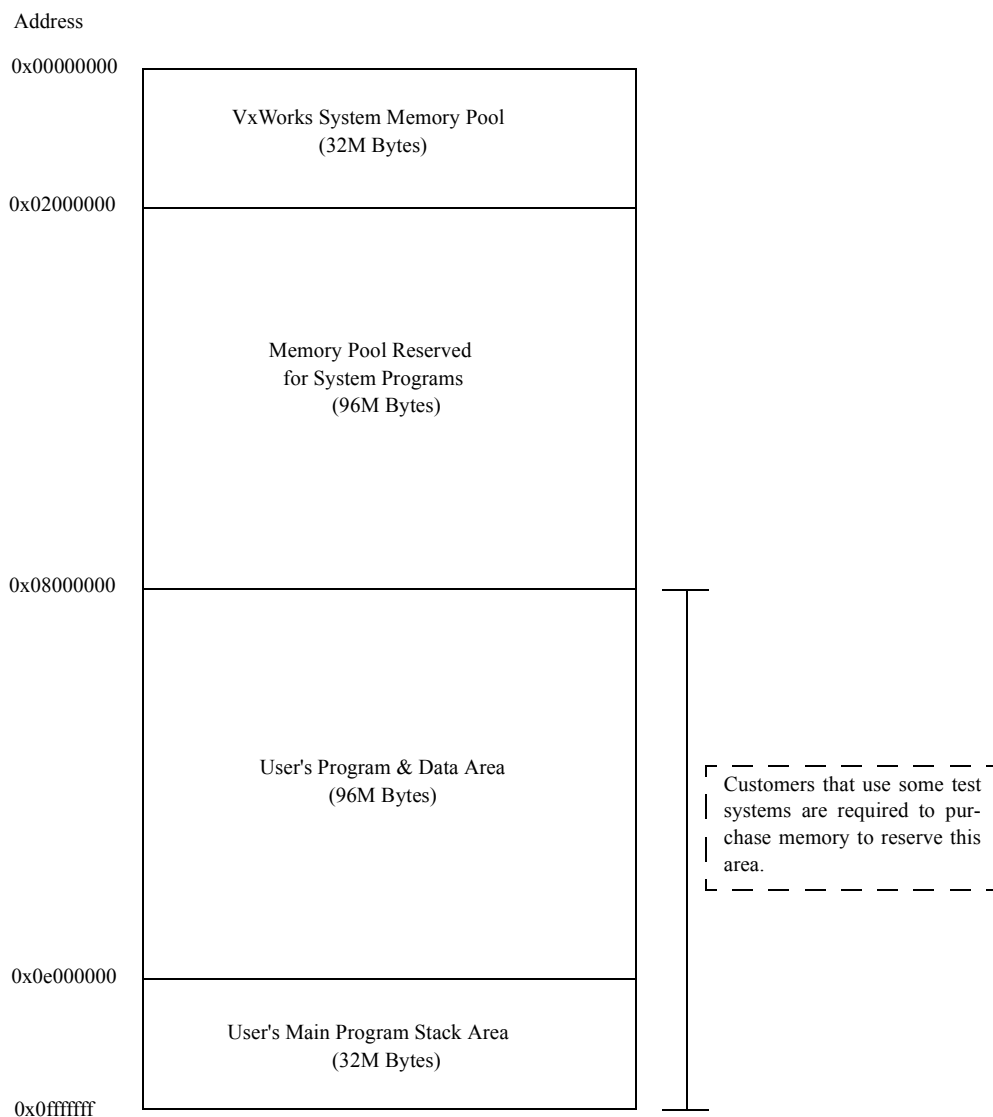
Item	Configuration
CPU	PowerPC
Main Memory	256 MB(128 MB x 2) ^{*1} (128 MB for system programs + 128 MB for user programs)
Interfaces	PCI: 1, Ethernet: 1(100BASE-TX), Serial: 1+2, GPIB: 1
Operating System	VxWorks
Tester Driver/Controller	ASX/U-51

^{*1} Some test systems require additional memory because the standard memory size is 128 MB.

➔ For more information, refer to [1. 2. 1 "Required Hardware" on page 1-3](#).

Main Memory

The tester processor has 256 MB of main memory. The layout of memory is shown in [Figure 8-1](#). VxWorks configured for the test processor does not perform memory management using virtual storage and access protection which are the MMU (Memory Management Unit) functions of the test processor. Therefore, logical addresses pointed to by C program pointers are the same as physical addresses, and the address space of the tester processor is shared by all programs including the system programs that operate on the tester processor.

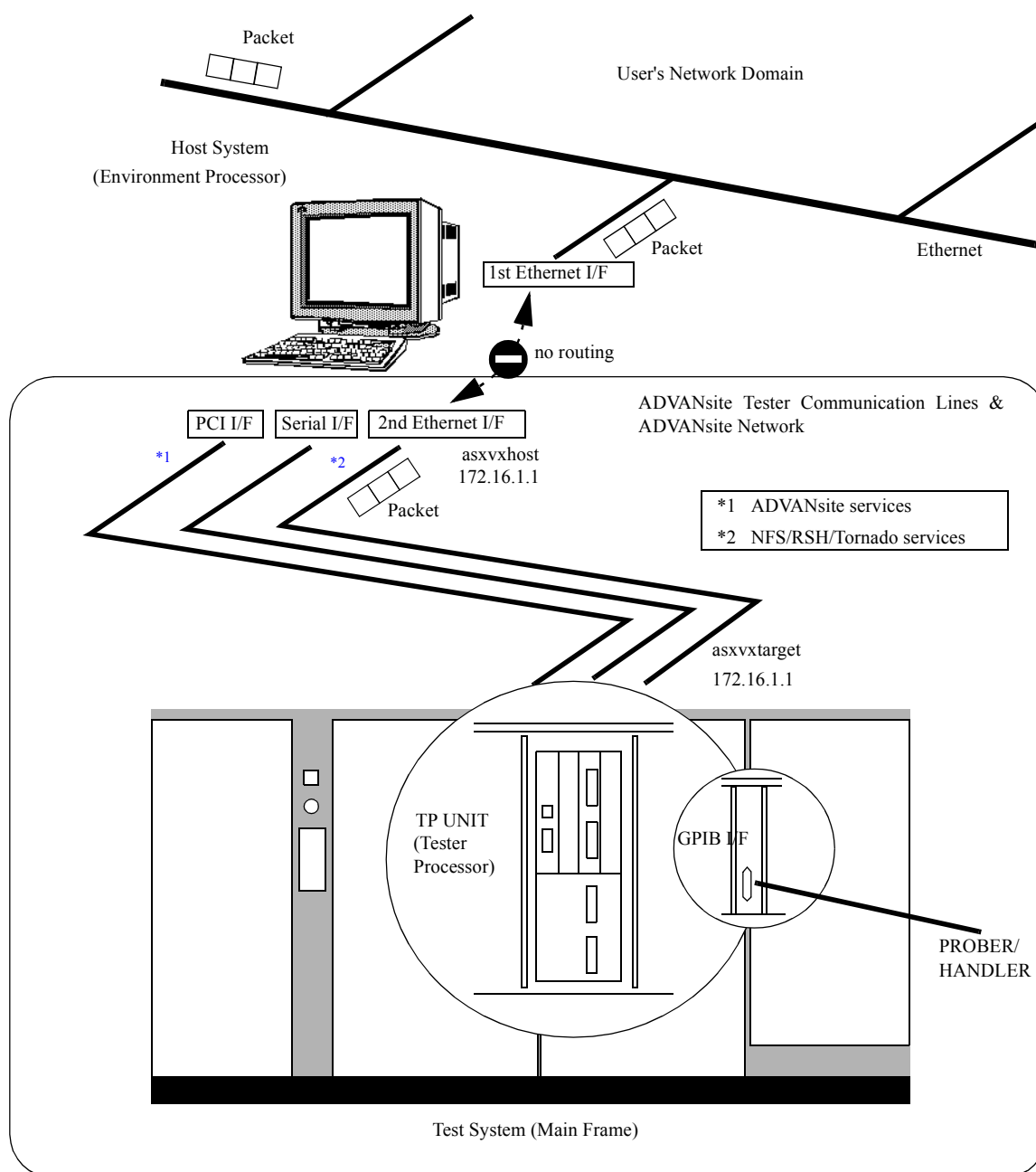
Figure 8-1 Tester Processor Memory Layout**Warning**

If a C program illegally accesses^{*1} the memory area of the system, the system program may be corrupted. In this case, an unexpected operation such as system halt will occur. Carefully evaluate and verify that a program operates correctly before using it.

**1 Regardless of the programmer's intention, illegal access to the system memory area results in a system conflict. An illegal access causes an immediate system failure in some cases and not in other cases.*

Interfaces

The user can program for the interfaces of the test processor listed in [Table 8-1](#). The outline and use of each interface is described below. (refer to [Figure 8-2](#).)

Figure 8-2 Interconnection of Tester Processor and Host System

- PCI I/F

The PCI interface is a bus interface used by the ASX/U-51 system software to provide the ADVANsite environment. This interface is used to connect the tester processor and host system. In a C program for the hybrid controller, I/O operations on the VKT and ASX file device are issued through this interface.

- Ethernet interface

The Ethernet interface connects the tester processor and host system as a local network. The hybrid controller uses NFS, RSH, and other host system services. The host system uses the tester processor services with the Tornado tool. Since these services are provided through a network, the interface is very important for the hybrid controller.

➔ For more information, refer to [8. 3 "ADVANsite Network" on page 8-6](#).

- Serial interface

The tester processor has three different serial interfaces.

Two of them are reserved for use by the system. The user can use the third serial interface. In a user C program, data can be read from and written to this serial interface by opening the device that corresponds to it.

➔ For more information on how to use the serial I/F, refer to [Appendix 1. "Using the Serial Interface" on page A1-1](#).

- GPIB I/F

The test system has a GPIB interface connected to the system bus for controlling a handler or prober. This interface is not mounted on the tester processor, but is included here for convenience because the user can access the interface from a C program.

➔ For the GPIB interface specifications, refer to the specifications of each test system.

➔ For more information on the C language programming interface for the GPIB interface, refer to [ASX/U-51 Hybrid Controller Reference Manual\(No. : 8409600\)](#).

8. 3 ADVANsite Network

The ADVANsite network is a local area network which links a test system and a host system using Ethernet. This local area network is provided with an ADVANsite test system supported by the hybrid controller. A host system compatible with the ADVANsite network has the user network interface and the ADVANsite network interface. (refer to [Figure 8-2](#).) The 1st network interface of the host system is used to connect to the user network and the 2nd network interface is used to configure an ADVANsite network.

The user can include a host system in the user network by assigning the network parameters to the 1st network interface, such as host name and IP address given by the network manager. Usually, the network parameters for the 1st network interface are set to the specified values by the installer when installing the basic OS (Solaris). The host name specified when the software is installed becomes the host name of the 1st network interface (defined in the `/etc/hostname.interface` file). The host name of the 1st network interface becomes the host name of the host system in the user network.

For the 2nd network interface of the host system, the user assigns the host name `asxvxhost` and IP address `172.16.1.10`^{*1} (the same settings as the factory settings) when installing the basic OS (Solaris). The tester processor is assigned the preset host name `asxvxtarget` and IP address `172.16.1.1`. These settings are common to all ADVANsite networks. The tester processor is connected to the host system through Ethernet as a component of the ADVANsite network. The host names `asxvxhost` and `asxvxtarget` become the host names of the host system and tester processor in the ADVANsite network.

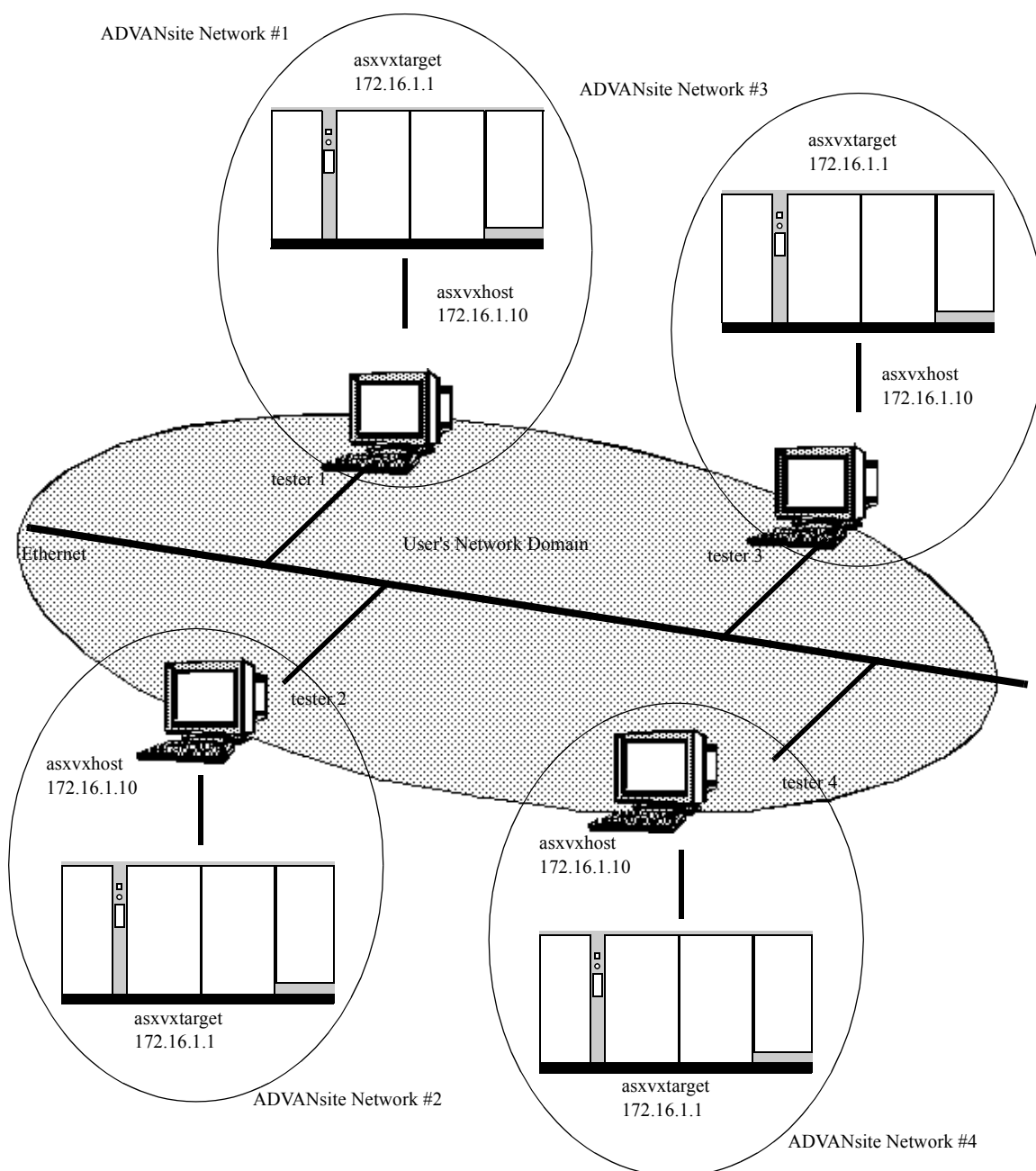
^{*1} The network address `172.16.0.0` is one of the addresses conventionally used for constructing a closed network environment. In some cases, users' network environment have not been set like this type of setting. In that case, use appropriate settings according to the user environment.

➔ For more information on how to set the network, refer to [ADVANsite Test System Software Installation/System Start User's Manual VOL1 Basic OS Installation\(No. : 8227728\)](#).

Each ADVANsite network is configured as a closed network so that its address will not collide with the address of another ADVANsite network. A closed network is a network in which the transfer of IP packets to/from other networks is not performed. A closed network can be configured by setting all systems with multiple network interfaces to be multi-home hosts^{*1}. In an ADVANsite network, the host system is configured as a multi-home host. This enhances the security of the ADVANsite environment because access from an external network to the ADVANsite network is disabled. Since VxWorks installed in a tester processor does not require a password for login, this mechanism is especially important for ensuring the security of the ADVANsite environment.

^{*1} A multi-home host is a machine which has several network interfaces, but does not execute a routing protocol nor transfer IP packets. Solaris allows a system to be a multi-home host by creating an `/etc/notrouter` file.

[Figure 8-3](#) shows an example of connecting four ADVANsite test systems, each of which is connected to the user network.

Figure 8-3 User Network and ADVANsite Networks

tester1, tester2, tester3, and tester4 are the host system names in the user network. Each of the host systems is connected to ADVANsite network #1, #2, #3 and #4, respectively.

Because each ADVANsite network is closed, tester1 cannot access any of ADVANsite Network #2, #3, and #4. tester1 can access only ADVANsite Network #1.

If the rlogin asxvxtarget command is executed at tester1, a remote login request is issued to asxvxtarget of ADVANsite network #1 but not to asxvxtarget of another ADVANsite network.

The opposite is also true. The asxvxtarget of ADVANsite network #1 cannot access tester2, tester3, or tester4. Since the host name tester1 does not exist in ADVANsite network #1, asxvxhost must be used as the host name to access the host system from asxvxtarget.

Tip

Figure 8-2 and Figure 8-3 show a minimum configuration network, or a network consisting of two nodes, a host system and tester processor. This network configuration may change when optional hardware is added.

The ADVANsite network is an open IP network that is not limited to specific uses. VxWorks has an internet protocol based on BSD 4.4 TCP/IP and a socket. The user can use them for programming.

➔ For more information on network programming, refer to ["VxWorks Network Programmer's Guide"](#).

The hybrid controller uses the ADVANsite network in the following cases.

- File I/O from a C program
For file input and output by the user's C program, the tester processor accesses files in the host system using the NFS service. Additionally, the NFS is usually used when the controller transfers a C program in the host system to the tester processor. The NFS is a service implemented on the Internet protocol, and is provided through the ADVANsite network.
 - Using Tornado
When a C program is debugged, a Tornado tool (for example, the CrossWind debugger) running on the host system communicates with the tester processor through an ADVANsite network.
- ➔ For more information on the file system for the tester processor, refer to [8. 5 "I/O System" on page 8-15](#).
- ➔ For more information on the characteristics peculiar to VxWorks implementation concerning the security for the tester processor to access the host system, refer to [8. 6 "File System and Security" on page 8-19](#).

8. 4 Implementation of VxWorks

This section explains the configuration of VxWorks, which is the operating system for the tester processor. This section is necessary to understand VxWorks and to create programs that use the VxWorks interface in the hybrid controller.

Tip

A program using the VxWorks-specific system interface is not portable. Advantest does not guarantee that VxWorks will be used as the operating system for future tester processors. For these reasons, use the VxWorks POSIX-compatible system interface for programming rather than the VxWorks-specific system interface.

8. 4. 1 VxWorks Configuration

VxWorks configured for tester processor contains the components listed in [Table 8-2](#). This table does not list all VxWorks components. It lists only those available to the user.

Table 8-2 VxWorks Configuration

Macro(Configuration Keyword)	Description
INCLUDE_ANSI_ALL INCLUDE_ANSI_ASSERT INCLUDE_ANSI_CTYPE INCLUDE_ANSI_LOCALE INCLUDE_ANSI_MATH INCLUDE_ANSI_STDIO INCLUDE_ANSI_STDLIB INCLUDE_ANSI_STRING INCLUDE_ANSI_TIME	Complete ANSI C library functions ANSI-C assert library functionality ANSI-C ctype library functionality ANSI-C locale library functionality ANSI-C math library functionality ANSI-C stdio library functionality ANSI-C stdlib library functionality ANSI-C string library functionality ANSI-C time library functionality
INCLUDE_FLOATING_POINT INCLUDE_FORMATTED_IO INCLUDE_IO_SYSTEM INCLUDE_PIPES INCLUDE_SELECT	Floating-point I/O Formatted I/O I/O system package Pipe driver select() facility
INCLUDE_SEM_BINARY INCLUDE_SEM_MUTEX INCLUDE_SEM_COUNT INGINCLUDE_MSG_Q INCLUDE_SIGNALS INCLUDE_STDIO INCLUDE_NETWORK INCLUDE_NFS INCLUDE_RLOGIN	Binary semaphore support Mutual exclusion semaphore support Counting semaphore support Message queue support Software signal facilities Standard buffered I/O package Network subsystem code Network File System(NFS) Remote login with rlogin

Macro(Configuration Keyword)	Description
INCLUDE_SHELL INCLUDE_SHOW_ROUT INESINCLUDE_SPY INCLUDE_TIMEX INCLUDE_WDB	C-expression interpreter (target shell) Show routines for system facilities Task activity monitor Function execution timer Target agent for Tornado target server
INCLUDE_CPLUS INCLUDE_CPLUS_IOSTREAMS INCLUDE_CPLUS_STL INCLUDE_CPLUS_STRING INCLUDE_CPLUS_STRING_IO INCLUDE_CPLUS_COMPLEX INCLUDE_CPLUS_COMPLEX_IO INCLUDE_CPLUS_IOSTREAMS_FULL	Bundled C++ support Basic iostreams classes Standard template library core String class I/O for string class Complex number class I/O for complex number class Full components of iostreams

The ASX/U-51 system software has no restrictions on using the VxWorks interface, excluding some exceptions. However, a program using this interface might not have source-level compatibility in hybrid controller in the future. The VxWorks-specific interface should only be used under the understanding that the user assumes full responsibility for its use.

Tip

The Tornado2 software bundled in the ASX/U-51 does not include the Board Support Package (BSP). The user should avoid BSP-dependent programming.

8. 4. 2 Multitasking

The VxWorks multitasking function enables to divide major program functions into small function blocks and to handle those functions by assigning any of those blocks to any of multiple tasks and linking each other. This section provides limitations and notes on the use of the VxWorks multitasking function.

Task Scheduling

To minimize the overhead in a task context switch, VxWorks is configured to use preemptive priority scheduling only. The user must not change the scheduling method (kernelTimeSlice()).

Task class

The VxWorks system has two levels of context: kernel level context (kernel context) and task level context (task context). For the VxWorks kernel, a task is just an entity containing the execution status of each program, such as priority, program counter and stack. The kernel processes task scheduling and kernel requests from programs, but ignores what individual tasks are doing. The ASX/U-51 system program, however, is not a component of the VxWorks kernel, but is implemented as some resident tasks on VxWorks. The user can also generate a task using the VxWorks function. However, if system tasks and user tasks have no order, the system program cannot control the test system.

ASX/U-51 adopts an extended VxWorks task concept and the task classes are classified according to the priority. [Table 8-3](#) defines task classes and their priority range.

Table 8-3 Task Classes and Range of Priority

Task Class	Range of Priority
System Class	0-99
Neutral Class	100-109
User Class	110-256

System-class tasks are referred to as system tasks, and user-class tasks are referred to as user tasks.

The neutral class is reserved for future extension; current system ignore tasks of this class. This specification may be changed in the future. Therefore, this class must be used only for debugging purposes.

A task of the system class or neutral class must not be generated from a user task. The user must not delete a system task, change its properties or execution status, or perform any other operations that might hinder the execution of the system task.

 Tip

If a neutral class task depends on a module specified by the /HYBRID utility program, the hybrid controller is disabled because the module cannot be unloaded. For the hybrid controller, make sure that a task of this class does not depend on a module specified by

 Limitation

If the user generates a system-class task^{*1} or performs any operation prohibited for a system task, the test system is disabled.

**1 When a system task is created from a VxWorks target shell or target agent.*

8. 4. 3 System Tasks

[Table 8-4](#) provides an outline of the system tasks running on the tester processor.

Table 8-4 System Tasks

Task Name	Overview
tExcTask	VxWorks system task: Executes exception processing at the task level.
tLogTask	VxWorks system task: Provides the logLib function.
tShell	VxWorks system task: Provides the target shell function.

Task Name	Overview
tRlogind tRlogInTask tRlogOutTask	VxWorks system task: Provides the rlogin function.
tWdbTask	VxWorks system task: Tornado target agent task
tpRun tpReRun	ASX/U-51 system task: Initial back end of the controller for the host system
tNetTask	VxWorks system task: Supports the task-level network function.
TP5erlog	ASX/U-51 system task: Monitors for errors.
TP5intr	ASX/U-51 system task: Processes interrupts from the test system.
TP5ifulsh	ASX/U-51 system task: Processes I/O requests from the controller.
TP5ctrlc	ASX/U-51 system task: Processes software interrupts of the controller.
TP5emerg	ASX/U-51 system task: Detects a tester processor error.
TP5evsrv	ASX/U-51 system task: Monitors for an asynchronous event.
TP5evcln	ASX/U-51 system task: Processes the request of the user program which has to be processed with the higher priority.
tSymSync	VxWorks system task: Synchronizes symbols with the Tornado target server.
TP5evcln	ASX/U-51 system task: Front end for asynchronous event processing
TP5main	ASX/U-51 system task: Core program for controlling the test system
tPortmapd	VxWorks system task: Provides the RPC portmap service for NFS and other functions.

8. 4. 4 Limitations: Restricted VxWorks Functions

This section explains the VxWorks functions that the user should not or cannot use. Most of these functions are used for system booting or are dependent on the system architecture or implementation. Usually, the user does not need to pay any attention to these functions.

The VxWorks program interfaces are compiled into a library named xxxLib (xxx: function name) for each function. Due to limitations common to all libraries, the user cannot use VxWorks functions whose names contain "Init" (for example, stdioInit()), functions that belong to libraries whose names contain "Arch" (for example, intArchLib), or device driver functions.

[Table 8-5](#) lists other restricted functions for each library. (XXX in the table matches any character string)

Table 8-5 Restricted VxWorks Functions

Library	Function Not Available
arpLib	All functions

Library	Function Not Available
bootLib	bootParamsPrompt()
cacheLib	All functions
clockLib	clock_setres(), clock_settime()
endLib	All functions
etherLib, etherMultiLib	All functions
excLib	All functions
fppLib	All functions
hostLib	hostAdd(), hostDelete()
ifLib	ifXXXSet()
intLib	All functions
ioLib	ioDefPathSet(), ioGlobalStdSet()
ipProto	All functions
kernelLib	kernelTimeSlice()
loadLib	loadModule(), loadModuleAt()
logLib	logFdSet(), logFdAdd(), logFdDelete()
memLib, memPartLib	memPartOptionsSet(), memPartRealloc(), memOptionsSet(), memPartCreate(), memPartAddToPool(), memPartAlloc(), memPartAlignedAlloc(), memPartFree(), memAddToPool()
moduleLib	moduleCreate(), moduleDelete(), moduleCreateHookAdd(), moduleCreateHookDelete()
muxLib	All functions
nfsLib	nfsAuthUnixPrompt(), nfsAuthUnixSet(), nfsIdSet()
netBufLib	All functions
rebootLib	rebootHookAdd()
remLib	remCurIdSet(), iam()
routeLib	All functions
selectLib	selWakeupXXX(), selNodeXXX()
shellLib	All functions
symLib	symTblCreate(), symTblDelete(), symAdd(), symRemove()
symSyncLib	All functions
sysLib	All functions, excluding sysClkRateGet(), sysModel(), sysBspRev(), sysMemTop(), sysPhysMemTop() and sysProcNumGet()
taskHookLib	All functions

Library	Function Not Available
tickLib	tickAnnounce(), tickSet()
usrLib	bootChange(), ld() ^{*1}
vxLib	All functions
wdLib	All functions

**1 This function can be used temporarily for debugging. However, since system memory resources are used by this function, system program operations may be affected. After debugging, be sure to unload any modules that have been loaded.*

8. 5 I/O System

This section explains the tester processor I/O system.

➔ For more information on the I/O system of VxWorks, refer to I/O System in ["VxWorks Programmer's Guide"](#).

8. 5. 1 Files and Devices

In VxWorks, an application can access an I/O device by opening a named file. Low-level I/O in VxWorks conforms to the POSIX semantics. More specifically, a file is opened by `open()` and the actual I/O device is accessed by applying required I/O operations (for example, `read()`, `write()`, `close()`, `ioctl()`) to the file descriptor obtained from the `open()` call.

I/O devices are classified into two types of devices: One is unstructured device such as the serial I/O channel (i.e. the terminals). Another is structured device such as the file systems. Individual devices have unique names. A file name is specified with a character string. A file name for an unstructured device is the device name itself. A file name for a file system device is the device name followed by the path name that specifies the file within the device.

For example, consider the following two file names. Suppose that the system has a device

`/tyCo/0` `asxvxhost:/foo/var`

The file name `"/tyCo/0"` refers to the serial channel device `"/tyCo/0"` itself. The file name `"asxvxhost:/foo/var"` refers to the file `"/foo/var"` on the file system device `"asxvx-host:"`.

Likewise, the device is specified explicitly by a file name in VxWorks. The I/O system searches for a device with a name that matches at least an initial substring of the specified file name to determine the device corresponding to the file name. If there are no such devices, the default device is used. For the tester processor, the default device is a device on the current default path that can be obtained by `getcwd()`.

The tester processor does not have a local file system device. Instead, it accesses the file system on the host system through the PCI bus interface and network. The file system devices implemented on the tester processor are the ASX file device, NFS device, and non-NFS network device.

The rest of this section explains these file system devices, and VKT input and output.

➔ For more information on the tester processor serial interface I/O, refer to [Appendix 1. "Using the Serial Interface"](#) on page A1-1.

8. 5. 2 ASX File Device

This device accesses a file on a host through the PCI bus interface. The device name `"asxFile:"` is assigned to this device. The ASX file device supports `open()`, `close()`, `read()`, `write()`, `lseek()`, `unlink()`, and `fstat()` functions.

Opening a file

The file name of an ASX file device is specified by a character string of up to 256 characters, including a terminating null character. If the file name contains a path specification, it is interpreted as follows: If the portion of the file name after the device specification begins with a slash '/', The file name is assumed to be an absolute pathname in the host file system. Otherwise, the file name is assumed to be a relative pathname from the current customer directory (This is generally the directory where the controller was started). Otherwise, if the file name does not contain a path specification, the specified file is opened in the current customer directory.

For example:

```
fd = open("asxFile:sample.dat", O_CREAT|O_WRONLY, 0);
```

If the file name shown above is specified, the sample.dat file located in the current customer directories opened for writing. And,

```
fd = open("asxFile:/export/home/someone/sample.dat", O_RDONLY, 0);
```

If the file name shown above is specified, the /export/home/someone/sample.dat file is opened for reading.

The third argument mode specified by the open() function is ignored. When the O_CREAT flag is specified, the access permission of the file depends on the value of umask(2) of the controller process.

Closing a file

An opened file for the ASX file device is closed automatically when a program terminates.

8. 5. 3 NFS Devices

A network file system (NFS) device accesses a file system in the host system based on the NFS protocol.

When the controller is invoked, all file systems exported to the tester processor are NFS-mounted on the processor. The device names of the mounted file systems in the tester processor are set to the file system names (paths) in the host system. For example, if the file system /export/home is exported from the host system to the tester processor, the device name of the file system becomes "/export/home."

➔ For more information on the method of exporting a file system from the host system (Solaris), refer to share(1M) and share_nfs(1M) in the Solaris online manual.

The NFS file systems mounted on the tester processor are unmounted when the controller terminates. If the file system export setting in the host system is changed when the controller is active, terminate the controller and then activate it again so that the new setting is reflected in the tester processor.

 Tip

If a device is not specified in the module name setting, the hybrid controller uses the NFS device to transfer the user's C program to the tester processor. In this situation, the following environmental conditions must be satisfied:

- *The controller should be invoked in a directory in a local file system of the host system.*
- *The file system containing the directory where the controller is being invoked should be exported to the tester processor.*

The hybrid controller does not operate if it is activated in the directory where the file system for another system is NFS mounted. This is because the NFS mount pointer on the host cannot be exported to the tester processor.

NFS Access Authentication

UNIX Authentication (AUTH_UNIX or AUTH_SYS) is used to authenticate access to NFS file systems. In this authentication, the tester processor uses the user ID and the group ID.^{*1} The user cannot set the NFS authentication parameter using `nfsAuthUnixPrompt()`, `nfsAuthUnixSet()`, and `nfsIdSet()`.

➔ For more information, refer to [8. 6 "File System and Security" on page 8-19](#).

**1 If the controller is started by a root user, an NFS file system cannot be accessed with root authority unless the NFS permits root access on the host side.*

8. 5. 4 Non-NFS Network Devices

A non-NFS network device accesses a file in the host system using the RSH protocol. The device name of a non-NFS network device is `"asxvxhost:"`.

The tester processor can perform RSH access with the name of the user who invoked the controller. No password is used for access authentication. Therefore, when the user accesses a file through this device, the `.rhosts` file in the user's home directory must have an entry for the tester processor (`asxvxtarget`).

➔ For more information on how to make this setting in the host system, refer to `rsd(1M)` and `rho-
sts(4)` in the Solaris online manual.

Caution

If the file of a non-NFS network device is opened, the entire file is copied into the system area of the tester processor. Consequently, if a large file is opened, memory space may be insufficient and the system may be disabled.

➔ For more information, refer to Non-NFS Network Devices described in ["VxWorks Programmer's Guide"](#).

8. 5. 5 Controller Terminal (VKT) I/O

VxWorks standard I/O are directed to the terminal device used for VxWorks logon, but not to the controller terminal in the tester processor. To perform I/O operations using the controller terminal, the file descriptor `Stc_consoleFd` or its corresponding file stream `Stc_consoleFp` is used. These are declared in the header file `AT/tpio.h` and also are defined as the macro names `tpcoFd` and `tpcoFp`. The file descriptor or file stream performs I/O operations using a dedicated device which corresponds to the controller terminal.

The file descriptor `Stc_consoleFd` and the file stream `Stc_consoleFp` are opened only once when the controller is invoked. The file stream `Stc_consoleFp` line buffers the input, and does not buffer the output. The I/O device of the controller terminal does not support polling using the `select()` function. As high-level interfaces for controller terminal input and output, the hybrid controller provides functions prefixed with `AT_` and have the same format as the C standard I/O functions like `AT_printf()` and `AT_scanf()`.

8. 5. 6 Default Path

In the tester processor, the VxWorks default path^{*1} is set to the path on the NFS file system device corresponding to the directory used to start the controller.

If the controller is invoked in a directory where a file system of a different machine are NFS-mounted, the default path is set to the RSH file system `asvxvzhost:` because the tester processor cannot access the directory.

^{*1} *Current default path name obtained by `getcwd()`*

8. 6 File System and Security

As mentioned in [8. 5 "I/O System" on page 8-15](#), the tester processor has NFS and RSH file systems that are accessed on the basis of the respective authentication protocols. This section explains the particular features of the access method for the implementation under VxWorks.

The first thing to note about VxWorks is that no password is necessary for login. As a result, the tester processor is defenseless against external illegal access. However, since IP routing between network segments is disabled in the host system, no machines other than the host system can perform remote login to the tester processor through the 1st network interface of the host system. Therefore, login to the tester processor is restricted to only those who can login to the host system or who can change the connections to the external interfaces of the tester processor.

Once logged in to the tester processor, the user can access file systems in the host system according to the authentication parameter settings to access each file system at that time. Unlike UNIX, VxWorks does not have a consistent user concept, but instead has settings in individual file systems for their respective authentication protocols. In the NFS file system, the authentication method, called AUTH_UNIX or AUTH_SYS, using user IDs is used for accessing. In the RSH file system, user names are used for accessing. The authentication parameters for these file systems are set by the controller and users are not allowed to set them. [Table 8-6](#) and [Table 8-7](#) list the authentication parameters set by the controller for each of the NFS and RSH file systems.

Table 8-6 NFS Authentication Parameter Settings

Tester Processor Status	NFS Setting (uid: user ID, gid: group ID)
Controller not active	uid = 998(asxvx) gid = 999(asx)
Controller active	uid = Real user ID of the user who started the controller gid = Real group ID of the user who started the controller

Table 8-7 RSH Authentication Parameter Settings

Tester Processor Status	RSH Setting
Controller not active	user = asxvx
Controller active	user = Real user name of the controller process (test)

As mentioned before, login to the tester processor is permitted not only for the user who activated the controller, but also for any other user who has an account in the host system. If a user logs in to the tester processor when someone is using the controller, the user can access file systems with the authority of the user who is using the controller. To protect personal files, start the controller using a common account that is different from a personal account. This will prevent personal files from being accessed from the tester processor.

 Tip

Files of the user who is using the controller can be accessed from any user logged in to the tester processor. As a result, if the controller is activated with a personal account, the user's privacy may be violated.

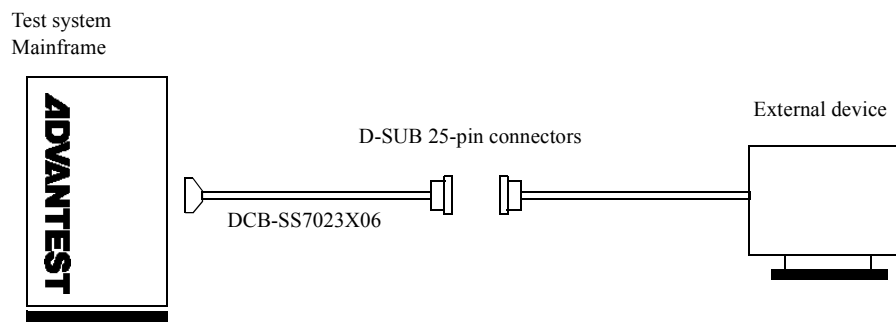
Appendix 1. Using the Serial Interface

Serial ports conforming to RS-232C are installed in the tester processor. This appendix explains how to use the ports.

A1. 1 Connecting External Devices

To connect an external device to the tester processor with serial interface, use serial cable DCB-SS7023X06, which is provided as a test system accessory, as shown in [Figure A 1-1](#). One of the cable connectors is a D-SUB 25-pin (female) connector. Connect the serial cable from the external device to this connector.

Figure A 1-1 Connecting an External Device to the Tester Processor with Serial Interface



The test system serial ports are located on the board top of the TP5 or TP6 unit in the mainframe. The serial ports that can be used by the user are as shown in [Table A 1-1](#).

Table A 1-1 Usable Serial Port

Tester Processor	Usable Serial Port (connector name)
TP5	J21
TP6	J7

➔ For the tester units and connector locations, refer to the tester maintenance manuals of each test system.

Connect the connector on the other end of the serial cable DCB-SS7023X06 to this serial port. Take care to connect to the correct port because there are two ports of the same shape.

 Caution

Be sure to route the test system serial cable outside the mainframe before connecting it to the serial cable of the external device.

If the external device serial cable is routed into the mainframe, noise can be introduced into the mainframe through the serial cable, causing the system to malfunction.

A1. 2 Driver Specifications

Table A-2 lists the specifications for the tester processor serial driver.

Table A 1-2 Serial Driver Specifications

Item	Specification
Device name	/tyRo/2
Baud rate	9600 baud
Data size	7 bit
Parity bit	Even parity
Stop bit	1 bit
Hardware flow control	DTR/RTS

A1. 3 Programming

For I/O access to the serial device, as with I/O access to other devices, first open the target device with `open()`, and then apply `read()`, `write()`, etc. for the obtained file descriptor.

```
int fd;

if((fd = open("/tyRo/2", O_RDWR, 0) == -1){
    AT_printf("Error: open\n");
    return(-1);
}

:
: /* applying read/write operations to fd */
:

close(fd);
```

Make sure to close the opened file descriptor when it is no longer needed.

List of Figures

Figure 2-1	ATL-Separated Method Operation Conceptual Diagram	2-3
Figure 3-1	ATL Program Linkage at Execution (ATL_GOSUB Command)	3-9
Figure 3-2	Branching to Internal Subroutines and Return	3-10
Figure 3-3	Branching to Internal Subroutine and Return	3-11
Figure 3-4	ATL-mixed Program Example	3-13
Figure 3-5	Executing Embedded ATL Subroutines and System Operation	3-17
Figure 3-6	Execution and Return of an Embedded ATL Subroutine (1)	3-18
Figure 3-7	Execution and Return of an Embedded ATL Subroutine (2)	3-18
Figure 3-8	Execution and Return of an Embedded ATL Subroutine (3)	3-19
Figure 3-9	Synchronous Event Handler Execution (1)	3-31
Figure 3-10	Synchronous Event Handler Execution (2)	3-32
Figure 3-11	Tracing and Analyzing the Stack Frame	3-39
Figure 3-12	Operations in MANUAL Mode	3-44
Figure 3-13	Operations in the AUTO Mode	3-45
Figure 3-14	Tests in the MANUAL and MANUAL RECYCLE Modes	3-46
Figure 4-1	Configuration of the Setting Screen	4-5
Figure 4-2	Setting Screen of Station Number	4-5
Figure 4-3	Setting Screen of Current Setting	4-6
Figure 4-4	Setting Screen of Operating Mode	4-6
Figure 4-5	Setting Screen of Module Name	4-6
Figure 4-6	Setting Screens When All Stations Are Specified	4-7
Figure 7-1	Link Editing for Object Files	7-7
Figure 7-2	Makefile Sample	7-12
Figure 7-3	Sample Makefile (Cont'd)	7-13
Figure 7-4	Dialog Box Asking Whether or Not to Run the Registry Daemon ..	7-16
Figure 7-5	Launcher Main Window	7-18
Figure 7-6	Settings in the Create Target Server Form	7-20
Figure 7-7	CrossWind Main Window	7-23
Figure 7-8	Open Source List Dialog Box	7-24
Figure 7-9	Attach Task Dialog Box	7-25
Figure 7-10	Button Bar	7-27
Figure 7-11	Browser Main Window	7-30
Figure 7-12	Task List Folder in the Data Panel	7-31
Figure 8-1	Tester Processor Memory Layout	8-3
Figure 8-2	Interconnection of Tester Processor and Host System	8-4

Figure 8-3	User Network and ADVANsite Networks	8-7
Figure A 1-1	Connecting an External Device to the Tester Processor with Serial Interface	A1-1

List of Tables

Table C-1	Notation	Convention-1
Table C-2	Terms with Specific Meanings or Purposes	Convention-1
Table 1-1	Required Hardware Configuration	1-3
Table 1-2	Required XATL/U-51 Cross Compiler	1-4
Table 2-1	Program Development Procedure with the ATL-Separated Method	2-2
Table 2-2	Debugging Environment of the Hybrid Controller	2-8
Table 3-1	Configuration of the TUsrMain Task	3-4
Table 3-2	Meanings of Termination Statuses	3-5
Table 3-3	Comparing the ATL-separated Method and the ATL-mixed Method	3-7
Table 3-4	ATL_exec() Command	3-10
Table 3-5	ATL Interface for the ATL-mixed Program	3-12
Table 3-6	Integers Defining Bit Width	3-20
Table 3-7	Passing Arguments to an ATL Subroutine	3-21
Table 3-8	Real Argument Types in an ATL Subroutine	3-23
Table 3-9	System Variables HBn	3-25
Table 3-10	Functions Used to Access System Variables	3-25
Table 3-11	ID Codes of System Variables	3-26
Table 3-12	Predefined Event Handlers	3-29
Table 3-13	Event Codes for Synchronous Events	3-30
Table 3-14	Default Event Handler (ATEV_DFL) Actions for Synchronous Events	3-30
Table 3-15	Operations After Terminating Synchronous Event Handler	3-31
Table 3-16	Event Codes for Asynchronous Events	3-33
Table 3-17	Default Event Handler Actions for Asynchronous Events	3-33
Table 3-18	Exceptions Detected by the Controller	3-36
Table 3-19	Exception Types and Messages	3-37
Table 3-20	I/O Functions for VKT	3-40
Table 7-1	Environment Variables Required to Run the C Compiler	7-2
Table 7-2	Major C Compiler Warning Options	7-3
Table 7-3	Major C Compiler Debug Option	7-3
Table 7-4	Major C Compiler Optimization Options	7-3
Table 7-5	XATL/U-51 Cross Compiler Options	7-4
Table 7-6	Default Compilation Options	7-4
Table 7-7	Major Linker Options	7-8
Table 7-8	Environment Variables Required for Executing Tornado	7-15
Table 7-9	Main Specification Formats for list Command	7-24

Table 7-10	Tasks Targeted for Debugged	7-26
Table 8-1	Tester Processor Configuration (TP5/TP6)	8-2
Table 8-2	VxWorks Configuration	8-9
Table 8-3	Task Classes and Range of Priority	8-11
Table 8-4	System Tasks	8-11
Table 8-5	Restricted VxWorks Functions	8-12
Table 8-6	NFS Authentication Parameter Settings	8-19
Table 8-7	RSH Authentication Parameter Settings	8-19
Table A 1-1	Usable Serial Port	A1-1
Table A 1-2	Serial Driver Specifications	A1-2